14 haftalık yol haritasında 12. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 12

Açgözlü Algoritmalar

Yerel En İyi Seçim, En Kısa Yol ve En Küçük Kapsayan Ağaç

MODÜL IV · GRAFLAR VE AÇGÖZLÜ STRATEJİ · H11–H12

> **HAFTANIN BÜYÜK SORUSU**
>
> *Her adımda o an en iyi görünen seçimi yapmak her zaman en iyi sonuca götürür mü?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 4
1.1 Öğrenme çıktıları 4
2. Geçen Haftadan ve AP-1'den Bağlantı 5
3. Kavramsal Anlatım: Yerel En İyi Seçim 5
3.1 Açgözlü paradigma: iki koşul 5
3.2 Para üstüne dönüş: kanonik sistem ve karşı örnek 6
3.3 Doğruluk kanıtlama teknikleri 6
3.4 Açgözlü ne zaman doğru, ne zaman yanlış? 7
3.5 Kaputun Altında: tembel silme, bit tamponu, navigasyon 7
4. Algoritmalar 9
4.1 Aralık çizelgeleme: en erken biten ilk 9
4.2 Aralık bölümleme: en az salon 12
4.3 Kesirli sırt çantası 14
4.4 Huffman kodlaması 16
4.5 Dijkstra: tek kaynaklı en kısa yollar 19
4.6 Birleşim–bul (ayrık kümeler) 23
4.7 Kruskal: en küçük kapsayan ağaç 26
4.8 Prim: ağacı tek köşeden büyütmek 28
5. Çözümlü Örnekler 31
5.1 Huffman ile kodlama ve çözme: bit tamponu (Örnek 6) 31
5.2 Negatif kenarda Dijkstra'nın yanlış sonucu (Örnek 9) 31
5.3 Açgözlü bir sezgisel: Welsh–Powell graf boyama (Örnek 13) 32
6. Haftanın Deneyi: Dizi mi, Öbek mi? 33
6.1 Soru ve düzen 33
6.2 (a) V ölçeklemesi 33
6.3 (b) Kesişim noktası: kuram ne kadar yanılır? 34
6.4 (c) Birleşim–bul: işlem başına izleme 35
6.5 Sınırlılıklar ve kendi ölçümünüz 35
7. Hata Avcısı 36
8. Dil Penceresi: Öncelik Kuyruğu, Sonsuz, Demetler, Bitler 37
9. Mühendis Gözüyle: Kanıt ya da Karşı Örnek 38
10. Mini-OBS 2 Adımı: v2.11 Sınav Programı 38
10.1 Gereksinimler 38
10.2 İpuçları 39

10.3 Kabul ölçütleri....39
10.4 Test senaryoları ve başlangıç noktası....40
11. Alıştırmalar ve Atölyeler....40
11.1 İzleme ve çıktı tahmini (A)....41
11.2 Karmaşıklık (B)....41
11.3 Değişmez yazma (C)....41
11.4 Hata bulma (D)....42
11.5 Parsons (E)....42
11.6 Kod yazma (F)....43
11.7 Diller arası okuma (G)....43
11.8 Atölye 1 · Açgözlü mü, değil mi? On problem kartı (gruplar, ≈ 30 dk)....44
11.9 Atölye 2 · Huffman ile kendi adınız ve bir paragraf (bireysel + eşli, ≈ 25 dk)....44
11.10 Atölye 3 · Kampüs haritası: Dijkstra ve fiber kablo (eşli, ≈ 30 dk)....45
12. Kendini Değerlendirme....45
13. İleri Bakış ve Merak Kutusu....45
14. Özet, Kavram Haritası ve Terim Sözlüğü....46
15. Kaynaklar....47
Ek A. Dil Karşılıkları....49
A.1 Örnek 1 · Para üstü....49
A.2 Örnek 2 · Aralık çizelgeleme....50
A.3 Örnek 3 · Aralık bölümleme....51
A.4 Örnek 4 · Kesirli sırt çantası....52
A.5 Örnek 5 · Huffman ağacı....53
A.6 Örnek 6 · Huffman kodlama ve çözme....54
A.7 Örnek 7 · Dijkstra (dizi)....55
A.8 Örnek 8 · Dijkstra (öbek)....56
A.9 Örnek 9 · Negatif kenar....57
A.10 Örnek 10 · Birleşim–bul....58
A.11 Örnek 11 · Kruskal....59
A.12 Örnek 12 · Prim....60
A.13 Örnek 13 · Welsh–Powell graf boyama....61

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Her adımda o an en iyi görünen seçimi yapmak her zaman en iyi sonuca götürür mü?

Açgözlü algoritma (greedy algorithm) çözümü adım adım kurar, her adımda o an **en iyi görünen** seçimi yapar ve kararını geri almaz. Geri izleme (H3) bütün seçenekleri dener, dinamik programlama (H13) bütün alt problemleri tabloya yazar; açgözlü tek bir yol izler. Bu cesaret bazen tam isabettir, bazen yanıltır. 3 paralı {1, 3, 4} sisteminde 6 birimlik para üstü için "her adımda sığan en büyük parayı al" kuralı 3 parça verir, oysa 2 parça yeter (Örnek 1). Aynı kural gündelik para sistemlerinde kusursuzdur. Yanıtı iki yönden arayacağız: açgözlünün **kanıtla** doğru olduğu problemler (aralık çizelgeleme, kesirli sırt çantası, Huffman, Dijkstra, en küçük kapsayan ağaç) ve **karşı örnekle** çürüttüklerimiz (para üstü, 0/1 sırt çantası, graf boyama).

Haftanın ortak grafı sekiz binalı kurgusal bir kampüstür (`girdiler/h12_kampus.txt`). Ana Kapı'dan her binaya en kısa yürüme yolları bir ağaç oluşturur (toplam 340 m), bütün binaları en az kabloyla bağlayan fiber ağı başka bir ağaçtır (toplam 280 m). İkisi de açgözlü algoritmayla bulunur ama farklı soruların yanıtıdır (Bölüm 4.5 ve 4.7).

> **GERÇEK DÜNYADAN · Yirmi dakikalık bir buluş ve bir dönem ödevi**
>
> Edsger W. Dijkstra, 2001'de yapılan ve Frana ile Misa'nın (2010) yayımladığı söyleşide en kısa yol algoritmasını bir sabah Amsterdam'da genç nişanlısıyla alışverişteyken, yorulup oturdukları bir kafe terasında yaklaşık yirmi dakikada tasarladığını anlatır; algoritmanın güzelliklerinden birini kâğıt kalem kullanmadan tasarlanmış olmasına bağlar. Söyleşideki "üç yıl sonra, 1959'da yayımlandı" ifadesine göre buluş 1956'dadır; Dijkstra aynı söyleşide 1956'yı diplomasını aldığı ve ARMAC bilgisayarının törenle açıldığı yıl olarak anar ve gösteri için matematikçi olmayanların da anlayabileceği bir problem (Rotterdam'dan Groningen'e en kısa yol) seçtiğini söyler. Üç sayfalık makale (Dijkstra, 1959) iki problem çözer: n düğümü en küçük toplam uzunlukla bağlayan ağaç ve iki düğüm arasındaki en kısa yol; bu haftanın iki büyük konusu tek makaledir.
>
> Huffman kodlamasının öyküsü ise bir sınıftan çıkar. Stix'e (1991) göre David A. Huffman 1951'de MIT'de Robert M. Fano'nun bilgi kuramı lisansüstü dersindeydi; öğrencilere dönem ödevi ile final sınavı arasında seçim hakkı verilmişti ve ödev en verimli ikili kodu bulmaktı. Huffman'ın yöntemi 1952'de yayımlandı (Huffman, 1952). Bu hafta yöntemi bir Ömer Seyfettin metnine uygulayıp sıkıştırma oranını entropiyle karşılaştıracağız (Örnek 6).

1.1 Öğrenme çıktıları

- **ÖÇ 12.1** Açgözlü stratejiyi tanımlar; açgözlü seçim özelliği ve en iyi alt yapı kavramlarını açıklar ve açgözlü yaklaşımın başarısız olduğu durumlar için karşı örnek kurar. (Bölüm 3, 5.3, Örnek 1, 13)
- **ÖÇ 12.2** Aralık çizelgeleme ve kesirli sırt çantası problemlerini açgözlü yöntemle çözer; doğruluğu değiş tokuş argümanıyla gerekçelendirir. (Bölüm 4.1–4.3, Örnek 2–4)
- **ÖÇ 12.3** Huffman kodlamasını öncelik kuyruğuyla gerçekler ve sıkıştırma oranını hesaplar. (Bölüm 4.4, 5.1, Örnek 5–6)
- **ÖÇ 12.4** Dijkstra algoritmasını öncelik kuyruğuyla gerçekler; negatif ağırlıklı kenarlarda neden başarısız olduğunu açıklar. (Bölüm 4.5, 5.2, 6, Örnek 7–9)
- **ÖÇ 12.5** En küçük kapsayan ağacı birleşim–bul destekli Kruskal algoritmasıyla bulur ve Prim algoritmasıyla karşılaştırır. (Bölüm 4.6–4.8, Örnek 10–12)

2. Geçen Haftadan ve AP-1'den Bağlantı

H11'in graf dosyası biçimine (`V E [yonlu|yonsuz]`, sonra `u v [w]`) bu hafta üçüncü sütun olarak **ağırlık** ekleniyor; komşu kaydı (hedef, ağırlık) çiftidir ve komşular yine artan numarayla dolaşılır. H11'in BFS'si ağırlıksız grafta en kısa yolu kuyruğa **eklenme sırası** sayesinde bulmuştu. Ağırlıklı grafta "önce yakındakiler" kuralı geçerlidir ama yakınlık ağırlık toplamıdır: FIFO kuyruk yerine **öncelik kuyruğu** (priority queue) gerekir. Bütün ağırlıklar 1 iken Dijkstra BFS'nin bulduğu uzaklıkları bulur ve köşeleri katman katman kesinleştirir; yalnız aynı katmandaki köşelerin sırası eşitlik kuralına bağlıdır.

H7'nin **öbeği** (heap) bu haftanın iş atıdır: Kahn algoritmasında (H11 Örnek 11) en küçük öbekle giriş derecesi 0 olan en küçük köşeyi seçmiştik; bu hafta aynı öbek (uzaklık, köşe), (bitiş, salon) ve (sıklık, düğüm) **demetlerini** tutacak. AP-1 H4'ün para üstü programı ise haftanın ilk açgözlü algoritmasıdır.

> TAHMİN ET · **Geri çağırma: dört soru (yanıtlar aşağıda)**
>
> - **S1 (H11).** `graf_ornek.txt` grafında BFS(0) ile 7 numaralı köşenin uzaklığı (kenar sayısı) kaçtır ve yol hangisidir?
> - **S2 (H11).** 10 kenarlı yönsüz grafın komşuluk listelerinde toplam kaç öğe vardır?
> - **S3 (H7).** 10 elemanlı bir en büyük öbeğe 97 eklenirse ve yeni eleman köke kadar çıkarsa kaç takas yapılır?
> - **S4 (AP-1 H4).** {1, 5, 10, 20, 50, 100, 200} sisteminde 387 birimlik para üstü açgözlü yöntemle kaç parçayla ödenir?

Yanıtlar. S1: 4 kenar, yol 0 → 1 → 3 → 5 → 7 (H11 Örnek 4). S2: 20 = 2E: yönsüz grafta her kenar iki listede görünür (H11 Örnek 2). S3: 3 takas = düzey sayısı − 1 (H7 Örnek 3); ekleme ve çıkarma O(log n)'dir, bu haftanın bütün öbek maliyetleri bu sayıdan gelir. S4: 8 parça: 200×1 + 100×1 + 50×1 + 20×1 + 10×1 + 5×1 + 1×2 (Örnek 1); bu sistemde açgözlü her tutar için en iyidir (Bölüm 3.2).

3. Kavramsal Anlatım: Yerel En İyi Seçim

3.1 Açgözlü paradigma: iki koşul

Açgözlü algoritma bir **seçim ölçütüyle** (greedy criterion) çalışır: "en erken biteni al", "en seyrek iki ağacı birleştir", "en yakın kesinleşmemiş köşeyi kesinleştir". Her seçim problemi daha küçük bir alt probleme indirir ve bir daha sorgulanmaz. En iyi sonuç için iki özellik birlikte gerekir (Cormen vd., 2022):

- **Açgözlü seçim özelliği** (greedy-choice property): ölçütün yaptığı ilk seçimi içeren **bir** en iyi çözüm vardır. Yani açgözlü ilk adımda "yanlış kapıya" girmez; en iyi çözümlerden en az biri o kapıdan geçer.
- **En iyi alt yapı** (optimal substructure): ilk seçimden sonra kalan alt problemin en iyi çözümü, seçimle birleşince bütün problemin en iyi çözümünü verir.

En iyi alt yapı tek başına yetmez: 0/1 sırt çantası ve para üstü de bu özelliğe sahiptir ve dinamik programlamayla (H13) çözülür. Açgözlüyü dinamik programlamadan ayıran, **açgözlü seçim özelliğidir**: alt problemlerden yalnız birini, seçimin bıraktığını çözmek yeter. Bu özellik ölçüte bağlıdır; aynı problemde bir ölçüt doğru, öteki yanlış olabilir (Bölüm 4.1). Bu yüzden her açgözlü algoritma ya **kanıtla** ya da en azından **karşı örnek aramayla** sınanır (Bölüm 9).

3.2 Para üstüne dönüş: kanonik sistem ve karşı örnek

c/H12_01_para_ustu.c · satır 29–40

```c
/* para[] azalan sıralı; her adımda sığan en büyük parayı alır.
 * Parça sayısını döndürür; ödenemezse -1. */
static int acgozlu(const int *para, int k, int tutar, int *adet)
{
    int parca = 0;
    for (int i = 0; i < k; i++) {
        adet[i] = tutar / para[i];       /* yerel en iyi seçim */
        tutar -= adet[i] * para[i];
        parca += adet[i];
    }
    return tutar == 0 ? parca : -1;
}
```

Yukarıdaki `acgozlu` fonksiyonu (Örnek 1, satır 29–40) AP-1 H4'teki para üstü programının ta kendisidir: para dizisi azalan sıralıdır ve her adımda `tutar / para[i]` kadar o paradan alınır. Örnek 1 bu kuralı **kaba kuvvetle**, yani her tutarın bütün kombinasyonlarını sayarak denetler: AP-1 sistemi {1, 5, 10, 20, 50, 100, 200} ve ABD sistemi {1, 5, 10, 25} için açgözlü 1–100 arası tutarların sırasıyla 100/100 ve 100/100 kadarında en iyidir; böyle sistemlere **kanonik** (canonical) denir. {1, 3, 4} sisteminde ise yalnız 76/100 tutarda en iyidir; ilk karşı örnek 6'dır, sonrakiler 10, 14, 18, 22, … . {1, 7, 10} sisteminde 14 için açgözlü 10×1 + 1×4 ile 5 parça verir, en iyi 7×2 ile 2'dir.

Neden? 6'nın 4 kombinasyonu denenince görülür: açgözlü ilk adımda 4'ü seçer ve kalan 2 yalnızca 1 + 1 ile ödenir; en iyi çözüm (3 + 3) 4'ü **hiç** kullanmaz. Açgözlü seçim özelliği bu sistemde yoktur: ilk seçimi (4) içeren hiçbir en iyi çözüm yoktur. En iyi alt yapı ise vardır (6'nın en iyi çözümü 3'ün en iyi çözümünü içerir); bu yüzden problem her sistemde H13'ün dinamik programlamasıyla çözülür.

3.3 Doğruluk kanıtlama teknikleri

Açgözlü bir algoritmanın doğruluğu çoğu zaman şu iki kalıptan biriyle gösterilir (Kleinberg & Tardos, 2006); en küçük kapsayan ağaçta üçüncü bir kalıp, kesim özelliği kullanılır:

- **Açgözlü önde kalır** (greedy stays ahead): açgözlünün kısmi çözümü her adımda herhangi bir en iyi çözümün aynı uzunluktaki kısmından “geride değildir” (ör. r'inci seçilen aralık, en iyi çözümün r'inci aralığından daha geç bitmez). Sonda açgözlü de en az o kadar iyidir. Dijkstra'nın değişmezi de bu türdendir (Bölüm 4.5).
- **Değiş tokuş argümanı** (exchange argument): bir en iyi çözüm alınır ve açgözlünün seçimiyle uyuşmayan bir parçası, çözüm kötüleşmeden açgözlünün seçimiyle **değiştirilir**; adım adım en iyi çözüm açgözlüye dönüştürülür (Şekil 12.6). Aralık çizelgeleme, kesirli sırt çantası ve Huffman bu yolla kanıtlanır.
- **Kesim özelliği** (cut property): köşeleri iki gruba ayıran her kesimde, kesimi geçen **en hafif** kenar bir en küçük kapsayan ağaçtadır. Kruskal ve Prim yalnızca bu kenarları ekler (Bölüm 4.7).

Kanıt yoksa açgözlü küçük örneklerde **kâhinle** (test oracle), yani bütün olasılıkları deneyen kaba kuvvetle karşılaştırılır. Kâhinin uyuşmazlık bulması ölçütün yanlışlığını kanıtlar; bulamaması yalnızca güveni artırır.

3.4 Açgözlü ne zaman doğru, ne zaman yanlış?

Şekil 12.1 haftanın problemlerini özetler: ölçüt, sonucun en iyi olup olmadığı ve bunu gösteren örnek. Kanıtlı ölçütlerde (yeşil) program "en iyi" diyebilir; kanıtsız ya da çürütülmüş ölçütler (kırmızı) **sezgiseldir** (heuristic): hızlı, çoğu zaman iyi, ama garantisiz. Graf boyamadaki Welsh–Powell bunun tipik örneğidir: sınav çakışma grafında en az oturum sayısını buluyor, iki parçalı taç grafında en iyinin iki katı renk kullanıyor (Bölüm 5.3).

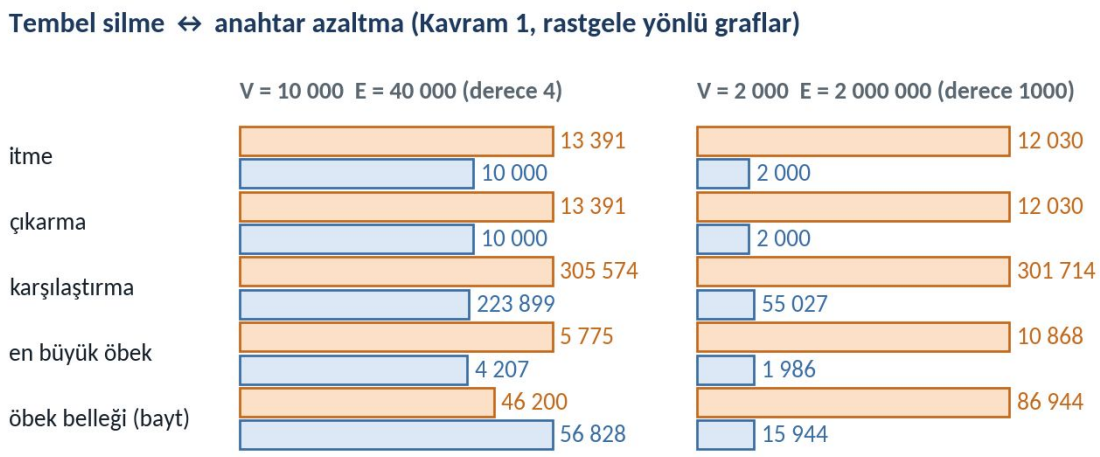
Şekil 12.1
Açgözlü ve en iyi: problem × ölçüt × "en iyi mi?" (bütün sayılar örneklerin çıktılarından)

Problem	Açgözlü ölçüt	En iyi?	Kanıt / karşı örnek	Örnek
Para üstü (AP-1 H4 sistemi)	en büyük para	✔ evet	100/100 tutarda en iyi	1
Para üstü {1, 3, 4}	en büyük para	✘ hayır	yalnız 76/100 tutarda	1
Aralık çizelgeleme	en erken biten ilk	✔ evet	değiş tokuş; 1000/1000	2
Aralık çizelgeleme	başka üç ölçüt	✘ hayır	karşı örnek; en kısa 953/1000	2
Aralık bölümleme	erken boşalan salon	✔ evet	salon = derinlik 1000/1000	3
Kesirli sırt çantası	birim değer	✔ evet	720 sıralamanın en iyisi	4
0/1 sırt çantası	birim değer	✘ hayır	160 < 220 (→ H13)	4
Önek kodu	Huffman: en küçük iki	✔ evet	B(T) = 224 bit	5
En kısa yol, w ≥ 0	Dijkstra: en küçük d	✔ evet	kâhin 3/3 graf	7–8
En kısa yol, w < 0 var	Dijkstra	✘ hayır	2 köşe yanlış	9
En küçük kapsayan ağaç	Kruskal / Prim	✔ evet	kaba kuvvet 280; 20/20	11–12
Graf boyama	Welsh–Powell	✘ hayır	taç: 4 renk, χ = 2	13

3.5 Kaputun Altında: tembel silme, bit tamponu, navigasyon

> KAPUTUN ALTINDA · **Anahtar azaltma yerine tembel silme**
>
> Dijkstra ve Prim bir anahtarı küçültünce öbekteki kaydı güncellemek ister. **Anahtar azaltma** (decrease-key) bunu yapar ama köşenin öbekte **nerede** olduğunu bilmeyi (her takasta güncellenen `konum[]`) gerektirir; `std::priority_queue` ve `heapq` bu işlemi sunmaz. Daha basit olan **tembel silmedir** (lazy deletion): köşe yeni uzaklığıyla yeniden itilir, eski kayıt çıktığında "zaten kesinleşti" denetimiyle **atlanır**. Bedeli öbeğin boyudur: öbek en çok 1 + (başarılı gevşetme) ≤ E + 1 kayıt tutar; anahtar azaltmada ise öbekte en çok V kayıt vardır ama V öğelik `konum[]` dizisi gerekir.
>
> Kavram 1 iki yöntemi aynı graflarda sayar (Şekil 12.2): seyrek grafta (V = 10 000, derece 4) eskimiş kayıt 3 391, öbek içi karşılaştırma 305 574 ↔ 223 899; burada `konum[]` dizisi yüzünden anahtar azaltmanın belleği bile büyüktür (46 200 ↔ 56 828 bayt). Yoğun grafta (V = 2 000, derece 1000) tembel silmenin bedeli belirginleşir: en büyük öbek 10 868 ↔ 1 986 kayıt, öbek belleği 86 944 ↔ 15 944 bayt. Program süreleri de stderr'e yazar, ama bunlar tek ölçümdür ve bu makinede çalıştırmadan çalıştırmaya kat kat değişir; karşılaştırmayı sayaçlarla yapın.

Şekil 12.2
Tembel silme ile anahtar azaltmanın sayaçları ve öbek belleği (Kavram 1; bu makinede 8 baytlık öğe)



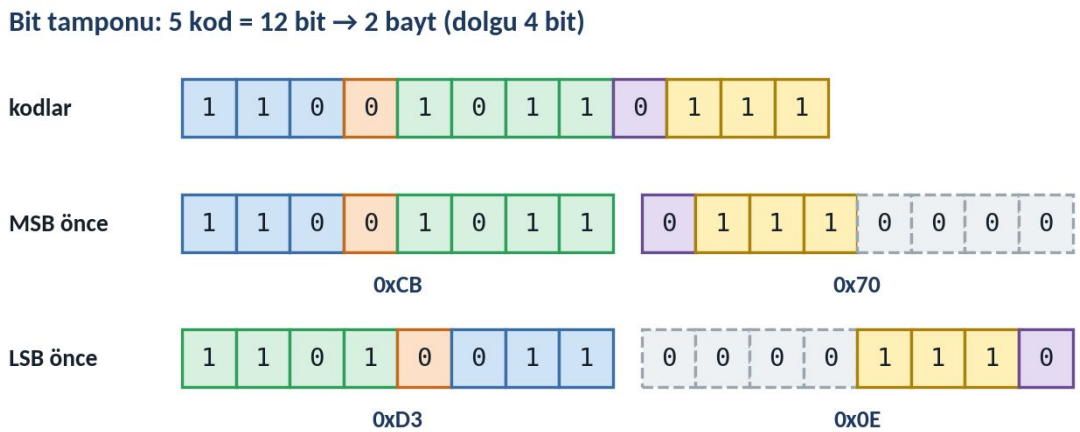
turuncu (üstte): tembel silme · mavi (altta): anahtar azaltma (konum dizili öbek); bellek: öğe × boy (+ konum[])

KAPUTUN ALTINDA · **Huffman kodları bitlere nasıl yazılır?**

Bellek bayt düzeyinde adreslenir; Huffman kodları ise değişken sayıda **bit**tir. Kodlar bir **bit tamponunda** (bit buffer) ardışık bitler olarak birleştirilir: akıştaki i'nci bit, `i >> 3` numaralı baytın bir bitine yazılır (AP-1 H4 bit işlemleri). Hangi bitine yazılacağı bir sözleşmedir. Kavram 2'de `110 0 1011 0 111` kodları 12 bit, yani 2 bayt eder; son baytın 4 biti **dolgudur** (padding). En anlamlı bit önce (MSB önce, Örnek 6'nın düzeni) baytlar 0xCB 0x70, en az anlamlı bit önce (LSB önce) 0xD3 0x0E olur (Şekil 12.3).

Çözücü kaç bit okuyacağını bilmezse dolguyu veri sanabilir; bu yüzden dosya biçimi simge sayısını ve ağacı da saklar (Örnek 6 bu maliyeti hesaplar). DEFLATE LZ77 ile Huffman'ı birleştirir ve baytları her zaman en az anlamlı bitten başlayarak doldurur; Huffman dışındaki öğeleri kendi en az anlamlı bitinden, Huffman kodlarını ise kodun en anlamlı bitinden başlayarak yazar (Deutsch, 1996). MSB önce düzenimiz bir ders seçimidir; kodlar ikili bir dosyaya yazılırsa (ör. final projesi seçeneği "Huffman tabanlı dosya sıkıştırıcı") bu sözleşme dosya biçiminin parçası olur.

Şekil 12.3
Bit tamponu (Kavram 2): beş kodun 12 biti iki bayta; MSB önce ve LSB önce düzen, dolgu bitleri



Gri kesikli hücreler dolgu bitleridir (0). MSB önce: bit i, bayt i/8'in 7 − i mod 8 numaralı bitine (Örnek 6).

LSB önce: baytı en az anlamlı bitten doldurur (DEFLATE veri öğeleri böyle paketlenir; RFC 1951).

> KAPUTUN ALTINDA · **Gerçek navigasyon: Dijkstra yetmez, yönlendirmek gerekir**
>
> Dijkstra kaynaktan **her yöne** eşit genişleyen bir dalgadır: hedef kuzeydeyse güneydeki binlerce kavşağı da kesinleştirir. Haritalama sistemleri aramayı iki yolla daraltır. Birincisi A* algoritmasıdır (Hart vd., 1968): anahtar d[u] yerine d[u] + h(u) olur; h(u), u'dan hedefe kalan yolun **alt sınırı** olan bir tahmindir (ör. kuş uçuşu uzaklık). h fazla tahmin etmezse yol yine en kısadır ama hedefe doğru olmayan köşeler daha az açılır. İkincisi **ön işlemedir**: yol ağı sorgulardan önce işlenir ve aramayı küçük bir alt ağa indiren ek bilgiler (kısayol kenarları, sınır noktaları) saklanır. Bunlar kapsam dışıdır (Bölüm 13); çekirdeklerinde bu haftanın gevşetmesi ve öncelik kuyruğu vardır.

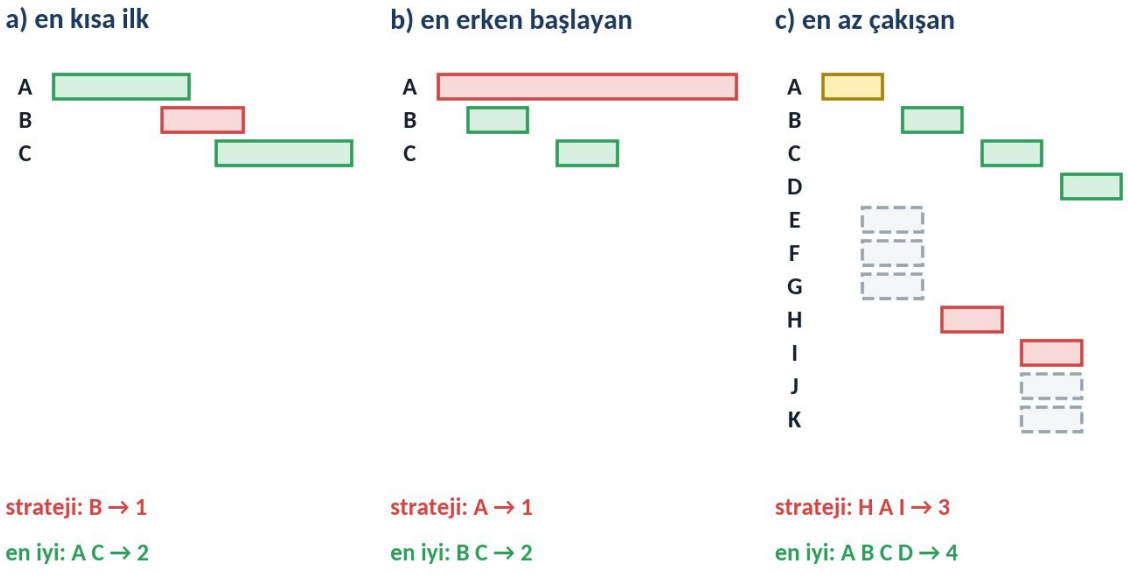
4. Algoritmalar

Sekiz amiral algoritmayı on bir adımlı standart anlatımla işliyoruz: 4.1–4.4 sıralama ya da öbek üzerine kurulu seçimler, 4.5–4.8 graf algoritmaları. C ve üç dil adımlarındaki kod **Kavram 5**'tendir (`kavram/H12_K05_cekirdek.*`): örneklerdeki fonksiyonların izleme ve sayaç satırları atılmış, adları ve adımları aynı kalmış çekirdekleri; çıktısı örneklerin sonuçlarıyla birebir aynıdır (on sonuç betikle karşılaştırıldı). Üç dil adımında yalnız farkların bulunduğu satırlar gösterilir; tam C++ ve Python sürümleri ve dil notları Ek A'dadır.

4.1 Aralık çizelgeleme: en erken biten ilk

1 · Problem. Tek bir salona n istek gelir; her istek yarı açık bir zaman aralığıdır, [bas, bit). Çakışmayan (biri öbürü başlamadan biten) istekler birlikte kabul edilebilir. **En çok** sayıda çakışmayan isteği seçin. Yarı açıklık önemlidir: 10'da biten toplantının yerine 10'da başlayan girebilir. Örnek 2'nin girdisi: A[4, 7) B[0, 3) C[3, 8) D[1, 4) E[7, 10) F[9, 12) G[6, 9) H[11, 14) I[2, 6) J[8, 13). **2 · Fikir.** Dört aday ölçüt: en kısa ilk, en erken başlayan ilk, en az çakışan ilk ve **en erken biten ilk**. Yalnız sonuncusu her zaman en iyidir: salonu en erken boşaltan istek geriye en çok zamanı bırakır. Örnek 2 ötekilerin her biri için kaba kuvvetle doğrulanmış bir karşı örnek verir (Şekil 12.4); en az çakışan ölçütü bile on bir aralıklı özel bir girdide 3'te kalır, en iyi 4'tür.

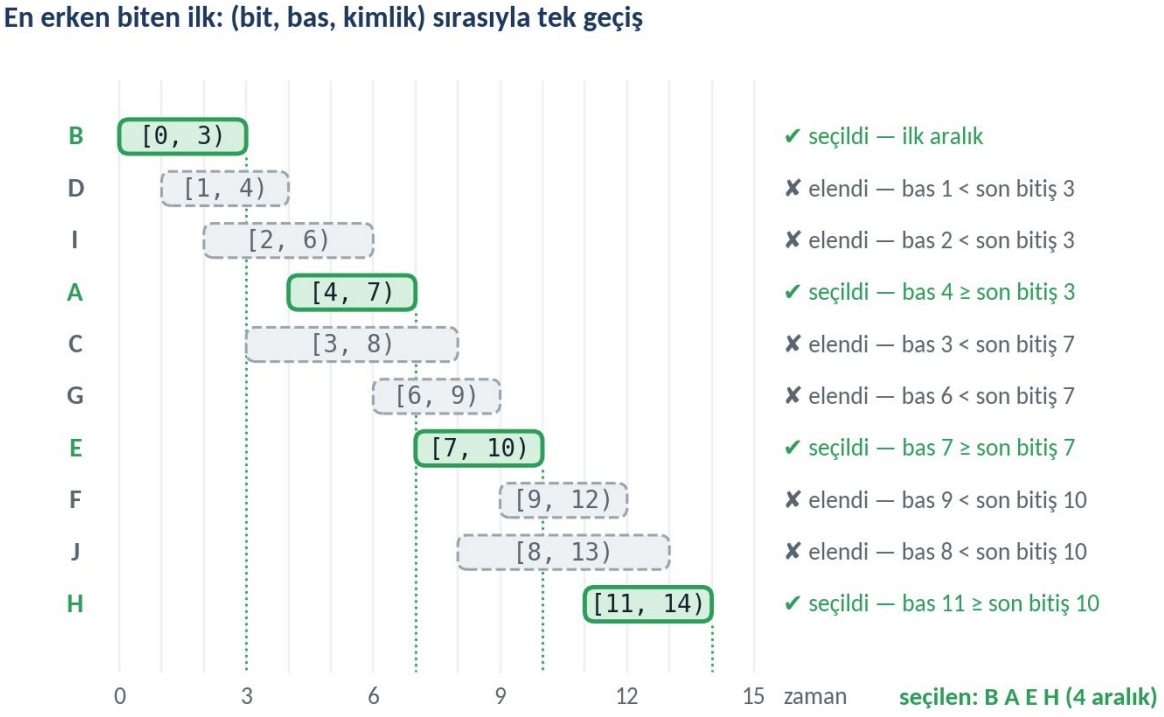
Şekil 12.4
Üç yanlış ölçütün karşı örnekleri (Örnek 2, kaba kuvvetle doğrulandı): strateji (kırmızı) ve en iyi çözüm (yeşil)



kırmızı: yalnız stratejinin seçtiği · yeşil: yalnız en iyi çözümde · sarı: ikisinde de · kesikli gri: hiçbirinde

3 · Elle izleme. Aralıkları (bit, bas, kimlik) sırasıyla dolaşırız; başlangıcı **son seçilenin bitişinden küçük olmayan** aralık seçilir (Şekil 12.5): B [0, 3) ilk aralıktır; D ve I, B bitmeden başladığı için elenir; A [4, 7) seçilir; C ve G A'yla çakışır. Seçilenler B, A, E, H: 4 aralık; kaba kuvvet (1024 alt küme) en çok 4.

Şekil 12.5
Aralık çizelgeleme zaman çizgisi (Örnek 2): seçilen (yeşil) ve elenen (gri kesikli) aralıklar; noktalı çizgiler son bitişler



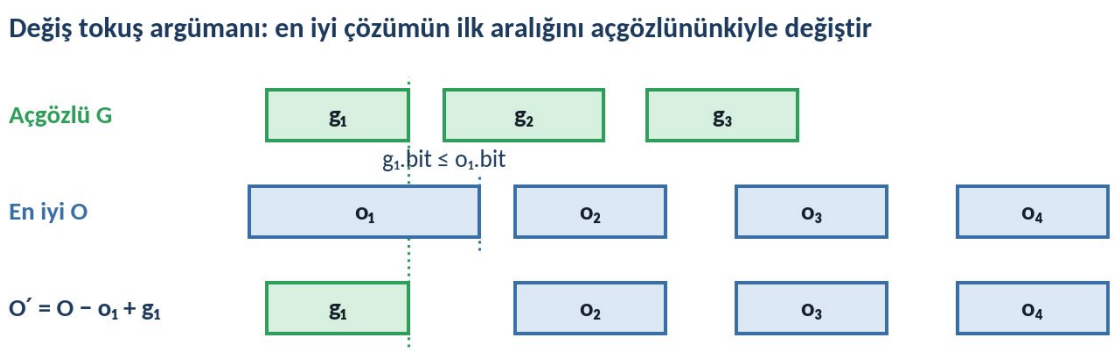
SÖZDE KOD · 4 · Sözde kod: aralık çizelgeleme

```
algoritma EN-ERKEN-BİTEN(A[0..n−1])        ▷ A[i] = [bas, bit)
    A'yı (bit, bas, kimlik) sırasına diz;  S ← boş;  son ← −∞
    her x için A'da (sırayla)
        eğer x.bas ≥ son ise
            S ← S ∪ {x};  son ← x.bit       ▷ açgözlü seçim
    döndür S
```

5 · Doğruluk. Değiş tokuş argümanı (Şekil 12.6): O bir en iyi çözüm, o_1 onun ilk aralığı, g_1 açgözlünün ilk seçimi olsun. g_1 en erken biten aralıktır: g_1.bit ≤ o_1.bit. O'da o_1 yerine g_1 konursa çakışma doğmaz (o_2 zaten o_1 bitince başlıyordu) ve sayı değişmez: g_1'i içeren bir en iyi çözüm vardır (**açgözlü seçim özelliği**). Kalan problem "g_1.bit'ten sonra başlayan aralıklarda aynı soru"dur (**en iyi alt yapı**); tümevarımla açgözlü en iyidir. **Önde kalma** biçimi kartın değişmezidir: Açgözlünün r'inci aralığı, en iyinin r'inci aralığından geç bitmez. Örnek 2 ikisini de rastgele örneklerde denetler: değiş tokuş 1000/1000, önde kalma 1000/1000 örnekte geçerli.

6 · Analiz. Sıralama Θ(n log n), tek geçiş Θ(n); aralıklar bitişe göre zaten sıralıysa Θ(n). Ek bellek sıralı kopya için O(n) (Tablo 12.1).

Şekil 12.6
Değiş tokuş argümanı: en iyi çözümün ilk aralığı açgözlünün seçimiyle değiştirilir; çözüm geçerli ve aynı büyüklükte kalır



Tablo 12.1
Aralık çizelgeleme kimlik kartı

ALGORİTMA KİMLİK KARTI **Aralık çizelgeleme (en erken biten ilk)** *(interval scheduling, earliest finish time first)*	
Strateji	Açgözlü: aralıkları bitiş zamanına göre sırala
Girdi önkoşulu	Ağırlıksız, yarı açık aralıklar [bas, bit)
Zaman (en iyi · ortalama · en kötü)	Θ(n log n) · Θ(n log n) · Θ(n log n)
Ek bellek	O(n)
Özellikler	Tek geçiş; seçim geri alınmaz. Rastgele 1000 örnekte kâhinle aynı (Örnek 2)
Değişmez / doğruluk	Açgözlünün r'inci aralığı, en iyinin r'inci aralığından geç bitmez
Kütüphane karşılığı	C: `qsort` yalnız sıralama için · C++: `std::sort` + `std::tie` · Python: `sorted(key=…)`
Ne zaman kullanılır	Tek kaynakta (salon, makine, anten) çakışmayan en çok isteği kabul etmek
Ne zaman kullanılmaz	Aralıkların ağırlığı farklıysa (DP ister, H13); bütün istekler için en az salon (4.2)

7 · C gerçekleştirimi. `qsort` kararlı olmadığından karşılaştırıcı (`bitise_gore`) tam sıralama tanımlar: (bit, bas, kimlik); eşit bitişlerde üç dil aynı sırayı verir. Örnek 2'deki aynı adlı fonksiyon (`c/H12_02_aralik_cizelgeleme.c`, satır 91–119) her kararın gerekçesini de yazar:

C **kavram/H12_K05_cekirdek.c · satır 108–119**

```c
static int en_erken_biten(Aralik *a, int n, Aralik *secilen)
{
    qsort(a, (size_t)n, sizeof a[0], bitise_gore);
    int k = 0, son_bitis = -1;
    for (int i = 0; i < n; i++) {
        if (a[i].bas >= son_bitis) {     /* açgözlü seçim */
```

```
            secilen[k++] = a[i];
            son_bitis = a[i].bit;
        }
    }
    return k;
}
```

8 · Üç dil karşılaştırması. C++'ta `std::sort` + `std::tie`, Python'da `sorted(key=…)` anahtar demeti aynı tam sıralamayı verir; seçilenler `std::vector` ve listededir:

C++ **H12_K05_cekirdek.cpp · satır 52–57**

```
    std::sort(a.begin(), a.end(), bitise_gore);
    int son_bitis = -1;
    for (const Aralik& x : a) {
        if (x.bas >= son_bitis) {        // açgözlü seçim
            secilen.push_back(x);
            son_bitis = x.bit;
```

Python **H12_K05_cekirdek.py · satır 24–29**

```
    a = sorted(a, key=lambda x: (x.bit, x.bas, x.kimlik))
    son_bitis = -1
    for x in a:
        if x.bas >= son_bitis:           # açgözlü seçim
            secilen.append(x)
            son_bitis = x.bit
```

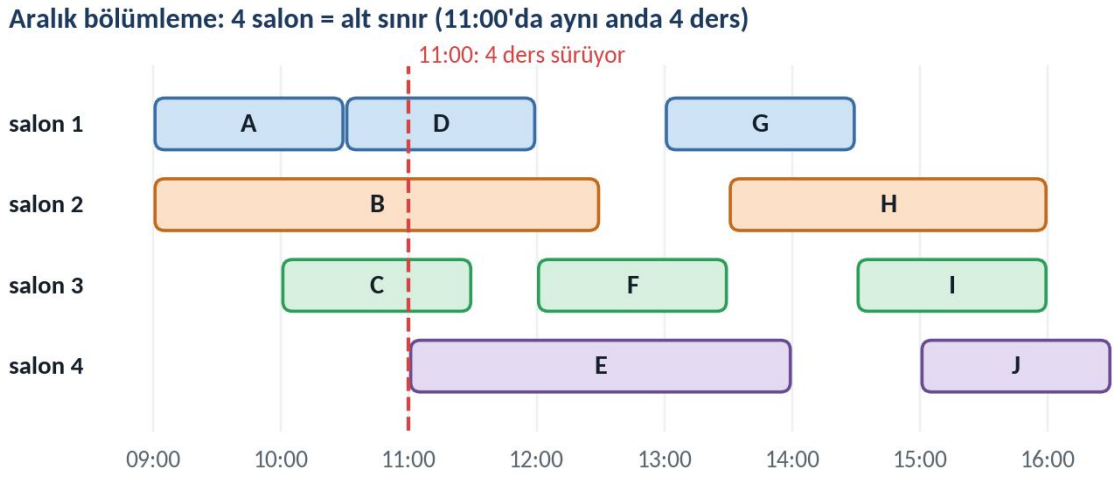
9 · Test. Sınırlar: boş girdi, tek aralık, aynı aralık üç kez (biri seçilir), bitişi sonrakinin başlangıcına eşit aralıklar (hepsi seçilir). Kâhin, 2^n alt kümeyi deneyen kaba kuvvettir: 1000 rastgele örnekte (n = 8, bas 0..20, uzunluk 1..8, tohum 42) açgözlü 1000/1000 örnekte en iyiyi buldu. **10 · Deney.** Aynı örneklerde ölçütlerin en iyiyi bulma oranları: en erken biten 1000/1000; en kısa ilk 953/1000; en erken başlayan 760/1000; en az çakışan 1000/1000. En az çakışan rastgele örneklerin **hepsinde** doğru çıktı; yanlışlığını yalnız kurulmuş karşı örnek gösterir (Bölüm 9). **11 · Varyantlar.** Ağırlıklı aralıklarda açgözlü yetmez, DP gerekir (H13); bütün istekleri en az salona yerleştirmek aralık bölümlemedir (4.2); en az noktayla bütün aralıkları delmek aynı sıralamayla çözülür (Alıştırma 12.10).

4.2 Aralık bölümleme: en az salon

1 · Problem. Bu kez **bütün** istekler karşılanacak; çakışmayan istekler aynı salonu paylaşabilir. En az kaç salon gerekir, hangi ders hangi salona? Örnek 3'ün girdisi on derstir (09:00–16:30). **2 · Fikir.** Dersleri **başlangıç** sırasıyla dolaş; boşalmış bir salon varsa onu kullan, yoksa yeni salon aç. "Boşalmış salon var mı?" sorusunu salonların **bitiş zamanlarını** tutan en küçük öbek yanıtlar: kökteki salon en erken boşalandır; o bile boş değilse hiçbiri boş değildir. Öbek öğesi (bitiş, salon) demetidir.

3 · Elle izleme. Şekil 12.7 on dersin salonlarını verir. D 10:30'da başlar; öbeğin kökü A'nın salonu (1), bitişi 10:30: yarı açık aralıkta <= sağlanır ve D salon 1'i alır; öbek (11:30,3) (12:00,1) (12:30,2) olur. E 11:00'de başladığında kökteki bitiş daha geçtir: salon 4 açılır.

Şekil 12.7
Salon ataması (Örnek 3): 4 salon; kesikli çizgi 11:00'da aynı anda süren 4 dersi gösterir (alt sınır)



SÖZDE KOD · 4 · Sözde kod: aralık bölümleme

```
algoritma BÖLÜMLE(D[0..n−1])                ▷ D[i] = [bas, bit)
    D'yi (bas, bit, kimlik) sırasına diz;  Ö ← boş en küçük öbek;  k ← 0
    her x için D'de (sırayla)
        eğer Ö boş değil ve Ö.en_küçük.bitiş ≤ x.bas ise
            (_, s) ← ÇIKAR(Ö)                ▷ en erken boşalan salon
        değilse  s ← k;  k ← k + 1           ▷ yeni salon
        salon[x] ← s;  EKLE(Ö, (x.bit, s))
    döndür k
```

5 · Doğruluk. Derinlik d, aynı anda süren en çok ders sayısıdır; her algoritma en az d salon açmak **zorundadır**. Açgözlü k'inci salonu açtığında x başlarken öbekteki bütün salonlar doluydu: k − 1 salonun her birinde x.bas anını kapsayan bir ders, artı x: o anda k ders aynı anda sürer, yani k ≤ d. Birlikte k = d: açgözlü **en iyidir** (Kleinberg & Tardos, 2006). Örnek 3 derinliği ayrı bir olay süpürmesiyle hesaplar (`derinlik`): 11:00'da aynı anda 4 ders sürer, yani en az 4 salon gerekir; 1000 rastgele örnekte salon = derinlik 1000/1000. **6 · Analiz.** Sıralama Θ(n log n); her derste en çok bir çıkarma ve bir itme, O(log n): toplam O(n log n). Bu girdide itme 10, çıkarma 6'dır (yeni salon açılan adımda çıkarma yok).

7 · C gerçekleştirimi. Öbek H7 öbeğinin genelleştirilmesidir: öğe (anahtar, no) `struct`'ı, karşılaştırıcı sözlük sırası (`kucuk_mu`). Tam sürüm `c/H12_03_aralik_bolumleme.c`, satır 136–169:

C kavram/H12_K05_cekirdek.c · satır 122–138

```c
static int bolumle(Aralik *a, int n, int *salon)
{
    qsort(a, (size_t)n, sizeof a[0], basa_gore);
    Obek obek = {.n = 0};                /* (bitiş, salon) */
    int salon_sayisi = 0;
    for (int i = 0; i < n; i++) {
        int s;
        if (obek.n > 0 && obek.a[0].anahtar <= a[i].bas) {
            s = en_kucugu_cikar(&obek).no;     /* boşalan salon */
        } else {
            s = salon_sayisi++;                /* yeni salon */
        }
```

```
134          salon[a[i].kimlik] = s;
135          obege_ekle(&obek, (Oge){a[i].bit, s});
136      }
137      return salon_sayisi;
138  }
```

8 · Üç dil karşılaştırması. C++'ta `std::priority_queue<…, std::greater<…>>` en küçük öbektir (varsayılanı en büyük); Python'da `heapq` listeyi en küçük öbek olarak yönetir:

C++ **H12_K05_cekirdek.cpp · satır 71–78**

```
71          if (!obek.empty() && obek.top().first <= x.bas) {
72              s = obek.top().second;              // boşalan salon
73              obek.pop();
74          } else {
75              s = salon_sayisi++;                 // yeni salon
76          }
77          salon[x.kimlik] = s;
78          obek.push({x.bit, s});
```

Python **H12_K05_cekirdek.py · satır 39–45**

```
39          if obek and obek[0][0] <= x.bas:
40              _, s = heapq.heappop(obek)          # boşalan salon
41          else:
42              s = salon_sayisi                    # yeni salon
43              salon_sayisi += 1
44          salon[x.kimlik] = s
45          heapq.heappush(obek, (x.bit, s))
```

9 · Test. Kâhin derinliktir; ayrıca hiçbir salonda çakışan iki ders olmadığı denetlenir. Sınır: [0, 60) ve [60, 120) tek salon (< yazılırsa 2: hata). **10 · Deney.** Mini-OBS 2 v2.11 algoritmayı 10^6 sınavda ölçer (Bölüm 10). **11 · Varyantlar.** Salon kapasitesi eklenince açgözlü en iyi değildir (v2.11 belgesi, meydan okuma c); sınavları oturumlara dağıtmak graf boyamadır (Bölüm 5.3).

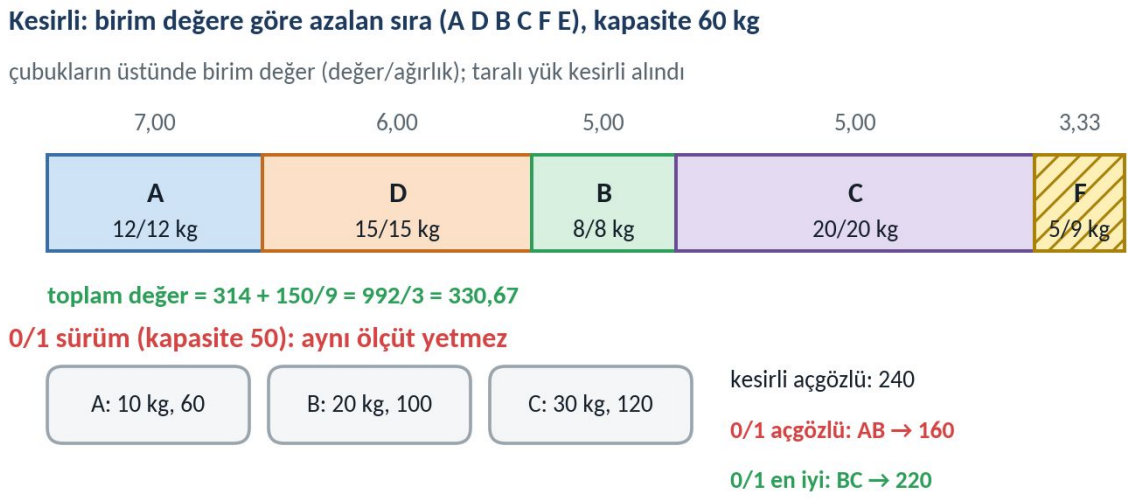
4.3 Kesirli sırt çantası

1 · Problem. Kapasitesi W kg olan çantaya, ağırlığı w_i ve değeri v_i olan n yükten **istediğimiz kesri** koyabiliyoruz (un, kum gibi bölünebilir yükler). Toplam değeri en büyük yapın. Örnek 4: 60 kg, 6 yük. **2 · Fikir.** Kilogram başına en çok kazandıranı önce al: **birim değer** v_i/w_i'ye göre azalan sırala, sığanların tamamını, ilk sığmayanın sığan kesrini al ve dur. **3 · Elle izleme.** Sıra A D B C F E; çanta F'nin ortasında dolar, toplam değer 314 + 150/9 = 992/3 = 330,67 (Şekil 12.8).

SÖZDE KOD · 4 · Sözde kod: kesirli sırt çantası

```
algoritma KESİRLİ(Y[0..n−1], W)            ▷ Y[i] = (w, v)
    Y'yi v/w'ye göre azalan diz        ▷ v₁·w₂ > v₂·w₁ ile
    toplam ← 0;  kalan ← W
    her y için Y'de (sırayla), kalan > 0 iken
        eğer y.w ≤ kalan ise  toplam ← toplam + y.v;  kalan ← kalan − y.w
        değilse  toplam ← toplam + y.v · kalan / y.w;  kalan ← 0
    döndür toplam
```

Şekil 12.8
Kesirli sırt çantası (Örnek 4): birim değer sırası ve son yükün kesri; altta 0/1 sürümde aynı ölçütün başarısızlığı



0/1 sırt çantası dinamik programlama ister (H13).

5 · Doğruluk. Değiş tokuş: bir çözüm, birim değeri yüksek bir yükten almadığı bir kilogram varken düşük birim değerli bir yükten bir kilogram almışsa, ikisini değiştirmek değeri artırır ya da korur; en iyi çözümde yükler birim değer sırasıyla dolar. Kartın değişmezi: Tamamen alınan her yükün birim değeri, alınmayan her yükünkinden küçük değildir; en çok bir yük kesirli alınır. **6 · Analiz.** Sıralama Θ(n log n), doldurma Θ(n); ek bellek O(n) (Tablo 12.2).

Tablo 12.2
Kesirli sırt çantası kimlik kartı

ALGORİTMA KİMLİK KARTI **Kesirli sırt çantası** *(fractional knapsack)*	
Strateji	Açgözlü: yükleri birim değere (değer/ağırlık) göre azalan sırala
Girdi önkoşulu	Yükler bölünebilir; birim değerler çapraz çarpımla karşılaştırılır
Zaman (en iyi · ortalama · en kötü)	Θ(n log n) · Θ(n log n) · Θ(n log n)
Ek bellek	O(n)
Özellikler	Değer tamsayı kesir; en çok bir yük kesirli alınır (Örnek 4)
Değişmez / doğruluk	Tamamen alınan her yükün birim değeri, alınmayan her yükünkinden küçük değildir; en çok bir yük kesirli alınır
Kütüphane karşılığı	C: `qsort` yalnız sıralama için · C++: `std::sort` · Python: `sorted` + `functools.cmp_to_key`
Ne zaman kullanılır	Bölünebilir kaynakların paylaştırılması (sıvı, zaman, bant genişliği)
Ne zaman kullanılmaz	Yükler bölünemiyorsa (0/1 sırt çantası: dinamik programlama, H13)

7 · C gerçekleştirimi. Kayan noktada 5,00 ile 5,00'in eşitliği güvenilmezdir; $v_1/w_1 > v_2/w_2$ karşılaştırması **çapraz çarpımla** ($v_1 \cdot w_2 > v_2 \cdot w_1$, taşmaya karşı `long long`) yapılır, toplam (pay, payda) tamsayı kesridir. Tam sürüm `c/H12_04_kesirli_sirt_cantasi.c`, satır 60–69; doldurma döngüsü (satır 90–128) sözde kodun birebir karşılığıdır. Karşılaştırıcı:

C **kavram/H12_K05_cekirdek.c · satır 146–155**

```c
static int birim_degere_gore(const void *p, const void *q)
{
    const Yuk *x = p, *y = q;
    long long sol = (long long)x->deger * y->agirlik;
    long long sag = (long long)y->deger * x->agirlik;
    if (sol != sag) {
        return sol > sag ? -1 : 1;
    }
    return (x->kimlik > y->kimlik) - (x->kimlik < y->kimlik);
}
```

8 · Üç dil karşılaştırması. `qsort` karşılaştırıcısı −1/0/1, `std::sort` yüklemi `bool` döndürür; Python'da −1/0/1 karşılaştırıcısı `functools.cmp_to_key` ile anahtara çevrilir ve tamsayı taşmaz:

C++ **H12_K05_cekirdek.cpp · satır 89–97**

```cpp
static bool birim_degere_gore(const Yuk& x, const Yuk& y)
{
    long long sol = static_cast<long long>(x.deger) * y.agirlik;
    long long sag = static_cast<long long>(y.deger) * x.agirlik;
    if (sol != sag) {
        return sol > sag;
    }
    return x.kimlik < y.kimlik;
}
```

Python **H12_K05_cekirdek.py · satır 50–55**

```python
def birim_degere_gore(p, q):
    """Birim değere göre azalan: v1/w1 > v2/w2  <=>  v1·w2 > v2·w1"""
    sol, sag = p.deger * q.agirlik, q.deger * p.agirlik
    if sol != sag:
        return -1 if sol > sag else 1
    return (p.kimlik > q.kimlik) - (p.kimlik < q.kimlik)
```

9 · Test. Kâhin, yüklerin bütün sıralamalarıyla (n! = 720) doldurup en büyük değeri bulur: 992/3 = 330,67; birim değer sırası da aynı değeri verir. **10 · Deney.** Başka ölçütler: “en değerli ilk” 1578/5 = 315,60, “en hafif ilk” 324. **11 · Varyantlar.** Yükler **bölünemezse** (0/1 sırt çantası) aynı ölçüt yanılır: kapasite 50, A(10 kg, 60) B(20 kg, 100) C(30 kg, 120) için açgözlü AB ile 160, kaba kuvvet (8 alt küme) BC ile 220'yi bulur. 0/1 sürümde açgözlü seçim özelliği yoktur (H13: DP); kesirli çözüm onun için bir **üst sınırdır**.

4.4 Huffman kodlaması

1 · Problem. Simgeleri ikili kodlarla yazacağız; hiçbir kod bir başkasının **öneki** olmasın (**önek kodu**, prefix code), böylece bit akışı ayraçsız tek türlü çözülsün. Sıklıklar bilindiğine göre toplam bit sayısını, $B(T) = \Sigma$ sıklık × kod uzunluğu, en küçük yapın. Cormen vd. (2022) örneğinin sıklıklarını (orada binlik birimdedir)

burada simge sayısı olarak kullanıyoruz: a:45 b:13 c:12 d:16 e:9 f:5, toplam 100 simge; sabit 3 bitle 300 bit. **2 • Fikir.** Önek kodu bir ikili ağaçtır: yapraklar simgeler, kökten yaprağa yol kod (sol 0, sağ 1); sık simgeler yüzeyde, seyrekler derinde olmalıdır. Huffman'ın açgözlü fikri: **en seyrek iki** ağacı birleştir; yeni ağacın sıklığı toplamlarıdır. n − 1 birleştirmede tek ağaç kalır.

3 • Elle izleme. Öbekte (sıklık, düğüm no) demetleri vardır; düğüm no oluşturulma sırasıdır (yapraklar 0–5, iç düğümler 6–10), eşit sıklıkta önce oluşturulan çıkar; ilk çıkan sol (0), ikinci sağ (1) çocuktur. Şekil 12.9 beş birleştirmeyi, Şekil 12.10 ağacı ve kod tablosunu gösterir: B(T) = 224 bit (sabit uzunlukta 300).

SÖZDE KOD • 4 • Sözde kod: Huffman ağacı

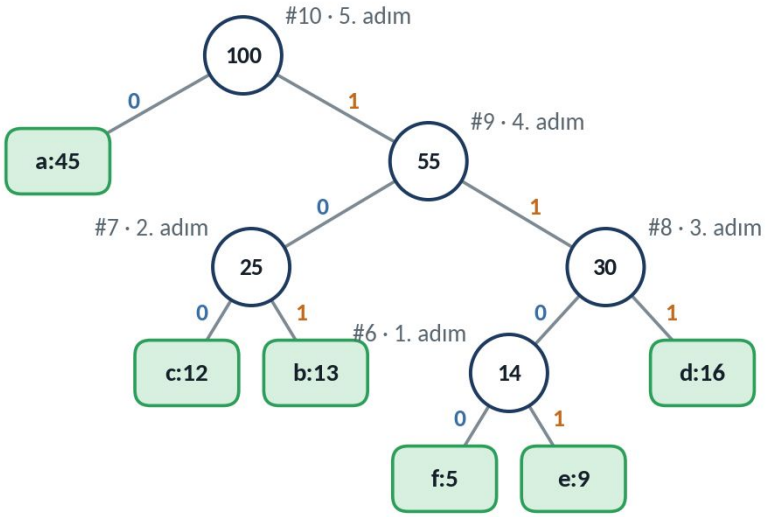
```
algoritma HUFFMAN(C)                          ▷ C: (simge, sıklık) çiftleri
    Ö ← en küçük öbek;  her c için C'de: EKLE(Ö, (c.sıklık, yaprak(c)))
    iken |Ö| > 1
        (fx, x) ← ÇIKAR(Ö);  (fy, y) ← ÇIKAR(Ö)
        z ← yeni iç düğüm (sol x → 0, sağ y → 1);  EKLE(Ö, (fx + fy, z))
    döndür ÇIKAR(Ö).düğüm                     ▷ kök; tek simgede kod "0"
```

5 • Doğruluk. İki lemma (kanıt taslağı; Cormen vd., 2022): (i) **açgözlü seçim:** en seyrek iki simgenin en derin düzeyde kardeş yaprak olduğu bir en iyi ağaç vardır; en derin iki kardeş yaprakla bu iki simge **yer değiştirilirse** (değiş tokuş) B(T) artmaz. (ii) **en iyi alt yapı:** bu iki simgeyi toplam sıklıklı tek simgeye indirgeyen problemin en iyi ağacı, yaprak iki çocuğa açılınca asıl problemin en iyi ağacını verir; her birleştirme B(T)'ye birleşen sıklıkların toplamını ekler. Bu yüzden B(T) iç düğüm sıklıkları toplamıdır: program iki yoldan da 224 bulur (iç düğüm toplamı 224). **6 • Analiz.** n farklı simgede n + (n − 1) = 2n − 1 itme ve 2(n − 1) + 1 = 2n − 1 çıkarma (sonuncusu kökü alır; Cormen tablosunda 11 itme, 11 çıkarma), her biri O(log n): Θ(n log n); n metin uzunluğu değil **farklı simge** sayısıdır (baytta n ≤ 256); sıklık sayımı Θ(m) (Tablo 12.3).

Şekil 12.9
Huffman kurulum adımları (Örnek 5, Cormen vd. sıklıkları): her satır öbeğin içeriği; sarı: sıradaki iki çıkarma, yeşil: yeni iç düğüm



Şekil 12.10
Huffman ağacı ve kod tablosu (Örnek 5): iç düğümlerde sıklık, yanlarında oluşturuldukları adım; B(T) = 224 bit



simge	sıklık	kod	bit
a	45	0	45
b	13	101	39
c	12	100	36
d	16	111	48
e	9	1101	36
f	5	1100	20

B(T) = Σ sıklık × uzunluk = 224 bit

sabit 3 bit/simge: 300 bit

Tablo 12.3
Huffman kodlaması kimlik kartı

ALGORİTMA KİMLİK KARTI	**Huffman kodlaması** *(Huffman coding)*
Strateji	Açgözlü + en küçük öbek: en düşük sıklıklı iki düğümü çıkar, toplamlarıyla yeni iç düğüm olarak geri it
Girdi önkoşulu	Simge sıklıkları önceden bilinir (iki geçiş: say, sonra kodla)
Zaman (en iyi · ortalama · en kötü)	Θ(n log n) · Θ(n log n) · Θ(n log n)
Ek bellek	O(n)
Özellikler	Cormen vd. tablosunda B(T) = 224 bit; tek simgede kod "0" (Örnek 5–6)
Değişmez / doğruluk	Öbekteki her düğüm ormandaki bir ağacın köküdür; kökün sıklığı yapraklarının toplamıdır
Kütüphane karşılığı	C: yok · C++: `std::priority_queue` + `std::greater` · Python: `heapq`
Ne zaman kullanılır	Simge sıklıkları dengesiz veriler
Ne zaman kullanılmaz	Sıklıklar neredeyse eşitse ya da kod tablosu verinin kendisine göre büyükse (küçük dosyalar)

7 · C gerçekleştirimi. Düğümler işaretçi yerine **dizilerdedir** (indis = düğüm no = eşitlik kırma anahtarı); `malloc` ve yok etme gerekmez. Tam sürüm `c/H12_05_huffman_agaci.c`, satır 160–197 her adımda öbeği yazar; kodları `kod_ata` kurar (tek düğümlü ağaçta "0"; Hata Avcısı 6):

C **kavram/H12_K05_cekirdek.c · satır 207–223**

```c
static void agac_kur(const int *siklik, int k, Agac *t)
{
    Obek obek = {.n = 0};
    t->n = 0;
    for (int b = 0; b < k; b++) {        /* yapraklar simge sırasıyla */
        int d = dugum_ekle(t, b, siklik[b], -1, -1);
```

```
        obege_ekle(&obek, (Oge){siklik[b], d});
    }
    while (obek.n > 1) {
        Oge x = en_kucugu_cikar(&obek);  /* en küçük iki ağaç */
        Oge y = en_kucugu_cikar(&obek);
        int f = x.anahtar + y.anahtar;
        int z = dugum_ekle(t, -1, f, x.no, y.no);
        obege_ekle(&obek, (Oge){f, z});
    }
    t->kok = en_kucugu_cikar(&obek).no;
}
```

8 · Üç dil karşılaştırması. C++'ta yapılandırılmış bağlama (`auto [fx, x] = obek.top();`) ve `pop` iki ayrı işlemdir; Python'da `heapq.heappop` ikisini birden yapar:

C++ **H12_K05_cekirdek.cpp · satır 145–152**

```
    while (obek.size() > 1) {
        auto [fx, x] = obek.top();       // en küçük iki ağaç
        obek.pop();
        auto [fy, y] = obek.top();
        obek.pop();
        int f = fx + fy;
        int z = dugum_ekle(t, -1, f, x, y);
        obek.push({f, z});
```

Python **H12_K05_cekirdek.py · satır 97–102**

```
    while len(obek) > 1:
        fx, x = heapq.heappop(obek)      # en küçük iki ağaç
        fy, y = heapq.heappop(obek)
        f = fx + fy
        z = dugum_ekle(t, -1, f, x, y)
        heapq.heappush(obek, (f, z))
```

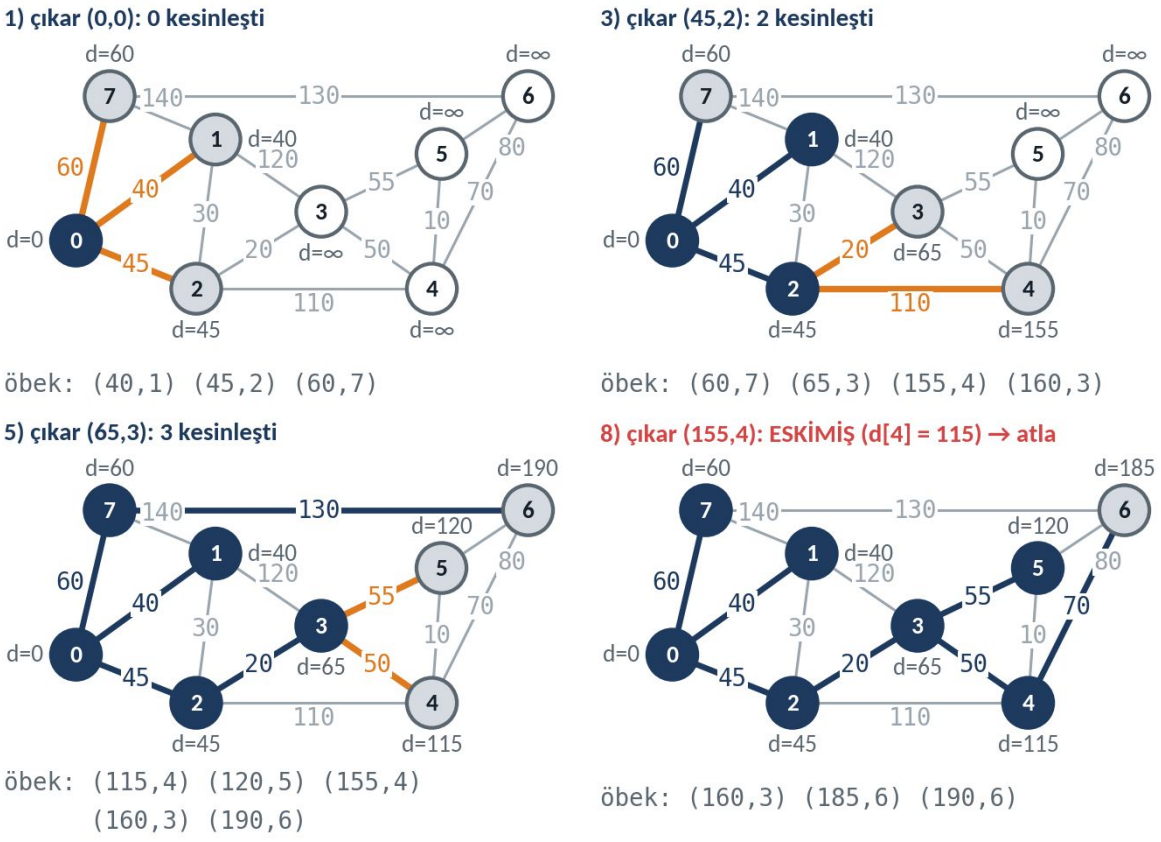
9 · Test. Önek özelliği bütün kod çiftlerinde denetlenir (program: “sağlanıyor”); B(T) iki yoldan hesaplanır; tek simgeli metin ayrı sınanır; Örnek 6 kodlanmış akışı çözüp özgün metinle karşılaştırır. **10 · Deney.** Örnek 6 bir Ömer Seyfettin metninde (52 424 bayt, 72 farklı bayt) 255 393 bit = 31 925 bayt üretir: oran %60,90; ortalama 4,8717 bit/simge, sıfırıncı derece entropi 4,8462 bit/simge. Hiçbir önek kodu entropinin altına inemez (bilgi kuramının kaynak kodlama sınırı); Huffman ondan en çok 1 bit/simge uzaklaşır, burada 0,0255 bit. Ağaç da saklanırsa (256 baytlık kod uzunluğu tablosu + 4 bayt simge sayısı) 32 185 bayt (%61,39). **11 · Varyantlar.** Kanonik Huffman (yalnız kod uzunlukları saklanır), uyarlamalı Huffman, DEFLATE (LZ77 + Huffman); kısa metinde ağacın maliyeti kazancı yer (Atölye 2).

4.5 Dijkstra: tek kaynaklı en kısa yollar

1 · Problem. Kenar ağırlıkları **negatif olmayan** ($w \geq 0$) bir grafta kaynak s'den bütün köşelere en kısa yol uzunluklarını d[v] ve yolları (ebeveyn dizisi) bulun. Örnek 7–8'in girdisi kampüs grafıdır: 0 Ana Kapı, 1 Rektörlük, 2 Kütüphane, 3 Mühendislik, 4 Yemekhane, 5 Yurtlar, 6 Spor Salonu, 7 Otopark; ağırlıklar metre. **2 · Fikir.** Suya atılan taşın dalgaları gibi köşelere **uzaklık sırasıyla** ulaşılır. Uzaklığı **kesinleşen** köşeler kümesi büyütülür: her adımda kesinleşmemişler içinde geçici uzaklığı en küçük olan u kesinleşir (açgözlü seçim); sonra u'dan çıkan her (u, v, w) kenarı **gevşetilir** (relaxation): d[u] + w < d[v] ise d[v] ←

d[u] + w, ebeveyn[v] ← u. En küçüğü bulmanın iki yolu iki sürümü verir: dizide doğrusal tarama (Örnek 7), öbekten çıkarma (Örnek 8).

Şekil 12.11
Dijkstra adımları (Örnek 8; 11 çıkarmadan dördü): kesinleşen küme (siyah), öbekteki köşeler (gri), d değerleri ve öbek içeriği; 8. çıkarma eskimiş bir kayıttır



siyah: kesinleşti · gri: öbekte (d sonlu) · beyaz: d = ∞ · kalın lacivert: ebeveyn kenarı · turuncu: bu adımda gevşetilen

3 · Elle izleme. Öbek (uzaklık, köşe) demetleri tutar; anahtar azaltma yerine **tembel silme** kullanılır (Bölüm 3.5): d[v] küçülünce (d[v], v) yeniden itilir, eski kayıt çıktığında atlanır. Şekil 12.11 çıkarmalardan dördünü gösterir; 3 kayıt eskimiştir: (155,4), (160,3), (190,6). Örneğin 4 önce 2 üzerinden 155 bulunmuş, sonra 3 üzerinden 115'e inmiştir.

SÖZDE KOD · 4 · Sözde kod: öbekli Dijkstra (dizi sürümü: 4–6. satırlar yerine kesinleşmemişler arasında en küçük d'yi tara)

```
1  algoritma DİJKSTRA-ÖBEK(G, s)                    ▷ bütün w ≥ 0
2      her v için: d[v] ← ∞;  ebeveyn[v] ← −1;  kesin[v] ← yanlış
3      d[s] ← 0;  Ö ← {(0, s)}                      ▷ en küçük öbek
4      iken Ö boş değil
5          (du, u) ← ÇIKAR(Ö)
6          eğer kesin[u] ise devam                  ▷ eskimiş kayıt: tembel silme
7          kesin[u] ← doğru                          ▷ çıkınca kesinleşir
8          her (u, v, w) kenarı için
9              eğer değil kesin[v] ve d[u] + w < d[v] ise
10                 d[v] ← d[u] + w;  ebeveyn[v] ← u;  EKLE(Ö, (d[v], v))
```

5 · Doğruluk. Değişmez: Kesinleşen her u için d[u] gerçek en kısa uzaklıktır; kesinleşmemiş v için d[v], yalnız kesinleşmiş köşelerden geçen en kısa yolun uzunluğudur. **Kanıt taslağı** (önde kalma): u seçilsin ve

s'den u'ya d[u]'dan kısa bir P yolu olsun. P kesinleşen kümeden ilk kez bir (x, y) kenarıyla çıkar (x kesin, y değil). x kesinleşirken (x, y) gevşetildiğinden d[y] ≤ d[x] + w(x, y) ≤ |P'nin y'ye kadarki kısmı|; P'nin kalanı **negatif olmadığından** bu ≤ |P| < d[u]. Ama u, kesin olmayanların en küçüğüydü: d[u] ≤ d[y]. Çelişki. Kanıtın tek dayanağı "yolun kalanı negatif olamaz"dır: negatif bir kenar, kesinleşmiş bir köşeye sonradan daha kısa yol açabilir ve kesinleşen köşe bir daha işlenmez (Örnek 9: 2 köşe yanlış; Bölüm 5.2). Tembel silmede köşe öbekten **çıkarken** kesinleşir, itilirken değil (Hata Avcısı 2).

6 · Analiz. Dizi sürümü V kez V hücre tarar: $\Theta(V^2)$ (kampüste 64 = V^2); kuramda yoğun grafta öbekten iyidir (ölçüm: Bölüm 6.3). Öbekli sürümde her başarılı gevşetme bir itmedir: itme = 1 + gevşetme ≤ E + 1 (kampüste 11 = 1 + 10); itme ve çıkarma O(log E) = O(log V) ($E \le V^2$). Toplam O((V + E) log V), seyrekte O(V log V). Ek bellek O(V) ve O(V + E) (Tablo 12.4 ve 12.5).

Tablo 12.4
Dijkstra (dizi) kimlik kartı

ALGORİTMA KİMLİK KARTI	**Dijkstra (dizi)** *(Dijkstra's algorithm, array-based)*
Strateji	Açgözlü: kesinleşmemiş köşelerden d'si en küçük olanı doğrusal taramayla seç, kesinleştir, çıkan kenarlarını gevşet
Girdi önkoşulu	Kenar ağırlıkları negatif değil (w ≥ 0)
Zaman (en iyi · ortalama · en kötü)	$\Theta(V^2)$ · $\Theta(V^2)$ · $\Theta(V^2)$
Ek bellek	O(V)
Özellikler	Kampüste taranan hücre 64 = V^2, kenar inceleme 28 (Örnek 7)
Değişmez / doğruluk	Kesinleşen her u için d[u] gerçek en kısa uzaklıktır; kesinleşmemiş v için d[v], yalnız kesinleşmiş köşelerden geçen en kısa yolun uzunluğudur
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Yoğun graflar ($E = \Theta(V^2)$), komşuluk matrisi, küçük V
Ne zaman kullanılmaz	Büyük seyrek graflar; negatif kenar (Bellman–Ford)

Tablo 12.5
Dijkstra (ikili öbek) kimlik kartı

ALGORİTMA KİMLİK KARTI	**Dijkstra (ikili öbek)** *(Dijkstra's algorithm, binary heap with lazy deletion)*
Strateji	Açgözlü + öncelik kuyruğu: (uzaklık, köşe) çiftlerini öbeğe it
Girdi önkoşulu	w ≥ 0
Zaman (en iyi · ortalama · en kötü)	O((V + E) log V) · O((V + E) log V) · O((V + E) log V)
Ek bellek	O(V + E)
Özellikler	Kampüste itme 11, atlanan 3, en büyük öbek 5 (Örnek 8)
Değişmez / doğruluk	Öbekteki her (d, v) için d ≥ d[v]; d = d[v] olan kayıt güncel, ötekiler eskimiş
Kütüphane karşılığı	C: yok · C++: `std::priority_queue<std::pair<int, int>, std::vector<…>, std::greater<…>>` · Python: `heapq` + `(uzaklık, köşe)` demetleri
Ne zaman kullanılır	Seyrek graflar (yol ağları, E = O(V))
Ne zaman kullanılmaz	Negatif kenar (Bellman–Ford)

7 · C gerçekleştirimi. SONSUZ `INT_MAX`'tır: `int`'in sonsuzu yoktur, en büyük değer gözcüdür ve d[u] + w yalnız d[u] ≠ SONSUZ iken toplanır (`INT_MAX + w` işaretli taşmadır: Hata Avcısı 4). Dizi sürümü (tam hâli `c/H12_07_dijkstra_dizi.c`, satır 262–330) en küçüğü `en_yakin` ile tarar:

C **kavram/H12_K05_cekirdek.c · satır 294–306**

```c
    int u;
    while ((u = en_yakin(g->n, d, kesin)) >= 0) {
        kesin[u] = true;                /* d[u] artık kesin */
        for (int i = 0; i < g->derece[u]; i++) {
            int v = g->komsu[u][i].hedef;
            int w = g->komsu[u][i].agirlik;
            if (!kesin[v] && d[u] != SONSUZ && d[u] + w < d[v]) {
                d[v] = d[u] + w;        /* gevşetme */
                ebeveyn[v] = u;
            }
        }
    }
}
```

Öbekli sürümde (`c/H12_08_dijkstra_obek.c`, satır 362–427; kapasiteyi `realloc` ile ikiye katlayan öbek, atlanan kayıt sayacı) öbekten çıkan u'nun d[u]'su sonludur, koruma gerekmez:

C **kavram/H12_K05_cekirdek.c · satır 320–336**

```c
    while (obek.n > 0) {
        int u = en_kucugu_cikar(&obek).no;
        if (kesin[u]) {
            continue;                   /* eskimiş kayıt */
        }
        kesin[u] = true;
        for (int i = 0; i < g->derece[u]; i++) {
            int v = g->komsu[u][i].hedef;
            int w = g->komsu[u][i].agirlik;
            if (!kesin[v] && d[u] + w < d[v]) {
                d[v] = d[u] + w;
                ebeveyn[v] = u;
                obege_ekle(&obek, (Oge){d[v], v});
            }
        }
    }
}
```

8 · Üç dil karşılaştırması. C'de demet bir `struct`'tır ve sözlük sırası elle yazılır; C++'ta `std::pair`, Python'da demet bu sırayı kendiliğinden verir (Bölüm 8):

C++ **H12_K05_cekirdek.cpp · satır 241–253**

```cpp
    while (!obek.empty()) {
        auto [du, u] = obek.top();
        obek.pop();
        if (kesin[u]) {
            continue;                   // eskimiş kayıt
        }
        kesin[u] = true;
```

```
248         for (const Komsu& k : g.komsu[u]) {
249             int v = k.hedef, w = k.agirlik;
250             if (!kesin[v] && d[u] + w < d[v]) {
251                 d[v] = d[u] + w;
252                 ebeveyn[v] = u;
253                 obek.push({d[v], v});
```

Python H12_K05_cekirdek.py · satır 162–171

```
162     while obek:
163         _, u = heapq.heappop(obek)
164         if kesin[u]:
165             continue                    # eskimiş kayıt
166         kesin[u] = True
167         for v, w in g[u]:
168             if not kesin[v] and d[u] + w < d[v]:
169                 d[v] = d[u] + w
170                 ebeveyn[v] = u
171                 heapq.heappush(obek, (d[v], v))
```

9 · Test. Kâhin dizi sürümüdür: rastgele yönlü graflarda (derece 4) iki sürümün uzaklıkları karşılaştırılır; V = 100'de itme 136, atlanan 36; V = 500'de itme 659, atlanan 159; V = 2 000'de itme 2 673, atlanan 673; hepsi "aynı mı? evet". Sınırlar: ulaşılamayan köşe (d = ∞, ebeveyn −1), tek köşe, sıfır ağırlık. **10 · Deney.** Bölüm 6: bu makinede seyrek grafta (V = 10 000, E = 4V) dizi öbekliden 152 kat yavaş, yoğunda ($E \approx V^2/2$) yalnız 1,23–1,68 kat. **11 · Varyantlar.** Hedef öbekten çıkınca durmak (Alıştırma 12.11); A* (Bölüm 3.5); negatif kenarda Bellman–Ford (Bölüm 13).

4.6 Birleşim–bul (ayrık kümeler)

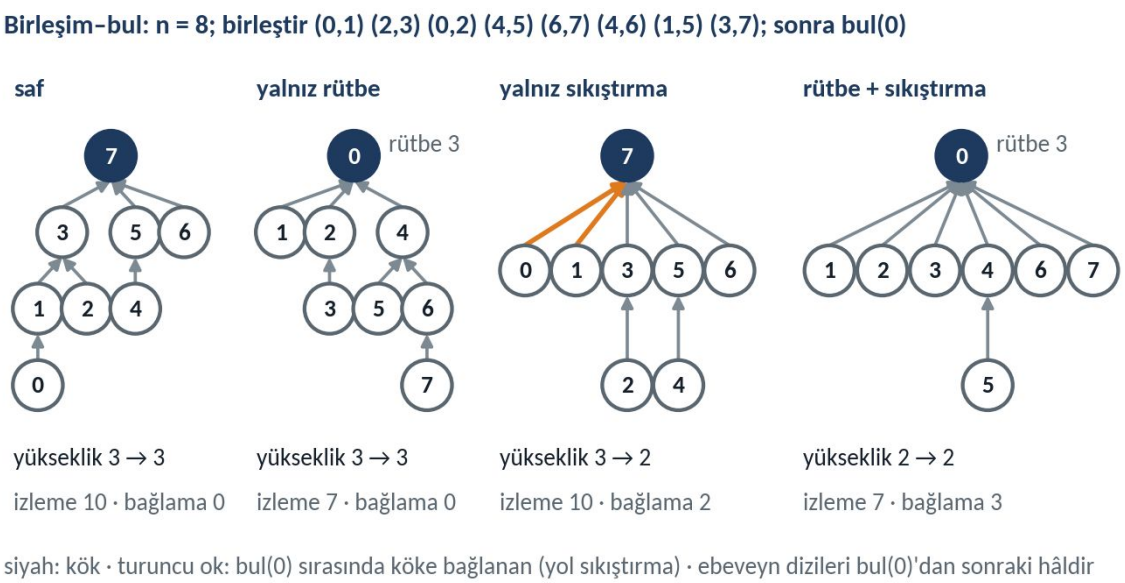
1 · Problem. n öğe başta tek elemanlı kümelerdedir. **bul**(x) x'in kümesinin temsilcisini verir (aynı kümede mi? → bul(x) = bul(y)), **birleştir**(x, y) iki kümeyi birleştirir; ikisi de hızlı olmalıdır. Kruskal bir kenarın döngü kurup kurmayacağını bu soruyla öğrenir. **2 · Fikir.** Her küme bir **ağaçtır**; her öğe yalnız ebeveynini tutar, kök kendi ebeveynidir ve temsilcidir. Bul zinciri köke kadar izler; birleştir bir kökü öbürünün altına bağlar. **Rütbeye göre birleştirme** (union by rank) kısa ağacı uzunun altına bağlar; **yol sıkıştırma** (path compression) bul sırasında geçilen her düğümü doğrudan köke bağlar.

3 · Elle izleme. Örnek 10 aynı sekiz birleştirmeyi dört sürümle yapıp bul(0) çağırır (Şekil 12.12): saf sürümde ağaç 3 düzeye çıkar; yalnız sıkıştırma bul(0)'da 0 ve 1'i köke bağlar; rütbe + sıkıştırmada ağaç baştan kısadır. n = 1000 öğelik zincirde her öğede bul çağrılınca bul başına izleme saf 499,500, yalnız rütbe 0,999, yalnız sıkıştırma 1,997, rütbe + sıkıştırma 0,999 olur.

SÖZDE KOD · 4 · Sözde kod: birleşim-bul (rütbe + yol sıkıştırma)

```
algoritma BUL(x)
    kök ← x;  iken ebeveyn[kök] ≠ kök:  kök ← ebeveyn[kök]
    iken ebeveyn[x] ≠ kök                    ▷ 2. geçiş: yol sıkıştırma
        sonraki ← ebeveyn[x];  ebeveyn[x] ← kök;  x ← sonraki
    döndür kök
algoritma BİRLEŞTİR(x, y)                    ▷ aynı kümedeyse yanlış
    kx ← BUL(x);  ky ← BUL(y);  eğer kx = ky ise döndür yanlış
    eğer rütbe[kx] < rütbe[ky] ise kx ile ky'yi takas et
    ebeveyn[ky] ← kx;  eğer rütbe[kx] = rütbe[ky] ise rütbe[kx] ← rütbe[kx] + 1
    döndür doğru
```

Şekil 12.12
Birleşim–bul ormanları (Örnek 10): dört sürümde bul(0)'dan sonraki ebeveyn ağaçları; turuncu oklar yol sıkıştırmanın köke bağladığı düğümler



5 · Doğruluk. Değişmez: Her düğümün ebeveyn zinciri kendi kümesinin kökünde biter; kökün rütbesi alt ağacının yüksekliğinin üst sınırıdır. Bul kökü değiştirmez, yalnız zinciri kısaltır: üyelik korunur. Rütbe ancak eşit rütbeli iki kök birleşince artar; rütbesi r olan kökün ağacında en az 2^r düğüm vardır, yalnız rütbeyle bile yükseklik ≤ $\log_2 n$. **6 · Analiz.** Saf sürümde bul Θ(n) olabilir; yalnız rütbe O(log n); ikisi birlikte m işlemde toplam O(m α(n)): α, Ackermann fonksiyonunun tersidir ve pratikteki her n için en çok 4'tür (Cormen vd., 2022). Tarjan (1975) sınırın bu algoritma için sıkı olduğunu göstermiştir. Bu **amortize** bir sınırdır (H8): tek bir bul uzun sürebilir ama sıkıştırdığı yol bir daha uzun sürmez. Kanıt kapsam dışıdır; sezgiyi Bölüm 6.4'ün ölçümü verir (Tablo 12.6).

Tablo 12.6
Birleşim–bul kimlik kartı

ALGORİTMA KİMLİK KARTI	**Birleşim–bul (rütbe + yol sıkıştırma)** *(union–find, disjoint-set union)*
Strateji	Her küme bir ağaç
Girdi önkoşulu	Elemanlar 0..n−1 tamsayıları (değilse karma tablosuyla eşlenir, H10)
Zaman (en iyi · ortalama · en kötü)	O(1) · O(α(n)) amortize · O(α(n)) amortize
Ek bellek	O(n)
Özellikler	Rütbe + sıkıştırmada bul başına izleme n = 10^4…10^6 boyunca ≈ 0,79 (Bölüm 6.4)
Değişmez / doğruluk	Her düğümün ebeveyn zinciri kendi kümesinin kökünde biter; kökün rütbesi alt ağacının yüksekliğinin üst sınırıdır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Kruskal'da döngü denetimi
Ne zaman kullanılmaz	Kümeyi bölmek ya da kenar silmek gerekiyorsa (yalnız birleştirir)

7 · C gerçekleştirimi. Tam sürüm (`c/H12_10_birlesim_bul.c`, satır 62–101) dört sürümü bayraklarla seçer ve izlemeleri sayar; yol sıkıştırma **iki geçişli ve yinelemelidir** (özyineleme yok); `birlestir` sözde kodun birebir karşılığıdır:

C **kavram/H12_K05_cekirdek.c · satır 352–364**

```c
352 static int bul(BirlesimBul *b, int x)
353 {
354     int kok = x;
355     while (b->ebeveyn[kok] != kok) {
356         kok = b->ebeveyn[kok];
357     }
358     while (b->ebeveyn[x] != kok) {
359         int sonraki = b->ebeveyn[x];
360         b->ebeveyn[x] = kok;
361         x = sonraki;
362     }
363     return kok;
364 }
```

8 · Üç dil karşılaştırması. Rütbeye göre birleştirmede kök takası C'de geçici değişkenle, C++'ta `std::swap`, Python'da çoklu atamayla yapılır:

C++ **H12_K05_cekirdek.cpp · satır 290–295**

```cpp
290     if (b.rutbe[kx] < b.rutbe[ky]) {
291         std::swap(kx, ky);
292     }
293     b.ebeveyn[ky] = kx;
294     if (b.rutbe[kx] == b.rutbe[ky]) {
295         b.rutbe[kx]++;
```

Python **H12_K05_cekirdek.py · satır 201–205**

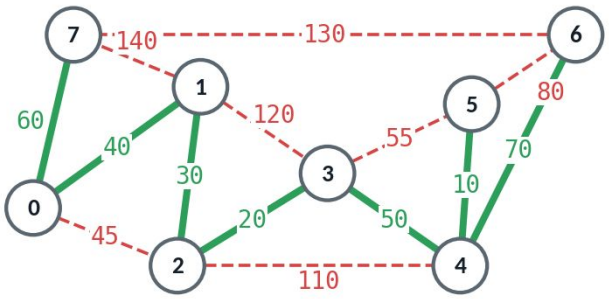
```python
201     if b.rutbe[kx] < b.rutbe[ky]:
202         kx, ky = ky, kx
203     b.ebeveyn[ky] = kx
204     if b.rutbe[kx] == b.rutbe[ky]:
205         b.rutbe[kx] += 1
```

9 · Test. Dört sürüm aynı işlem dizisinde aynı küme sayısını verir (rastgele n = 1000'de dördü de 177); bul(x) = bul(y) yanıtları saf sürümle karşılaştırılır. **10 · Deney.** Bölüm 6.4. **11 · Varyantlar.** Boyuta göre birleştirme, yol yarılama; uygulamalar: Kruskal, bağlı bileşenlerin çevrimiçi sayımı, görüntüde bölge etiketleme.

4.7 Kruskal: en küçük kapsayan ağaç

Şekil 12.13
Kruskal (Örnek 11): kabul edilen kenarlar (yeşil, kalın) ve döngü kurduğu için reddedilenler (kırmızı, kesikli); sağda kenar sırası

Kruskal (kampüs): kenarlar (w, u, v) sırasıyla; 7 kabul, toplam 280




```
 1. ( 10, 4, 5)    KABUL kökler 4, 5
 2. ( 20, 2, 3)    KABUL kökler 2, 3
 3. ( 30, 1, 2)    KABUL kökler 1, 2
 4. ( 40, 0, 1)    KABUL kökler 0, 2
 5. ( 45, 0, 2)    RET: döngü kökler 2, 2
 6. ( 50, 3, 4)    KABUL kökler 2, 4
 7. ( 55, 3, 5)    RET: döngü kökler 2, 2
 8. ( 60, 0, 7)    KABUL kökler 2, 7
 9. ( 70, 4, 6)    KABUL kökler 2, 6
10. ( 80, 5, 6)    RET: döngü kökler 2, 2
11. (110, 2, 4)    RET: döngü kökler 2, 2
12. (120, 1, 3)    RET: döngü kökler 2, 2
13. (130, 6, 7)    RET: döngü kökler 2, 2
14. (140, 1, 7)    RET: döngü kökler 2, 2
```

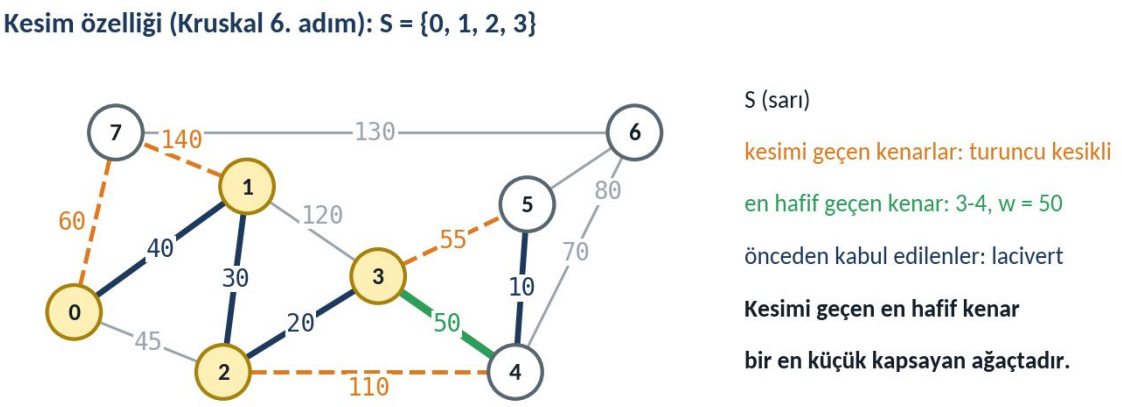
1 · Problem. Bağlı, yönsüz, ağırlıklı bir grafta bütün köşeleri bağlayan, döngüsüz (V − 1 kenarlı) ve toplam ağırlığı en küçük kenar kümesi: **en küçük kapsayan ağaç** (minimum spanning tree). Kampüste bütün binaları en az kabloyla bağlayan fiber ağıdır; Ana Kapı'dan en kısa yollar ağacıyla (4.5) karıştırılmamalıdır: o ağacın toplamı daha büyüktür (340 m ↔ 280 m; Şekil 12.11 ve 12.13). **2 · Fikir.** Kruskal (1956): kenarları (w, u, v) sırasıyla dolaş; uçları **farklı** ağaçlardaysa kabul et ve ağaçları birleştir, aynı ağaçtaysa **reddet** (döngü kurardı). "Aynı ağaçta mı?" birleşim–bulun sorusudur. **3 · Elle izleme.** Şekil 12.13: 7 kabul, 7 ret, toplam 280. Beşinci kenar (45, 0, 2) reddedilir: 0 ve 2, 0-1 ve 1-2 üzerinden zaten aynı ağaçtadır. Dokuzuncu adımda ağaç tamamdır (V − 1 = 7 kenar); erken durdurma kalan beş kenarı incelemezdi.

SÖZDE KOD · 4 · Sözde kod: Kruskal

```
algoritma KRUSKAL(G)
    A ← boş;  her v için tek elemanlı küme {v};  kenarları (w, u, v) sırasına diz
    her (u, v, w) için (sırayla)
        eğer BUL(u) ≠ BUL(v) ise              ▷ döngü kurmaz
            A ← A ∪ {(u, v)};  BİRLEŞTİR(u, v)
    döndür A                                  ▷ |A| = V − 1 (bağlı grafta)
```

5 · Doğruluk. Kesim özelliği: köşeler S ve V − S diye ayrıldığında, kesimi geçen kenarların **en hafifi** bir en küçük kapsayan ağaçtadır. Kanıt taslağı (değiş tokuş): bu e kenarını içermeyen bir en küçük kapsayan ağaç T olsun; T'ye e eklenince oluşan döngü kesimi başka bir e′ kenarıyla da geçer; w(e) ≤ w(e′) olduğundan T − e′ + e de kapsayan ağaçtır ve daha ağır değildir. Kruskal bir kenarı kabul ettiğinde S = u'nun ağacı alınırsa kenar kesimi geçen en hafif kenardır: daha hafif kenarların hepsi incelenmiş ve ya kabul edilmiş ya da kesimi geçmiyordur. Örnek 11 bunu her kabulde denetler; altıncı adımda S = {0, 1, 2, 3}, S'den çıkan en hafif kenarın ağırlığı 50'dir ve bu, o adımda kabul edilen (50, 3, 4) kenarıdır (Şekil 12.14).

Şekil 12.14
Kesim özelliği (Örnek 11, 6. adım): S = {0, 1, 2, 3} (sarı); kesimi geçen kenarlar turuncu kesikli, en hafifi (50, 3, 4) yeşil



6 · Analiz. Sıralama O(E log E); en çok 2E bul ve V − 1 birleştirme neredeyse sabit (α) süreli: toplam O(E log E) = O(E log V). Kampüste bul 28, birleştir 7, işaretçi izleme 17'dir (Tablo 12.7).

Tablo 12.7
Kruskal kimlik kartı

ALGORİTMA KİMLİK KARTI **Kruskal** *(Kruskal's algorithm)*	
Strateji	Açgözlü: kenarları (w, u, v) sırasıyla dolaş
Girdi önkoşulu	Yönsüz graf (bağlı değilse sonuç en küçük kapsayan ormandır)
Zaman (en iyi · ortalama · en kötü)	O(E log E) · O(E log E) · O(E log E)
Ek bellek	O(V + E)
Özellikler	Kampüste 7 kabul, 7 ret, toplam 280 (Örnek 11)
Değişmez / doğruluk	Kabul edilen kenarlar bir ormandır ve bir en küçük kapsayan ağacın alt kümesidir (kesim özelliği)
Kütüphane karşılığı	C: `qsort` yalnız kenar sıralaması için · C++: `std::sort` + `std::tie` · Python: `sorted`
Ne zaman kullanılır	Kenar listesi hazır, seyrek graflar
Ne zaman kullanılmaz	Yönlü graflar (kapsayan dallanma başka bir problemdir)

7 · C gerçekleştirimi. Tam sürüm (`c/H12_11_kruskal.c`, satır 347–408) kenarları (min(u, v), max(u, v), w) biçimine getirip `qsort` ile sıralar ve her kabulde kesimi ve ormanı yazar. Döngü denetimi `birlestir`'in dönüş değerindedir:

kavram/H12_K05_cekirdek.c · satır 400–414

```c
static int kruskal(Kenar *e, int m, int n, Kenar *agac, int *k)
{
    BirlesimBul b;
    bb_kur(&b, n);
    qsort(e, (size_t)m, sizeof e[0], kenar_karsilastir);
    int toplam = 0;
    *k = 0;
```

```
    for (int i = 0; i < m; i++) {
        if (birlestir(&b, e[i].u, e[i].v)) {   /* döngü yoksa kabul */
            agac[(*k)++] = e[i];
            toplam += e[i].w;
        }
    }
    return toplam;
}
```

8 · Üç dil karşılaştırması. Kenar sırası C'de üç anahtarlı karşılaştırıcı, C++'ta `std::tie`'lı lambda, Python'da demetin kendisidir:

C++ H12_K05_cekirdek.cpp · satır 304–311

```
    std::sort(e.begin(), e.end(), [](const Kenar& x, const Kenar& y) {
        return std::tie(x.w, x.u, x.v) < std::tie(y.w, y.u, y.v);
    });
    int toplam = 0;
    for (const Kenar& k : e) {
        if (birlestir(b, k.u, k.v)) {    // döngü yoksa kabul
            agac.push_back(k);
            toplam += k.w;
```

Python H12_K05_cekirdek.py · satır 214–217

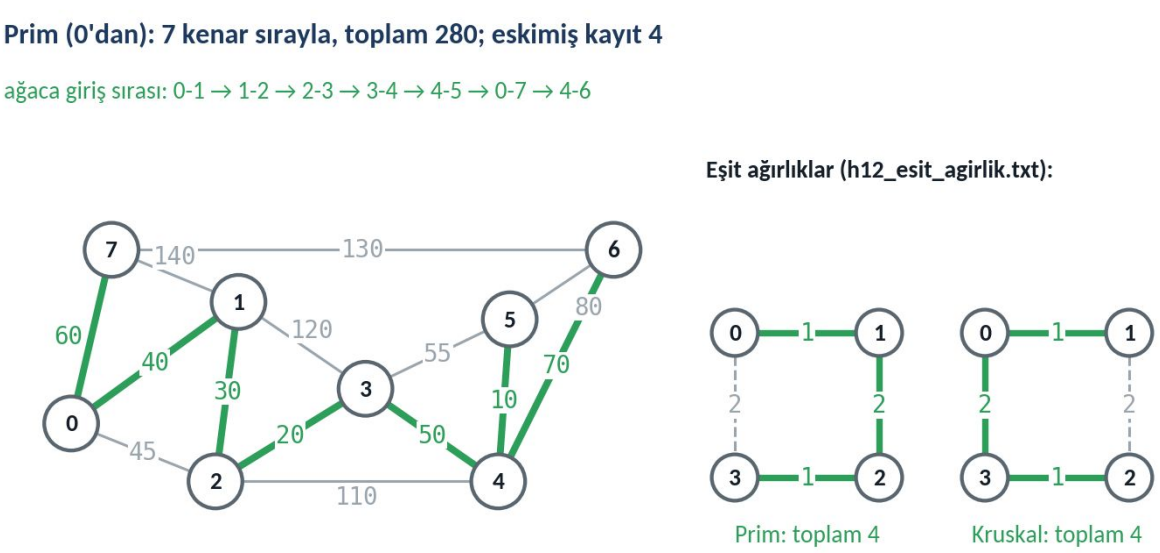
```
    for w, u, v in sorted((w, u, v) for u, v, w in kenarlar):
        if birlestir(b, u, v):           # döngü yoksa kabul
            agac.append((u, v, w))
            toplam += w
```

9 · Test. Kâhin kaba kuvvettir: 3 432 adet 7 kenarlı alt kümeden 1 040 tanesi kapsayan ağaçtır ve en küçük toplam 280; Kruskal en iyidir. Bağlı olmayan grafta sonuç en küçük kapsayan **ormandır**. **10 · Deney.** Bölüm 6.4'ün birleşim–bul ölçümü Kruskal'ın döngü denetiminin maliyetidir. **11 · Varyantlar.** En büyük kapsayan ağaç (sıra ters), k kümeye ayırma (Alıştırma 12.12), Borůvka algoritması; Kruskal'ın makalesi Borůvka'nın (1926) çalışmasına başvurur.

4.8 Prim: ağacı tek köşeden büyütmek

1 · Problem. Aynı problem: en küçük kapsayan ağaç. **2 · Fikir.** Prim (1957) ağacı tek köşeden büyütür: her adımda ağaçtaki köşeleri dışarıdakilere bağlayan kenarların **en hafifini** ekler. Yapısı Dijkstra'nın aynısıdır, yalnız öncelik anahtarı farklıdır: Dijkstra'da d[u] + w (kaynağa uzaklık), Prim'de w (ağaca uzaklık, `anahtar[v]`). Algoritmayı daha önce Jarník (1930) yayımlamıştı (Jarník–Prim). **3 · Elle izleme.** 0'dan kenarlar 0-1, 1-2, 2-3, 3-4, 4-5, 0-7, 4-6 sırasıyla girer; toplam 280, Kruskal'ınkiyle aynı kenarlar; 12 itme, 4 eskimiş kayıt (Şekil 12.15).

Şekil 12.15
Prim (Örnek 12): kampüste ağaca giriş sırası; sağda eşit ağırlıklı grafta Prim ve Kruskal'ın farklı ama eşit toplamlı (4) ağaçları



SÖZDE KOD · 4 · Sözde kod: Prim (öbekle, tembel silme)

```
algoritma PRİM(G, s)
    her v için: anahtar[v] ← ∞;  ebeveyn[v] ← −1;  ağaçta[v] ← yanlış
    anahtar[s] ← 0;  Ö ← {(0, s)}
    iken Ö boş değil
        (a, u) ← ÇIKAR(Ö);  eğer ağaçta[u] ise devam      ▷ eskimiş
        ağaçta[u] ← doğru;  toplam ← toplam + a
        her (u, v, w) kenarı için
            eğer değil ağaçta[v] ve w < anahtar[v] ise     ▷ Dijkstra: d[u] + w
                anahtar[v] ← w;  ebeveyn[v] ← u;  EKLE(Ö, (w, v))
```

5 · Doğruluk. Kesim özelliği, S = o anki ağacın köşeleri: öbekten çıkan geçerli kayıt S'den çıkan en hafif kenardır; ağaç her adımda bir en küçük kapsayan ağacın alt kümesi kalır. **6 · Analiz.** Öbekli sürüm O(E log V) (Tablo 12.8); dizi sürümü $\Theta(V^2)$'dir ve kuramda yoğun graflarda daha iyidir; Dijkstra'da olduğu gibi sınırı ölçüm belirler (Bölüm 6.3).

Tablo 12.8
Prim (ikili öbek) kimlik kartı

ALGORİTMA KİMLİK KARTI **Prim (ikili öbek)** *(Prim's algorithm, Jarník–Prim algorithm)*	
Strateji	Açgözlü + öncelik kuyruğu: ağaçtan çıkan en hafif kenarla yeni köşe ekle
Girdi önkoşulu	Yönsüz, bağlı graf
Zaman (en iyi · ortalama · en kötü)	O(E log V) · O(E log V) · O(E log V)
Ek bellek	O(V + E)
Özellikler	Kampüste Kruskal ile aynı kenarlar, toplam 280; itme 12, atlanan 4 (Örnek 12)
Değişmez / doğruluk	Ağaca giren kenarlar, ağaç köşelerini kapsayan ve bir en küçük kapsayan ağacın alt kümesi olan bir ağaçtır
Kütüphane karşılığı	C: yok · C++: `std::priority_queue` · Python: `heapq`

Ne zaman kullanılır	Komşuluk listesi elde olan graflar
Ne zaman kullanılmaz	Bağlı olmayan graf (yalnız başlangıcın bileşenini kapsar)

7 · C gerçekleştirimi. Tam sürüm (`c/H12_12_prim.c`, satır 355–429) ağaç kenarlarını toplar ve Kruskal'la karşılaştırır. Gevşetme döngüsünde Dijkstra'dan farklı tek satır vurgulanmıştır:

C `kavram/H12_K05_cekirdek.c · satır 437–443`

```c
        for (int i = 0; i < g->derece[u]; i++) {
            int v = g->komsu[u][i].hedef;
            int w = g->komsu[u][i].agirlik;
            if (!agacta[v] && w < anahtar[v]) {  /* Dijkstra: d[u] + w */
                anahtar[v] = w;
                ebeveyn[v] = u;
                obege_ekle(&obek, (Oge){w, v});
```

8 · Üç dil karşılaştırması. Öbek ve demetler Dijkstra'dakiyle aynıdır; fark gevşetme koşulundadır:

C++ `H12_K05_cekirdek.cpp · satır 336–341`

```cpp
        for (const Komsu& k : g.komsu[u]) {
            int v = k.hedef, w = k.agirlik;
            if (!agacta[v] && w < anahtar[v]) {  // Dijkstra: d[u] + w
                anahtar[v] = w;
                ebeveyn[v] = u;
                obek.push({w, v});
```

Python `H12_K05_cekirdek.py · satır 235–239`

```python
        for v, w in g[u]:
            if not agacta[v] and w < anahtar[v]:  # Dijkstra: d[u] + w
                anahtar[v] = w
                ebeveyn[v] = u
                heapq.heappush(obek, (w, v))
```

9 · Test. Kâhin Kruskal'dır. Ağırlıklar farklıysa en küçük kapsayan ağaç **tektir**; eşit ağırlıklarda toplam aynı, ağaç farklı olabilir: `h12_esit_agirlik.txt` grafında Prim 0-1, 1-2, 2-3, Kruskal 0-1, 0-3, 2-3 seçer; toplam ikisinde de 4. Rastgele graflarda (V = 50, E = 200, ağırlık 1..10) toplamlar 20/20 eşit, ağaçlar 19/20 farklı. **10 · Deney.** Maliyet öbekli Dijkstra'nınkiyle aynı yapıdadır (Bölüm 6). **11 · Varyantlar.** Dizi tabanlı Prim ($\Theta(V^2)$); Kruskal ile Prim arasındaki seçimi Tablo 12.9 özetler.

Tablo 12.9
Kruskal ve Prim: aynı problem, iki açgözlü algoritma

	Kruskal	Prim
Büyüttüğü yapı	orman (birçok ağaç birleşir)	tek ağaç (bir köşeden)
Seçim ölçütü	bütün kenarların en hafifi (döngü kurmuyorsa)	ağaçtan çıkan en hafif kenar
Yardımcı yapı; süre	sıralama + birleşim–bul; O(E log E)	öbek; O(E log V) (öbek), $\Theta(V^2)$ (dizi)
Uygun olduğu graf	seyrek; kenar listesi hazır	komşuluk listesi elde; yoğun grafta dizi sürümü
Eşit ağırlıkta ağaç	(w, u, v) sırasına göre	başlangıç köşesine ve (anahtar, köşe) sırasına göre

5. Çözümlü Örnekler

Merdivenin kalan basamakları: Huffman ile gerçek bir dosyanın kodlanması ve çözülmesi (Örnek 6), Dijkstra'nın önkoşulunun neden vazgeçilmez olduğu (Örnek 9) ve açgözlünün **garanti vermediği** bir problem, graf boyama (Örnek 13). Örnek 1 Bölüm 3.2'de, Örnek 2–5 ve 7–12 Bölüm 4'te işlendi.

5.1 Huffman ile kodlama ve çözme: bit tamponu (Örnek 6)

Problem. Örnek 5'in ağacıyla bir metni bit akışına çevirip baytlara yazmak, sonra baytlardan metni geri elde etmek. **Çözüm.** `kodla` her baytın kodunu sırayla tampona ekler; bit i, `tampon[i >> 3]` baytının `0x80 >> (i & 7)` bitidir (en anlamlı bit önce, Bölüm 3.5). Tampon `calloc` ile sıfırlanmış ayrıldığı için yalnız 1 bitler yazılır; çağıran `free` eder:

c/H12_06_huffman_kodlama.c · satır 147–162

```c
    *bayt = (toplam + 7) / 8;
    uint8_t *tampon = calloc(*bayt > 0 ? *bayt : 1, 1);  /* sıfırlarla dolu */
    if (tampon == NULL) {
        return NULL;
    }
    size_t i = 0;
    for (size_t k = 0; k < n; k++) {
        for (const char *c = kodlar[metin[k]]; *c != '\0'; c++) {
            if (*c == '1') {
                tampon[i >> 3] |= (uint8_t)(0x80u >> (i & 7));
            }
            i++;
        }
    }
    return tampon;
}
```

“ey edip adanada pide ye” (23 bayt) için kodlar ␣=101 a=110 d=111 e=00 i=1001 n=1000 p=010 y=011; 68 bit 9 bayta sığar ve son baytın 4 biti dolgudur: 1D 3C AB BE 8D F5 53 CA C0. **Çözme** ağaçta yürümektir: kökten başla, bit 0 ise sola, 1 ise sağa git; yaprağa varınca simgeyi yaz ve köke dön (önek özelliği sayesinde hiçbir kod başka bir kodun ortasında bitmez). Çözücü kaç bit okuyacağını (68) bilmelidir; yoksa dolgu bitlerini veri sanabilir.

Büyük metindeki sonuçlar (oran, entropi, ağacın maliyeti) Bölüm 4.4'te verildi; kodlanmış akışın FNV-1a özeti (H10) üç dilin bit bit aynı akışı ürettiğini doğrular.

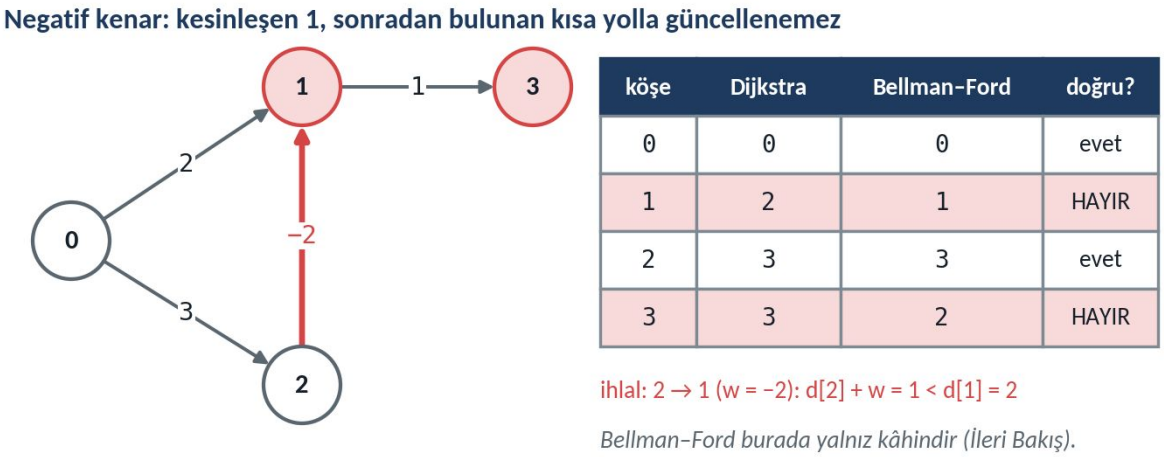
5.2 Negatif kenarda Dijkstra'nın yanlış sonucu (Örnek 9)

Problem. Dört köşeli yönlü grafta bir kenarın ağırlığı negatiftir: 0→1 (2), 0→2 (3), 1→3 (1), 2→1 (−2). Dijkstra 0'dan doğru uzaklıkları bulur mu? **Çözüm.** Hayır: Dijkstra 1'i $d = 2$ ile kesinleştirir; 2 kesinleştiğinde 2 → 1 kenarı $3 + (-2) = 1$ uzunlukta bir yol açar, ama 1 bir daha işlenmez ve onun üzerinden 3'e giden yol da eski değerde kalır (Şekil 12.16). Kâhin, kenar listesini her turda baştan sona gevşeten **Bellman–Ford** algoritmasıdır (yalnız doğrulama için; İleri Bakış). Programın iki algoritmayı köşe köşe karşılaştıran çıktısı (4. bölüm) Şekil 12.16 içindeki tablodur.

Akla gelen “düzeltme”, bütün ağırlıklara en küçük ağırlığın mutlak değerini (2) eklemektir. Bu, yolların **sırasını** değiştirir: k kenarlı bir yol $k \times 2$ ceza alır. Kaydırılmış grafta Dijkstra 0 → 3 için 0 → 1 → 3 yolunu bulur (kaydırılmış uzunluk 7, özgün uzunluk 3), oysa gerçek en kısa uzaklık 2'dir: üç kenarlı doğru yol, iki

kenarlı yanlış yola cezada yenilir. Ders: önkoşul (w ≥ 0) algoritmanın **doğruluk kanıtının** parçasıdır; bozulduğunda program hata vermez, sessizce yanlış sonuç üretir. Bu yüzden Örnek 9 en küçük ağırlığı önceden denetleyip “Dijkstra'nın önkoşulu (w >= 0) SAĞLANMIYOR” uyarısı basar.

Şekil 12.16
Negatif kenar karşı örneği (Örnek 9): 2 → 1 (−2) kenarı kesinleşmiş 1'e daha kısa yol açar; 2 köşede Dijkstra / Bellman–Ford



köşe	Dijkstra	Bellman-Ford	doğru?
0	0	0	evet
1	2	1	HAYIR
2	3	3	evet
3	3	2	HAYIR

ihlal: 2 → 1 (w = −2): d[2] + w = 1 < d[1] = 2

Bellman–Ford burada yalnız kâhindir (İleri Bakış).

5.3 Açgözlü bir sezgisel: Welsh–Powell graf boyama (Örnek 13)

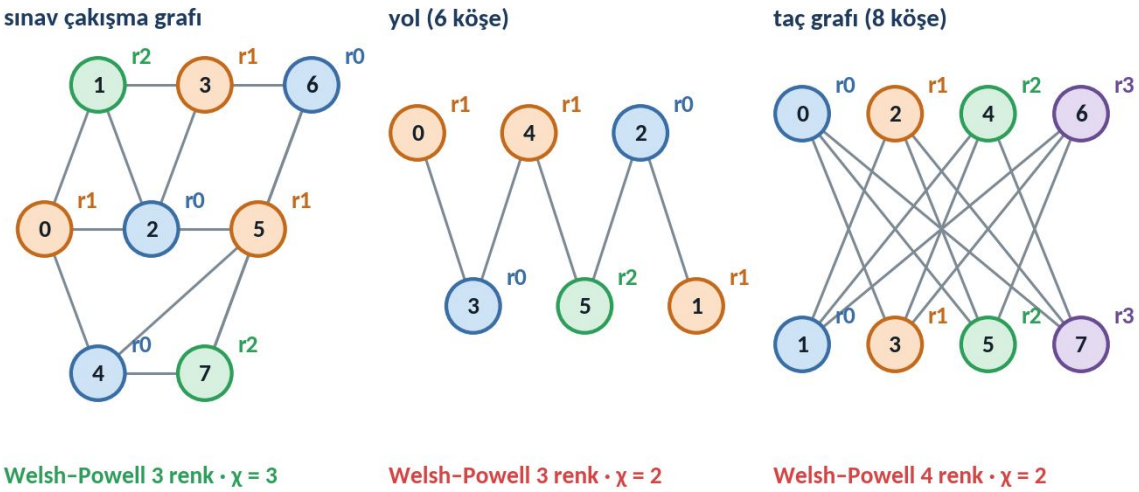
Problem. Sınav programı: ortak öğrencisi olan iki dersin sınavı aynı oturuma konamaz. Dersler köşe, “ortak öğrenci var” ilişkisi kenardır (**çakışma grafı**); oturum ataması bir **graf boyamasıdır** (komşu köşeler farklı renk) ve en az oturum sayısı grafın **kromatik sayısıdır** (χ). Bu problem NP-zordur (H14): bilinen hiçbir algoritma onu her grafta polinom sürede çözmez. **Çözüm (sezgisel).** Welsh ve Powell (1967): köşeleri derece azalan sıraya diz (eşitlikte küçük numara); her renk için listeyi baştan dolaş ve o renkte komşusu olmayan boyanmamış her köşeyi o renge boya. Çekirdek (Örnek 13'ün tam sürümü: `c/H12_13_graf_boyama.c`, satır 283–328):

C `kavram/H12_K05_cekirdek.c · satır 474–492`

```c
    int boyanan = 0, c = 0;
    while (boyanan < g->n) {            /* c rengi için bir tur */
        for (int i = 0; i < g->n; i++) {
            int x = sira[i].no;
            bool uygun = renk[x] == -1;
            for (int j = 0; uygun && j < g->derece[x]; j++) {
                if (renk[g->komsu[x][j].hedef] == c) {
                    uygun = false;      /* c renkli komşu var */
                }
            }
            if (uygun) {
                renk[x] = c;
                boyanan++;
            }
        }
        c++;
    }
    return c;
}
```

Sekiz dersli sınav çakışma grafında Welsh–Powell 3 renk kullanır ve kaba kuvvet (1 377 boyama) χ = 3'ü bulur: burada sonuç en iyidir. Ama iki karşı örnek, yöntemin **garanti vermediğini** gösterir (Şekil 12.17): 6 köşeli bir yol grafında 3 renk (χ = 2), 8 köşeli taç grafında (bütün dereceler eşit; eşitlik kuralı sırayı belirler) 4 renk (χ = 2). İki grafta da başka bir sırayla aynı ilk uyan boyama 2 renk verir: sonuç **sıraya** bağlıdır.

Şekil 12.17
Welsh–Powell (Örnek 13): sınav çakışma grafında en iyi; yol ve taç grafında en iyiden fazla renk (rN: renk numarası)



6. Haftanın Deneyi: Dizi mi, Öbek mi?

6.1 Soru ve düzen

Kurama göre dizi $\Theta(V^2)$, öbek $O((V + E) \log V)$: seyrek grafta öbek, yoğun grafta ($E \approx V^2$) dizi kazanmalıdır; kesişim $V^2 \approx E \log_2 V$'den $E^* \approx V^2 / \log_2 V$ kestirilir. Deney üç soruyu ölçer: (a) V büyürken ölçeklenme; (b) V sabitken E artınca kesişim ve kestirime uzaklığı; (c) birleşim–bulun dört sürümünde işlem başına izleme. Düzen: -02; 1 ısınma + 5 tekrar, ortanca (duvar saati); yayılımı 2 katı aşan ya da duvar/işlemci oranı 1,3'ü aşan ölçüm bir kez yinelenir. Graflar ortak LCG ile üç dilde aynıdır: **rastgele** ($u \to u + 1$ halkası + $D - 1$ rastgele hedef, ağırlık 1..1000; $E = V \cdot D$) ve **kötü durum** (bant biçimli DAG: $u \to v$, $u < v \le u + D$, ağırlık $2(v - u) - 1$; her gevşetme başarılı). Makine: Intel(R) Xeon(R) Processor @ 2.80GHz, 2 çekirdek (sanal makine); gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; Python 3.11.15; toplam süre 103,9 s. Ölçüm sırasında makinede başka işler de çalışıyordu (1 dakikalık yük ortalaması ölçümden önce 1,68, sonra 2,22; 2 çekirdek): süreler "bu makinede ölçülen" değerlerdir; sayaçlar (itme, gevşetme) üç dilde aynıdır (43/43 yapılandırma).

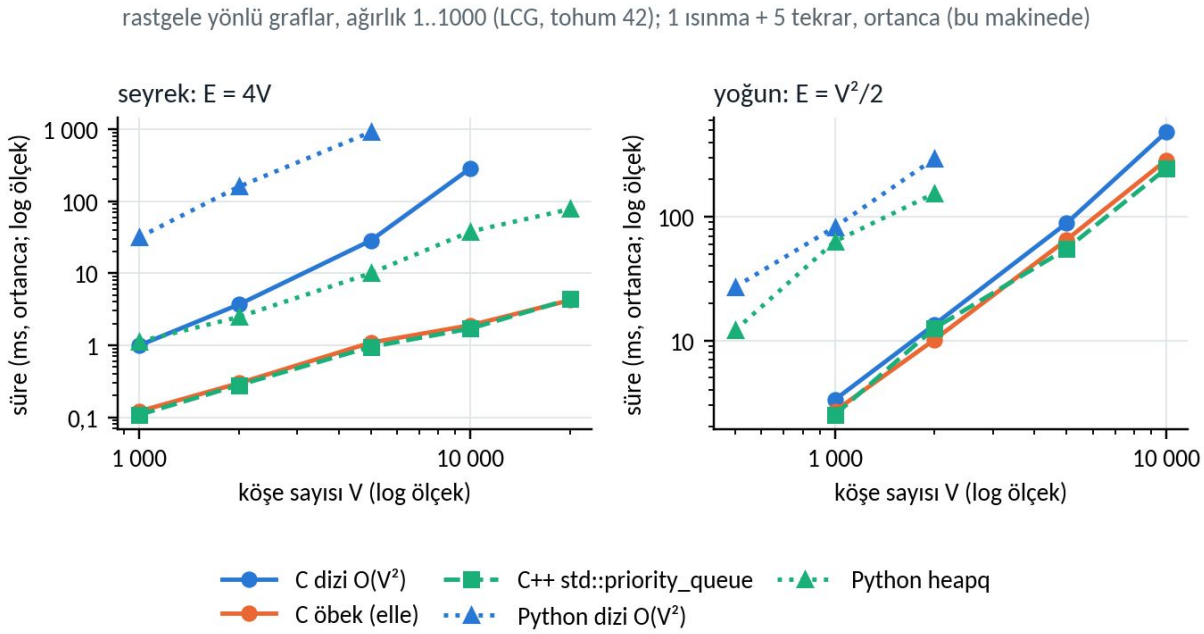
6.2 (a) V ölçeklemesi

Tablo 12.10
*Ölçekleme: ortanca süre (bu makinede; seyrek $E = 4V$, yoğun $E \approx V^2/2$; * kestirim, ölçülmedi; — ölçülmedi)*

V (graf)	C dizi	C öbek	C++ priority_queue	Python dizi	Python heapq
1 000 (seyrek)	0,990 ms	0,120 ms	0,108 ms	31,6 ms	1,12 ms
5 000 (seyrek)	28,5 ms	1,09 ms	0,948 ms	916,0 ms	10,0 ms
10 000 (seyrek)	288,5 ms	1,90 ms	1,71 ms	≈ 3,66 s*	38,0 ms

V (graf)	C dizi	C öbek	C++ priority_queue	Python dizi	Python heapq
20 000 (seyrek)	—	4,23 ms	4,38 ms	—	77,9 ms
500 (yoğun)	—	—	—	27,1 ms	12,1 ms
1 000 (yoğun)	3,32 ms	2,69 ms	2,51 ms	82,6 ms	63,2 ms
5 000 (yoğun)	88,7 ms	65,2 ms	55,0 ms	—	—
10 000 (yoğun)	483,9 ms	287,3 ms	245,5 ms	—	—

Şekil 12.18
Dijkstra'nın dizi ve öbek sürümleri: V ölçeklemesi, solda seyrek (E = 4V), sağda yoğun (E ≈ V²/2); log–log ölçek (bu makinede)



Log–log eğimler (Tablo 12.10, Şekil 12.18; süre ~ V^b; kuram: dizi 2, öbek seyrekte ≈ 1, yoğunda ≈ 2): C dizisi seyrekte 2,43, yoğunda 2,15; C öbeği seyrekte 1,18, yoğunda 2,03; Python heapq seyrekte 1,48. Seyrek grafta dizi ÷ öbek katı C'de V = 1000'de 8,2, V = 10 000'de 152; yoğun grafta 1,23–1,68 kat (öbek yoğunda bile yavaş değil: aşağıda). C dizisinin seyrekte 5 000 → 10 000 katı 10,14 çıktı (kuram 4): olası açıklama d[] ve kesin[] dizilerinin birinci düzey önbelleği aşmasıdır, ama bu bir **hipotezdir**; önbellek sayaçları bu ortamda ölçülmedi. Kütüphane ÷ elle öbek oranı C++'ta ortanca 1,01 (aynı algoritma), Python'da 0,65: heapq C ile yazılmıştır, elle Python öbeğinden hızlıdır.

6.3 (b) Kesişim noktası: kuram ne kadar yanılır?

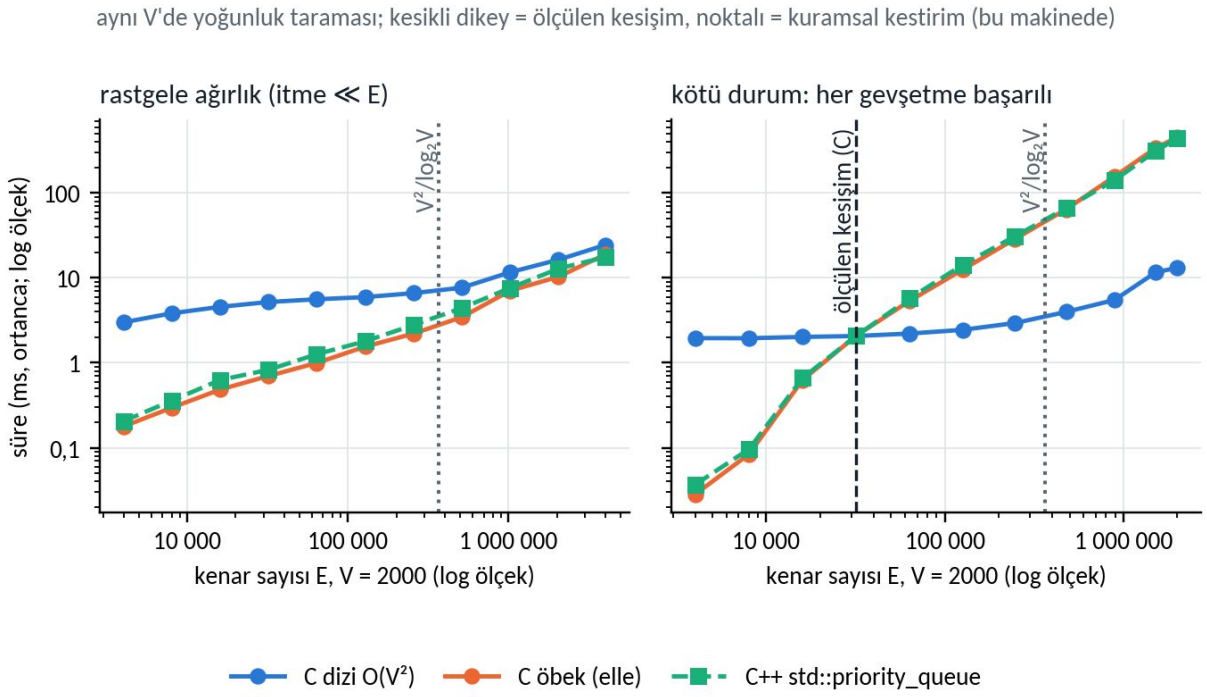
V = 2 000 sabitken D = 2 … V − 1 taranır. Kuramsal kestirim $E^* \approx V^2/\log_2 V$ = 364 771'dir (E/V ≈ 182). Ölçüm iki çok farklı tablo verdi (Şekil 12.19):

- **Rastgele ağırlıklarda kesişim YOK.** C'de öbek/dizi süre oranı en seyrekten en yoğuna 0,059 → 0,772 arasında kalır: en yoğun grafta (E = 3 998 000) bile öbek daha hızlıdır. Nedeni itme sayısıdır: rastgele ağırlıkta başarılı gevşetme azdır, en yoğun grafta itme/E = 0,0032 (kötü durumda 1). Öbeğin maliyeti E log V değil kabaca E + (itme)·log V'dir; O((V + E) log V) bir **üst sınırdır**, her girdinin maliyeti değil.
- **Kötü durumda (bant grafı, itme/E = 1) kesişim var:** C'de E^* ≈ 31 717 (E/V ≈ 15,9), C++'ta 39 020. Kuramsal kestirim (364 771) ölçülenin **11,5 katıdır**: kuram sabitleri yok sayar. Bu makinede

dizinin bir hücreyi taraması ≈ 0,489 ns (ardışık bellek, kolay dal tahmini), öbeğin bir itme·log adımı ≈ 10,829 ns'dir; sabitlerle düzeltilmiş kestirim ($c \cdot E \cdot \log_2 E = a \cdot V^2$ çözümü) 13 194, ölçülenin 0,42 katıdır: kaba model önbellek ve dal etkilerini içermez.

- **Python (V = 500):** kötü durumda kesişim kararlıdır (elle öbek $E^* \approx 5\,074$); rastgele ağırlıkta en yoğun uçta dizi ile öbek başa baştır (son oran 1,13) ve fark ölçüm gürültüsünün içinde kalır: bu nokta "belirsiz" raporlanır, kesişim diye okunmaz.

Şekil 12.19
Kesişim taraması (V = 2 000, C): solda rastgele ağırlık (kesişim yok), sağda kötü durum; kesikli dikey ölçülen kesişim, noktalı kuramsal $V^2/\log_2 V$



Aynı algoritmanın C ve C++ kesişimleri bile farklıdır (E/V ≈ 15,9 ve 19,5): kesişim bölgesinde iki eğri neredeyse paraleldir, küçük bir süre farkı E^*'yi belirgin kaydırır. **Sonuç:** "yoğun grafta dizi" kuralı gevşetmelerin çoğu başarılı olduğunda geçerlidir; sınırı kuram değil ölçüm belirler ve sınır ağırlıkların dağılımına bağlıdır. Bu ölçümde (V = 2 000, bu makinede) rastgele ağırlıklı graflarda öbekli sürüm en yoğun uçta bile hızlıydı; kendi graflarınızda sınırı yeniden ölçün.

6.4 (c) Birleşim–bul: işlem başına izleme

n birleştirme + n bul (toplam 3n bul çağrısı) rastgele işlem dizisinde işlem başına işaretçi izleme (üç dilde aynı sayaç) ve C'de süre: saf sürüm n = 10 000'de 256,9901 izleme/işlem (ağaç yüksekliği 1 058); yalnız rütbe 1,2032 → 1,2593 → 1,2811, yalnız sıkıştırma 1,5345 → 1,7344 → 1,9249, ikisi birden 0,7845 → 0,7905 → 0,7910 ($n = 10^4 \rightarrow 10^5 \rightarrow 10^6$; $\log_2 n$ 13,3 → 19,9). Rütbe + sıkıştırmada izleme pratikte sabittir; bu α(n)'nin bu n'lerde küçük bir sabit olmasıyla tutarlıdır ama α(n)'yi **ölçmez**. Süre ise artar (7,26 → 11,73 → 16,99 ns/işlem): işlem sayısı sabitken dizi önbelleğe sığmaz ve rastgele erişim önbellek kaçırmasına dönüşür (izleme ve süre grafiği web sayfasının Haftanın Deneyi bölümündedir).

6.5 Sınırlılıklar ve kendi ölçümünüz

Tek makine, 2 çekirdekli sanal makine ve eşzamanlı yük: kısa ölçümler gürültülüdür; Python'da (V = 500, D = 128) aynı heapq ölçümü art arda üç kez çalıştırılınca 6,29, 5,51, 11,01 ms verdi (2,0 kat).

Kesişim tek V'de tarandı; birleşim–bul deneyi rastgele işlem dizileri kullanır, en kötü durum dizileri (Tarjan, 1975) ölçülmedi. Kendi makinenizde `python3 kodlar/deney/h12_olcum_calistir.py` ile tekrarlayın ve E*'nin nereye düştüğünü raporlayın.

7. Hata Avcısı

Altı kasıtlı hata, haftanın en sık yanılgılarıdır. Dosyalar `kodlar/hata_avcisi/` altındadır ve başlıklarında "BU DOSYA KASITLI HATA İÇERİR" yazar; her birinin düzeltilmiş sürümü ASan + UBSan ve Valgrind altında temizdir. Belirtiler bu makinede, bu çalıştırmada alınmış gerçek çıktılardır (Tablo 12.11).

Tablo 12.11
Hata Avcısı: hata, belirti, neden, çözüm

#	Hata	Belirti (gerçek çıktı)	Neden	Çözüm
1	Yanlış ölçüt: "en kısa aralık ilk" (Python)	iki kolay örnekte doğru; [(0, 5), (4, 7), (6, 11)] için yalnız (4, 7): 1 aralık, kâhin 2; 500 rastgele örnekte en kısa ilk 479/500	kısa bir aralık iki uzunu birden engelleyebilir; ölçüt kanıtlanmadan ve karşı örnek aranmadan seçildi	bitişe göre sırala; kâhinle rastgele sınama (düzeltilmiş: 500/500)
2	Dijkstra'da köşeyi öbekten çıkarmadan kesin saymak (C)	d[3] = 160 (doğrusu 65); d[4] = 155 (doğrusu 115); d[5] = 165 (doğrusu 120); d[6] = 190 (doğrusu 185)	`kesin[v] = true` gevşetmede: sonradan bulunan kısa yol giremez	köşe öbekten ÇIKINCA kesinleşir
3	Tembel silmede eskimiş kaydı atlamamak (C++)	uzaklıklar doğru ama iş artar: kampüste kenar inceleme 39 (doğrusu 28); yoğun grafta 2 230 452 (doğrusu 499 951, 4,46 kat)	çıkan kaydın eskimiş olduğu denetlenmiyor; köşe her kopyasında yeniden işlenir	`if (kesin[u]) continue;` (ya da du > d[u])
4	SONSUZ için INT_MAX + w taşması (C)	UBSan: `signed integer overflow: 2147483647 + 2`; -O2 ile köşe 4'e d = -2147483647 yazıldı	ulaşılamaz köşe (d = SONSUZ) de seçiliyor ve toplam korumasız; işaretli taşma tanımsız davranıştır	seçimde `d[v] != SONSUZ`, gevşetmede `d[u] != SONSUZ &&`
5	Kruskal'da döngü denetimi yapmamak (C)	7 kenar, toplam 250 < 280; köşe 6 ve 7 hiç bağlanmadı	uçların aynı ağaçta olup olmadığına bakılmıyor; V − 1 kenar ağaç demek değildir	birleşim–bul: `bul(u) != bul(v)` ise kabul
6	Huffman'da tek simgeli metin (C, Python)	C: `0 bit -> 0 bayt; çözülen "" (aynı mı? HAYIR)` (sessiz veri kaybı); Python: `IndexError: bytearray index out of range`	tek düğümlü ağaçta kökten yaprağa yol boştur: kod uzunluğu 0	tek simgede kod "0"; çözücüde tek düğümü ayrıca ele al

Hata 2 ve 3: öbekten çıkma anı. İkisi de Dijkstra'nın değişmezini bozar ama farklı belirti verir. Erken kesinleştirmede (2) bir köşe itilirken kesin sayılır; 3'e önce 1 üzerinden 160 bulunur ve sonra 2 üzerinden gelen 65 uzunluğundaki daha kısa yol kabul edilmez:

C `hata_avcisi/H12_HA2_erken_kesinlestirme.c · satır 88–93`

```c
                int v = KENAR[i][1 - yon], w = KENAR[i][2];
                if (!kesin[v] && d[u] + w < d[v]) {
                    d[v] = d[u] + w;
                    ebeveyn[v] = u;
                    kesin[v] = true;     /* HATA: çıkarılmadan kesin */
                    obege_ekle((Oge){d[v], v});
```

Eskimiş kaydı atlamamakta (3) sonuç doğrudur, bu yüzden hata testlerde görünmez; yalnız sayaçlar ve süre ele verir. Kampüste köşeler `0 1 2 7 3 4 5 4 3 6 6` sırasıyla "kesinleşir" (doğrusu `0 1 2 7 3 4 5 6`): 4, 3 ve 6 iki kez işlenmiştir. Hatalı satır (dosyanın 43. satırı) çıkan kaydın uzaklığını hiç kullanmaz.

Hata 4: sonsuzun taşması. Köşe 3 ve 4 ulaşılamazdır. Seçim döngüsü (dosyanın 34. satırı) SONSUZ uzaklıklı köşeyi de seçer ve gevşetmede (satır 47) `d[u] + w` toplamı `2147483647 + 2` olur. İşaretli taşma C'de **tanımsız davranıştır** (ISO/IEC, 2018): bu derleyicide, bu çalıştırmada sonuç negatif bir "uzaklık"tı (−2147483647). UBSan satırı ve sütunu gösterir:

UBSAN

```
$ gcc ... -fsanitize=undefined H12_HA4_sonsuz_tasmasi.c && ./H12_HA4_sonsuz_tasmasi
H12_HA4_sonsuz_tasmasi.c:47:39: runtime error: signed integer overflow: 2147483647 +
2 cannot be represented in type 'int'
```

Hata 5 ve 6 kısa birer satırdır: Kruskal'da "ilk V − 1 kenarı al" (sıralı kenarlar döngü kurabilir) ve Huffman'da tek simgeli metnin boş kodu (`H12_HA6_tek_simge.c`, satır 67: kökten yaprağa yol boş, derinlik 0). Altıncısı C'de sessizdir: 0 bit yazılır ve çözücü boş metin döndürür; Python sürümü boş tampona erişip `IndexError` ile çöker.

8. Dil Penceresi: Öncelik Kuyruğu, Sonsuz, Demetler, Bitler

Tablo 12.12
Dil Penceresi: haftanın dil farkları (C · C++ · Python)

Öğe	C	C++	Python
En küçük öncelik kuyruğu	yok: H7 öbeğinin genellemesi (struct + karşılaştırıcı, elle)	std::priority_queue<std::pair<int, int>, std::vector<…>, std::greater<…>>	heapq (liste üzerinde en küçük öbek)
Varsayılan yön	(elle yazılır)	en BÜYÜK öbek (std::less)	en KÜÇÜK öbek
Anahtar azaltma	konum dizisiyle elle	yok: tembel silme	yok: tembel silme
Demet / sözlük sırası	struct + kucuk_mu (elle)	std::pair, std::tuple, std::tie	demet (tuple) kendiliğinden
Sonsuz uzaklık	INT_MAX / LLONG_MAX gözcü; d[u] != SONSUZ koruması	std::numeric_limits<int>::max(); has_infinity = false	math.inf (float; yazdırınca inf) ya da aynı tamsayı sabiti
Taşma	işaretli taşma tanımsız davranış	C ile aynı	tamsayı taşmaz
Sıralama aracı	qsort (kararsız; tam sıralama gerekir)	std::sort (kararsız) + std::tie	sorted (kararlı) + anahtar demeti
Bit tamponu	uint8_t dizisi (calloc)	std::vector<std::uint8_t>	bytearray; int.to_bytes / from_bytes

Demetler. Python ve C++ demetleri (çiftleri) **sözlük sırasıyla** karşılaştırır: önce ilk öğe, eşitse ikinci. `heapq` ve `std::priority_queue` (uzaklık, köşe) demetlerini bu yüzden ek kodsuz sıralar; C'de karşılaştırıcı elle yazılır (Kavram 3). Tuzak: ikinci öğe karşılaştırılamayan bir nesneyse (Huffman'da (sıklık, Dugum)), eşit sıklıkta Python `TypeError: '<' not supported between instances of 'Dugum' and 'Dugum'` verir; çözüm benzersiz bir ikincil anahtardır (düğüm no). C++'ta kendi `struct`'ımız için `operator<` ya da karşılaştırıcı yoksa `priority_queue` **derlenmez**. Varsayılan yönler de farklıdır:

Kavram 3'te `operator<` tanımlı `struct` varsayılan (en büyük) öbekte `(4, 2) (4, 1) (1, 5) (1, 3)` sırasıyla çıkar (Alıştırma 12.13).

Sonsuz. C'de `int`'in sonsuzu yoktur: `INT_MAX` gözcü değerdir (Kavram 4: `INT_MAX = 2147483647`) ve gözcüye bir şey eklemeden önce denetlenir (Hata Avcısı 4). Python'da `math.inf` bir `float`'tır, her tamsayıdan büyüktür; yazdırılınca `inf` çıkar, `int(math.inf)` `OverflowError` verir. Örnekler üç dilde aynı çıktı için aynı tamsayı sabitini (`SONSUZ = 2147483647`) kullanır; Python'da `SONSUZ + 5 = 2147483652`: taşma yoktur.

Kütüphane karşılıkları. Haftanın algoritmalarının hiçbiri üç dilin standart kütüphanesinde **hazır değildir**; kütüphaneler yalnız yapı taşlarını (öbek, sıralama) verir. Dosya sonu "Kütüphane karşılığı" notları bu yüzden "algoritma elle" der.

9. Mühendis Gözüyle: Kanıt ya da Karşı Örnek

Açgözlü çözüm ya kanıtlanır ya da karşı örnekle sınanır; sezgisel sonuçlar "en iyi" diye raporlanmaz. Verisi bu haftadandır: "en az çakışan ilk" Örnek 2'nin rastgele örneklerinde 1000/1000 doğruydu, yanlışlığını yalnız kurulmuş bir karşı örnek gösterdi; "en kısa ilk" Hata Avcısı 1'de 479/500 doğruydu ve iki kolay elle örnekte hata görünmedi. Rastgele test **güven** verir, **kanıt** vermez.

- **Kanıtlanmışsa:** kanıtın önkoşullarını program denetlesin (Dijkstra'da w ≥ 0; Örnek 9 en küçük ağırlığı önceden denetler). Önkoşul bozulunca algoritma hata vermez, sessizce yanlış sonuç üretir.
- **Kanıt yoksa:** küçük girdilerde kâhinle (kaba kuvvet) karşılaştırma, karşı örnek arayan rastgele test (Atölye 1'in aracı en küçüğünü bulur); bulunan karşı örnek kalıcı bir teste dönüşür.
- **Sezgiselse:** sonucu bir sınırla ya da kâhinle birlikte raporlayın: Örnek 13 "Welsh–Powell 4 renk, en iyi 2: 2 renk FAZLA" yazar. Büyük girdide bir alt sınır (en büyük klik) verilir; Mini-OBS 2 v2.11'in 2. kabul ölçütü de "en iyi" sözcüğünü yasaklar (Bölüm 10.3).
- **Ölçüm:** büyük-O sabitleri ve girdinin dağılımını gizler: dizi/öbek kesişimi kestirimden bir mertebe sapıyor, rastgele ağırlıkta hiç oluşmuyor (Bölüm 6.3).

10. Mini-OBS 2 Adımı: v2.11 Sınav Programı

Bu hafta (v2.11 belgesi § 10): Mini-OBS'ye **sınav programı** ekliyoruz. Ders kayıtlarından ortak öğrencisi olan dersleri birleştiren bir **çakışma grafı** kuruyoruz; **Welsh–Powell** açgözlü renklendirmesiyle sınavları oturumlara (günlere) atıyoruz ve sonucu bir **alt sınırla** (klik) birlikte raporluyoruz; küçük örneklerde **kaba kuvvetle** en iyi oturum sayısını (kromatik sayı) bulup açgözlü sonucu karşılaştırıyoruz ve açgözlünün **kötü** olduğu bir örnek (taç grafı) gösteriyoruz. Her gün için **aralık bölümleme** (öbekle) en az salon sayısını veriyor; bunun **en iyi** olduğunu derinlik (en çok eşzamanlı sınav) hesabıyla programda doğruluyoruz.

10.1 Gereksinimler

- `sinav.h/.c`: kayıt dosyası (`NO KOD [KOD …]`; numara tek, kod katalogda, satırda aynı kod iki kez yok; hatada satır numarası, çıkış 1); çakışma grafı (her öğrencinin ders çiftleri **karma tablosunda** sayılır; ağırlık = ortak öğrenci; yönsüz CSR).
- Welsh–Powell: derece **azalan**, eşitlikte **küçük ders kodu** (kararlı sıralama); ilk uyan boyama; boyamanın geçerliliğini denetleyen fonksiyon.
- Alt sınır: açgözlü klik; üst sınır: Δ + 1 (ya da Welsh–Powell sınırı). Kaba kuvvet: geri izlemeyle k = 1, 2, … renklendirme (≤ 30 ders, en az 10 ders desteklenmeli).

- Aralık bölümleme: başlangıca göre sıralama + salon bitişlerinin **en küçük öbeği** (v2.6 öbeği); yarı açık aralık (`bitiş ≤ başlangıç` ise salon boş). Derinlik: olay süpürmesi (aynı anda bitiş önce).
- `sinav_programi`: oturumlar, doğrulama, alt/üst sınır, `--kaba` ile χ ve karşılaştırma, oturum oturum salon planı ve "salon = derinlik" doğrulaması.
- Testler: Welsh–Powell ↔ renk renk anlatım (matrisle); χ ↔ ikinci bir kesin yöntem (altküme DP'si ya da bütün renk atamalarının denenmesi); salon ↔ her anda doğrudan sayım; çakışma grafı ↔ matrisle sayım.
- Ölçüm: ders sayısına göre süre ($V = 10^3 \ldots 10^5$, isteğe bağlı 10^6); renk sayısı dağılımı (açgözlü ↔ χ ya da alt sınır).

10.2 İpuçları

- Ders çiftlerini sayarken her öğrencinin ders listesini **sıralı** tutun ve yalnız a < b çiftlerini sayın; anahtar a·V + b tek bir `long long`'dur. Kenar sayısı farklı anahtar sayısıdır, Σ C(k, 2) değil.
- Derece sırasını `qsort` yerine kararlı sayma sıralamasıyla yapın (dereceler 0..Δ); eşitlikte küçük kod kendiliğinden korunur.
- İlk uyan boyamada her köşe için yasak renkleri bir **damga** dizisiyle işaretleyin (`yasak[c] = i + 1`); diziyi her adımda sıfırlamayın.
- Açgözlü sonucu "en iyi" diye yazmayın: klik = açgözlü ise ya da kaba kuvvet doğrularsa yazın; değilse "en iyiden en çok X fazla" deyin.
- Kaba kuvvette **simetri kırma** kullanın: i. köşe en çok "kullanılan en büyük renk + 1"i dener; yoksa her boyamayı k! kez gezersiniz.
- Aralık bölümlemede öbekte **salonların bitişlerini** tutun (sınavların değil); kökte en erken boşalan salon. Karşılaştırmada <= mı < mi olacağını aralığın yarı açık olmasından çıkarın ve arka arkaya iki sınavla test edin.
- Taç grafını kendiniz kurun: Ai–Bj kenarı yalnız i ≠ j iken. Bütün dereceler eşit olduğunda eşitlik kuralı sırayı belirler.

10.3 Kabul ölçütleri

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri (426) ve `test_kahin` (300 deneme) geçer.
2. `./sinav_programi` (örnek kayıtlar) şunları basar: `Çakışma grafı: 48 sınav (öğrencisi olan ders) · 469 kenar`, `Oturum ataması (açgözlü renklendirme; okunan komşu 938): 12 oturum`, `Doğrulama: 469 çakışma kenarının hepsinde iki sınav farklı oturumda: EVET` ve `Alt sınır: 11 derslik klik`; "en iyi" sözcüğü kullanılmaz.
3. Aynı çıktıda `En az salon sayısı: 3 (oturumların en büyüğü; alt sınır 3). Her oturumda salon = en çok eşzamanlı sınav: EVET; aynı salonda çakışan sınav yok: EVET` satırı vardır.
4. `./sinav_programi testler/sinav_kotu.txt --katalog testler/sinav_kotu_katalog.txt --kaba` Welsh–Powell sırasını `S1(3) S2(3) … S8(3)`, 4 oturum, kromatik sayı χ = 2 ve `açgözlü EN İYİ DEĞİL` basar.
5. `./sinav_programi testler/sinav_kucuk.txt --kaba`: açgözlü 5, klik 5, χ = 5, `açgözlü EN İYİ`.
6. `./sinav_programi testler/sinav_bozuk.txt` `satır 4: bilinmeyen ders kodu: BLM999` iletisiyle 1 ile çıkar; `--kaba` 30'dan çok sınavda üstel aramayı yapmaz ve bunu yazar.

7. `./test_kahin --hata-goster renk` ve `--hata-goster sinir` uyuşmazlık ve en küçük karşı örnek basar, 1 ile çıkar; `make asan` ve `make valgrind` temizdir.
8. Ölçüm raporunda: V = 10^3…10^5 (en az dört nokta) Welsh–Powell süresi ve okunan komşu = 2E; aralık bölümlemede salon = derinlik; en az bir yoğunlukta n ≤ 10 derslik en az 100 rastgele grafta açgözlü ↔ χ dağılımı ve yorumu.

10.4 Test senaryoları ve başlangıç noktası

Tablo 12.13
Mini-OBS 2 v2.11 test senaryoları (v2.11 belgesi § 10)

#	Girdi	Beklenen çıktı / davranış
1	taç grafı (8 ders, `sinav_kotu.txt`)	Welsh–Powell 4 oturum, χ = 2 (A grubu / B grubu)
2	aynı graf, sıra A1 A2 A3 A4 B1 …	ilk uyan 2 renk (sıra her şeydir)
3	P4 yolu 0–1–2–3, sıra 0 3 1 2	3 renk; derece sırasıyla 2
4	K_4; C_5; kenarsız 5 köşe	χ = 4; 3; 1
5	yıldız (merkez 2) + yalıtık köşeler	derece sırası 2, sonra küçük numaralar artan
6	[0, 60), [30, 90), [60, 120), [90, 150)	2 salon; [60, 120) 1. salonda ([0, 60)'tan sonra)
7	[0, 1), [1, 2)	1 salon (yarı açık; `<` ile 2 — hata)
8	on aralıklı ders kitabı örneği	derinlik 3 = 3 salon
9	`7 C A B / 8 B A / 9 C B`	kenarlar A–B (2), B–C (2), A–C (1); Σ C(k, 2) = 5
10	`7 A D` (D katalogda yok)	biçim hatası, satır 1, `bilinmeyen ders kodu: D`
11	aynı numara iki satırda	`yinelenen öğrenci numarası`, satır 2
12	`sinav_kucuk.txt`	açgözlü 5 = klik 5 = χ 5

- **Verilir:** kendi v2.10'unuz (ya da referans v2.10), `sinav.h` (bildirimler, sözleşmeler, sahiplik), `ornek_veri/sinav_kayitlari.txt` ve dört test dosyası, `olcer_sinav`'ın ölçüm düzeneği (rastgele graf ve aralık üretimi, sayaç ve süre), v2.11 belgesi Şekil 1'deki taç grafı.
- **Yazılır:** `sinav.c` (kayıt, çakışma grafı, Welsh–Powell, klik, kaba kuvvet, aralık bölümleme, derinlik), `sinav_programi.c`; testler (birim + kâhin); ölçüm ve yorum.
- **Değiştirilmez:** v1.0 kuralları ve menü, v2.9'un karma tablosu, v2.10'un graf ve katalog modülleri, v2.6'nın öbeği, ortak modüller.
- **Meydan okuma:** (a) DSatur (Brélaz, 1979) sezgiselini ekleyip v2.11 belgesi § 8.4'ün dağılımını tekrarlayın; (b) oturum sayısı sabitken (ör. 10 gün) aynı gün iki sınavı olan öğrenci sayısını en aza indiren yerel arama; (c) salon kapasitesi (öğrenci sayısı) eklenince aralık bölümleme neden artık en iyi değildir? Karşı örnek kurun; (d) kaba kuvveti en büyük klikle başlatıp (k = klik boyu) düğüm sayısındaki kazancı ölçün.

11. Alıştırmalar ve Atölyeler

Türler: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez, D) hata bulma, E) Parsons (Parsons & Haden, 2006), F) kod yazma, G) diller arası okuma. Dosyalar `kodlar/alistirmalar/` altındadır; önce **çalıştırmadan** tahmin edin, yazdığınız kodu küçük girdilerde kâhinle karşılaştırın ve C/C++'ı AddressSanitizer ile çalıştırın. Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 12.1 ★☆☆ *Çıktı tahmini*

`alistirmalar/H12_A01_cikti_tahmini.c`, Bölüm 3.2'deki `acgozlu`nun parçaları yazdıran bir eşini iki sistemde dört tutar için çağırır: {25, 10, 5, 1} ile 30 ve 63, {4, 3, 1} ile 6 ve 10; satır biçimi `tutar: paraxadet paraxadet … -> k parça`'dır (yalnız kullanılan paralar yazılır). Dört satırın çıktısını yazın. Hangilerinde sonuç en az parça **değildir**, en iyisi nedir? Farkı Bölüm 3.1'in iki koşuluyla açıklayın.

Alıştırma 12.2 ★★☆ *İzleme*

Yönsüz ağırlıklı grafta (`girdiler/h12_a02_graf.txt`; 6 köşe, 9 kenar) (a) öbekli Dijkstra'yı 0'dan izleyin: her adımda çıkan kayıt (eskimiş mi?), gevşetmeler, d[] ve öbek. Kaç eskimiş kayıt oluşur? (b) Kruskal'ın kararlarını (w, u, v) sırasıyla yazın; Prim (0'dan) aynı ağacı mı verir? Doğrulama: `./H12_08_dijkstra_obek ../girdiler/h12_a02_graf.txt 0` ve `./H12_11_kruskal ../girdiler/h12_a02_graf.txt`.

Alıştırma 12.3 ★★☆ *İzleme*

“mississippi” için Huffman ağacını elle kurun (Örnek 5'in eşitlik kuralıyla): sıklıklar, birleştirmeler, kod tablosu, toplam bit ve sabit 8 bite göre oran (ağaç hariç). `./H12_A14_huffman_metin mississippi` ile doğrulayın. Eşitlik kuralı farklı olsaydı kodlar ve toplam bit değişir miydi?

11.2 Karmaşıklık (B)

Alıştırma 12.4 ★★☆ *Karmaşıklık*

(a) Aralık bölümleme neden O(n log n)'dir; derinlik d iken öbek en çok kaç öğe tutar? (b) Tembel silmede öbek en çok kaç kayıt tutar, log E neden O(log V)'dir? (c) V = 10 000, E = 40 000 için $V^2 \div (V + E)\cdot\log_2 V$ oranını Bölüm 6.2'de ölçülen katla (152) karşılaştırın; yoğun grafta ($E = V^2/2$) ölçülen kat (1,68) kuramdan neden farklıdır?

Alıştırma 12.5 ★★☆ *Karmaşıklık*

Kruskal'ın süresini sıralama, 2E bul ve V − 1 birleştirme diye ayırın. Birleşim–bul (a) saf, (b) yalnız rütbeli, (c) rütbe + sıkıştırmalı iken en kötü toplam ve baskın parça nedir? Kenarlar sıralı gelirse ne değişir?

11.3 Değişmez yazma (C)

Alıştırma 12.6 ★★☆ *Değişmez yaz*

Örnek 3'teki `bolumle` (`c/H12_03_aralik_bolumleme.c · satır 136–169`) için döngünün her adımının başında doğru olan bir değişmez yazın (öbekte ne var?). Ondan, yeni salon açıldığı anda **derinliğin** salon sayısından az olamayacağını ve algoritmanın en az salonu kullandığını çıkarın.

11.4 Hata bulma (D)

Alıştırma 12.7 ★★☆ *Hata bul*

Bu “Prim” kampüs grafında bir kapsayan ağaç buluyor ama toplamı 340: Kruskal'ın sonucuyla (Örnek 11) uyuşmuyor. Hatalı satırı bulun ve Prim ile Dijkstra'nın **anahtar** farkıyla açıklayın. Hatalı program hangi ağacı buluyor?

alistirmalar/H12_A06_hata_bul_prim.c · satır 48–54

```
48          for (int v = 0; v < V; v++) {
49              int w = W[u][v];
50              if (w > 0 && !agacta[v] && anahtar[u] + w < anahtar[v]) {
51                  anahtar[v] = anahtar[u] + w;
52                  ebeveyn[v] = u;
53              }
54          }
```

Alıştırma 12.8 ★★☆ *Hata bul*

Kesirli sırt çantası Örnek 4'ün altı yüküyle 60 kg için 330 buluyor; kâhin (Örnek 4) daha büyük bir değer veriyor. Hatalı satırı bulun; hata neden yalnız bazı girdilerde görünür? İki yüklü en küçük karşı örneği kurun ve kayan noktasız düzeltin.

alistirmalar/H12_A07_hata_bul_kesirli.py · satır 15–26

```
15  def doldur(yukler, kapasite):
16      sira = sorted(yukler, key=lambda y: y[2] // y[1], reverse=True)
17      toplam = Fraction(0)
18      kalan = kapasite
19      for ad, agirlik, deger in sira:
20          if kalan == 0:
21              break
22          alinan = min(agirlik, kalan)
23          toplam += Fraction(deger * alinan, agirlik)
24          kalan -= alinan
25          print(f"  {ad}: {alinan}/{agirlik} kg")
26      return toplam
```

11.5 Parsons (E)

Alıştırma 12.9 ★☆☆ *Parsons*

`kruskal(Kenar *e, int m, int n, Kenar *agac, int *k)` gövdesinin satırları karıştırıldı, girintileri silindi; iki satır çeldiricidir. Doğru sırayı ve girintiyi yazın; çeldiriciler neden yanlış? Gövdeyi `kavram/H12_K05_cekirdek.c`'deki `kruskal`'ın yerine koyup denetleyin.

alistirmalar/H12_A08_parsons_kruskal.c · satır 10–23

```
toplam += e[i].w;
for (int i = 0; i < m; i++) {
}
qsort(e, (size_t)m, sizeof e[0], kenar_karsilastir);
agac[(*k)++] = e[i];
b.ebeveyn[e[i].u] = e[i].v;
if (birlestir(&b, e[i].u, e[i].v)) {
*k = 0;
}
return toplam;
BirlesimBul b;
```

```
if (e[i].w < toplam) {
int toplam = 0;
bb_kur(&b, n);
```

11.6 Kod yazma (F)

Alıştırma 12.10 ★☆☆ *Kod yaz*

En az noktayla delme. Yarı açık aralıklar [b, e) verildiğinde, her aralığın en az bir noktayı içerdiği (b ≤ p < e) en az sayıda tamsayı noktası bulun. Örnek 2'nin sıralamasını kullanan bir açgözlü strateji önerin, C'de yazın ve Örnek 2'nin on aralığında (A [4, 7), B [0, 3), C [3, 8), D [1, 4), E [7, 10), F [9, 12), G [6, 9), H [11, 14), I [2, 6), J [8, 13)) çalıştırın. Kâhin: 0 … 14 aralığındaki nokta kümelerini deneyen kaba kuvvet. Nokta sayısı ile en çok çakışmasız aralık sayısı arasındaki ilişkiyi açıklayın (önde kalma). C++ ve Python sürümleri de.

Alıştırma 12.11 ★★☆ *Kod yaz*

Hedefte erken dur. Örnek 8'in öbekli Dijkstra'sını, yalnız s → t uzaklığı istendiğinde t öbekten çıktığı anda duracak biçimde değiştirin. Durmanın neden doğru olduğunu Dijkstra'nın değişmeziyle gösterin. Kampüs grafında 0 → 3 ve 0 → 6 için kesinleşen köşe sayısını tam çalıştırmayla karşılaştırın. t'ye ulaşılamıyorsa ne olur? Gevşetme sırasında t'ye ilk kez bir değer yazıldığında durmak neden **yanlıştır**?

Alıştırma 12.12 ★★★ *Kod yaz*

k kümeye ayırma. Kruskal'ı bileşen sayısı k olunca durdurun: elde edilen k küme, farklı kümelerdeki iki köşe arasındaki en küçük uzaklığı (**aralık**, spacing) en büyük yapan ayrımdır. (a) Kampüs grafında k = 3 için kümeleri ve aralığı bulun. (b) Aralığın, Kruskal'ın bir sonraki kabul edeceği kenarın ağırlığı olduğunu gösterin. (c) Doğruluğu kesim özelliğine benzer bir değiş tokuş argümanıyla gerekçelendirin; küçük rastgele graflarda kaba kuvvetle (bütün k bölmeleri) sınayın.

11.7 Diller arası okuma (G)

Alıştırma 12.13 ★★☆ *Diller arası okuma*

Aynı dört (uzaklık, köşe) kaydı C++'ta varsayılan `std::priority_queue`'ya, Python'da `heapq`'ya itilip çıkarılıyor (eşi: `alistirmalar/H12_A12_oncelik_kuyrugu.py`). İki programın çıktısını çalıştırmadan yazın. Hangisi Dijkstra'nın beklediği sırayı verir? C++ sürümünü üç yolla düzeltin: `std::greater`, anahtarı eksiyle itmek, kendi karşılaştırıcınız. Eksiyle itmenin INT_MIN için tehlikesi nedir?

alistirmalar/H12_A12_oncelik_kuyrugu.cpp · satır 16–25

```
16      std::priority_queue<std::pair<int, int>> pq;
17      pq.push({45, 2});
18      pq.push({40, 1});
19      pq.push({60, 7});
20      pq.push({40, 3});
21      while (!pq.empty()) {
22          auto [d, v] = pq.top();
23          pq.pop();
24          std::cout << '(' << d << ',' << v << ") ";
25      }
```

11.8 Atölye 1 · Açgözlü mü, değil mi? On problem kartı (gruplar, ≈ 30 dk)

Her grup her kart için: (1) stratejiyi bir cümleyle yazın; (2) **kanıt** (değiş tokuş, önde kalma) ya da **karşı örnek** arayın; (3) en küçük karşı örneği ya da kanıt taslağını tahtaya yazın. `alistirmalar/H12_A15_karsi_ornek_ara.py` bir stratejiyi küçük rastgele örneklerde kaba kuvvetle karşılaştırır ve en küçük karşı örneği yazar; şimdilik aralık çizelgeleme için iki strateji içerir: kendi stratejinizi `STRATEJILER`'e ekleyin, öteki kartlar için kaba kuvveti uyarlayın.

Tablo 12.14
Atölye 1 problem kartları

Kart	Problem	Önerilen açgözlü strateji
1	Para üstü, sistem {1, 6, 9}; en az parça	sığan en büyük parayı al
2	Para üstü, sistem {1, 2, 5, 10, 20, 50, 100, 200}; en az parça	sığan en büyük parayı al
3	Tek oda; toplantıların **toplam süresini** en büyük yapan çakışmasız seçim	en uzun toplantı önce
4	Kesirli sırt çantası (Örnek 4'ün yükleri)	en değerli yük önce
5	0/1 sırt çantası; **bütün yüklerin ağırlığı eşit**	en değerli yük önce
6	Tek makine; her işin süresi ve son teslim anı var; en büyük gecikmeyi en aza indir	son teslim anı en erken iş önce
7	Tek kasa; müşterilerin bekleme sürelerinin toplamını en aza indir	işlem süresi en kısa müşteri önce
8	Sınav programı (graf boyama); en az oturum	derece azalan sırayla ilk uygun renk
9	Kampüste A binasından B binasına en kısa yol	bulunduğun binadan en kısa yolu izle
10	n sıralı dosyayı ikişer ikişer birleştir; iki dosyayı birleştirmenin maliyeti boylarının toplamı; toplam maliyet en az	her adımda en kısa iki dosyayı birleştir

Aracın başlangıç çıktısı: “en erken biten: 2000 örnekte karşı örnek bulunamadı (bu bir kanıt değildir)” ve “en kısa ilk: karşı örnek (3 aralık) [(0, 5), (4, 6), (5, 10)]: strateji 1, en iyi 2”. **Tartışma.** Karşı örnek bulunamaması kanıt değildir (aracın ilk satırı bunu söyler). Kanıtlanan kartlarda hangi kanıt tekniği işe yaradı? Yanlış çıkan kartlarda doğru çözüm hangi yöntemle bulunur (kaba kuvvet, dinamik programlama, NP-zor)? Kart 10'un Huffman ile ilişkisi nedir?

11.9 Atölye 2 · Huffman ile kendi adınız ve bir paragraf (bireysel + eşli, ≈ 25 dk)

ALGORİTMA ATÖLYESİ · **Elle ağaç, programla doğrulama**

- **Adınız.** Adınızı ve soyadınızı (Türkçe karakterleriyle) bir metin olarak alın. Önce UTF-8 baytlarını sayın: ç, ğ, ı, ö, ş, ü ikişer bayttır ve bu iki baytın her biri ayrı bir “simge”dir. Sıklık tablosunu, birleştirme adımlarını (öbekte (sıklık, düğüm no)) ve kod tablosunu kâğıtta kurun; toplam biti hesaplayın.
- **Doğrulama.** `./H12_A14_huffman_metin "Adınız Soyadınız"` (ya da Python eşi) aynı eşitlik kuralıyla adımları, kod tablosunu ve toplam biti basar (`alistirmalar/H12_A14_huffman_metin.c`). Elle ağacınız farklıysa hangi adımda ayrıldınız: eşitlik mi, sıklık mı?
- **Paragraf.** `girdiler/H12_06_huffman_kodlama.txt`'den (Ömer Seyfettin öyküleri) bir paragraf seçip dosyaya yazın ve `< paragraf.txt` ile çalıştırın. Oranı Örnek 6'nın bütün metin için bulduğu oranla karşılaştırın (Bölüm 5.1). Ağacı saklamanın maliyetini (256 baytlık kod uzunluğu tablosu) ekleyince paragraf hâlâ küçülüyor mu? Kaç baytlık metinden sonra kazanç başlar?

- **Eşli.** Eşinize yalnız kod tablonuzu ve bit dizinizi verin; eşiniz çözsün. Tablo yerine yalnız kod **uzunlukları** verilseydi çözülebilir miydi (DEFLATE'in yaptığı gibi; Deutsch, 1996)?

11.10 Atölye 3 · Kampüs haritası: Dijkstra ve fiber kablo (eşli, ≈ 30 dk)

Kurgusal bir kampüs: 10 bina, 18 yol (`girdiler/h12_kampus_atolye.txt`; ağırlık = yol uzunluğu, metre). Binalar: 0 Ana Kapı, 1 Rektörlük, 2 Merkezi Kütüphane, 3 Mühendislik Fakültesi, 4 Fen-Edebiyat Fakültesi, 5 Yemekhane, 6 Kültür Merkezi, 7 Spor Kompleksi, 8 Yurtlar, 9 Teknopark. Haritayı dosyadaki yol listesinden kâğıda çizerek başlayın.

ALGORİTMA ATÖLYESİ · **İki soru, bir harita**

- **Öğrenci A (yaya).** Ana Kapı'dan (0) her binaya en kısa yürüme mesafesini öbekli Dijkstra ile elle bulun: öbek paneli, d[] ve ebeveyn tablosu. En kısa yollar ağacını haritaya çizin.
- **Öğrenci B (fiber).** Bütün binaları birbirine bağlayan en kısa toplam uzunlukta fiber kablo ağını (yollar boyunca döşenir) Kruskal ile bulun; reddettiğiniz her kenarın kurduğu döngüyü yazın. Prim ile (0'dan) aynı toplamı bulduğunuzu denetleyin.
- **Birlikte.** İki ağacı karşılaştırın: aynı mı, toplam uzunlukları ne? Ana Kapı'dan Teknopark'a fiber ağı üzerinden giden yol en kısa yürüme yolu mu? Sonuçları `./H12_08_dijkstra_obek ../girdiler/h12_kampus_atolye.txt 0` ve `./H12_11_kruskal ../girdiler/h12_kampus_atolye.txt` ile doğrulayın.
- **Ya şöyle olsaydı?** Bir yol (ör. 5–6) kazı nedeniyle kapansa hangi algoritmayı baştan çalıştırmak zorunda kalırsınız? Bütün yolların uzunluğuna 50 m eklenirse en kısa yollar ağacı ve fiber ağı değişir mi (Örnek 9'daki kaydırma tartışmasıyla karşılaştırın)?

12. Kendini Değerlendirme

- ☐ Açgözlü seçim özelliği ile en iyi alt yapıyı açıklayabiliyor, karşı örnek kurabiliyorum.
- ☐ Aralık çizelgelemeyi değiş tokuşla, bölümlemeyi derinlik alt sınırıyla kanıtlayabiliyorum.
- ☐ Kesirli sırt çantasını kayan noktasız çözebiliyor, 0/1 sürümde ölçütü çürütebiliyorum.
- ☐ Huffman ağacını öbekle kurup bit tamponuyla kodlayıp çözebiliyor, oranı hesaplayabiliyorum.
- ☐ Dijkstra'yı dizi ve öbekle yazıp değişmezini ve negatif kenar hatasını açıklayabiliyorum.
- ☐ Birleşim–bulu, Kruskal'ı ve Prim'i yazıp kesim özelliğiyle gerekçelendirebiliyorum.
- ☐ Sezgisel bir sonucu “en iyi” demeden, kâhin ya da alt sınırla raporlayabiliyorum.
- ☐ Dizi ile öbek arasında ölçümle seçim yapabiliyor, kestirimin neden saptığını açıklayabiliyorum.

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Açgözlünün yetmediği yerde ve ötesinde**

- **H13 · Dinamik programlama.** Açgözlünün yanıldığı {1, 3, 4} para üstü (Örnek 1) ve 0/1 sırt çantası (Örnek 4: açgözlü 160, en iyi 220) gelecek haftanın giriş sorusudur: en iyi alt yapı var, açgözlü seçim özelliği yok; DP bütün alt problemleri tabloda çözer.
- **Bellman–Ford ve Floyd–Warshall.** Negatif kenarlarda tek kaynaktan (Bellman–Ford, $O(V \cdot E)$, negatif döngüyü de bulur) ve bütün çiftler arası (Floyd–Warshall, $O(V^3)$, bir DP) en kısa yollar; Algoritma Analizi dersinde.
- **Navigasyon.** A*, Dijkstra'nın anahtarına hedefe kalan uzaklığın iyimser kestirimini ekler (Hart vd., 1968; Bölüm 3.5); Yapay Zekâ dersinin arama konusudur.

- **Veri sıkıştırma ve bilgi kuramı.** DEFLATE biçimi LZ77 ile Huffman kodlamasını birleştirir (Deutsch, 1996); ZIP arşivleri ve PNG görüntüleri bu biçimi kullanır. Entropi kayıpsız sıkıştırmanın alt sınırıdır (Bölüm 4.4).

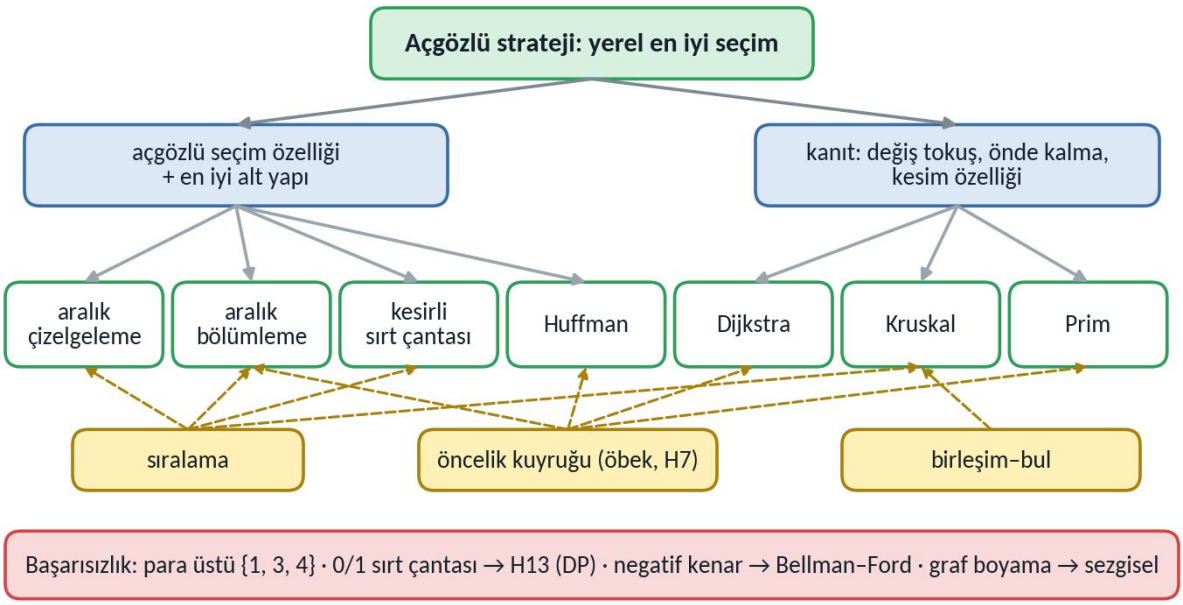
MERAK KUTUSU · **Açgözlü ne zaman kesin işe yarar? Matroidler**

Kruskal'ın neden her grafta en iyi olduğunun genel bir açıklaması vardır. Bir E kümesi ve "bağımsız" denen alt kümelerinin ailesi şu iki özelliği taşısın: (1) bağımsız bir kümenin her alt kümesi bağımsızdır; (2) **değiş tokuş özelliği:** A ve B bağımsızsa ve |A| < |B| ise B'de, A'ya eklendiğinde A'yı bağımsız bırakan bir öğe vardır. Böyle bir yapıya **matroid** denir ve her matroidde "ağırlığa göre sırala, bağımsızlığı bozmayanı al" açgözlüsü en ağır bağımsız kümeyi bulur.

Bir grafın döngüsüz kenar kümeleri (ormanlar) bir matroiddir; Kruskal bu açgözlünün ta kendisidir (en hafif kenardan başlayarak). Soru: aralık çizelgelemede "çakışmasız aralık kümeleri" bir matroid midir? İpucu: A = {[0, 10)} ve B = {[0, 5), [5, 10)} için değiş tokuş özelliğini deneyin. Değilse, açgözlü orada neden yine de doğru çalışıyor? (Matroid yeterli bir koşuldur, gerekli değil.)

14. Özet, Kavram Haritası ve Terim Sözlüğü

Şekil 12.20
Hafta 12 kavram haritası



- Açgözlü algoritma her adımda o an en iyi görünen seçimi yapar ve geri dönmez. Doğru olması için **açgözlü seçim özelliği** ve **en iyi alt yapı** gerekir; doğruluk değiş tokuş, önde kalma ya da kesim özelliğiyle kanıtlanır, kanıt yoksa karşı örnek aranır.
- Aralık çizelgeleme (en erken biten), aralık bölümleme (öbek; salon = derinlik), kesirli sırt çantası (birim değer) ve Huffman (en küçük iki sıklık) kanıtlı açgözlülerdir; para üstü ({1, 3, 4}), 0/1 sırt çantası ve graf boyama açgözlünün yetmediği problemlerdir.
- Dijkstra kesinleşen kümeyi büyütür ve gevşetir; değişmezi $w \geq 0$ varsayar. Dizi $\Theta(V^2)$, öbek (tembel silme) $O((V + E) \log V)$; bu makinede ($V = 2\,000$) kesişim kuramın $V^2/\log_2 V$ kestiriminden 11,5 kat küçük çıktı ve rastgele ağırlıklarda hiç oluşmadı.

- Kruskal (kenar sırasıyla + birleşim–bul) ve Prim (öbekle büyüyen ağaç) aynı toplamı bulur; ikisi de kesim özelliğine dayanır. Rütbe + yol sıkıştırmalı birleşim–bulda işlem başına işaretçi izleme pratikte sabittir (O(α(n)) amortize); süre ise önbellek etkisiyle n'le artar (Bölüm 6.4).

Tablo 12.15
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
açgözlü algoritma; açgözlü seçim özelliği; en iyi alt yapı	greedy algorithm; greedy-choice property; optimal substructure	iki koşul birlikte
değiş tokuş argümanı; önde kalma; karşı örnek	exchange argument; greedy stays ahead; counterexample	kanıt teknikleri
aralık çizelgeleme; aralık bölümleme; derinlik	interval scheduling; interval partitioning; depth	salon = derinlik
kesirli / 0-1 sırt çantası; birim değer	fractional / 0-1 knapsack; value density	0/1: H13
önek kodu; Huffman kodlaması; entropi	prefix code; Huffman coding; entropy	önek özelliği
öncelik kuyruğu; öbek; tembel silme; anahtar azaltma	priority queue; heap; lazy deletion; decrease-key	öbek = heap
en kısa yol; gevşetme; kesinleşen küme	shortest path; relaxation; settled set	w ≥ 0
en küçük kapsayan ağaç; kesim özelliği	minimum spanning tree; cut property	Kruskal, Prim
birleşim–bul (ayrık kümeler); yol sıkıştırma; rütbe	union–find (disjoint sets); path compression; union by rank	α(n)
graf boyama; kromatik sayı; sezgisel	graph coloring; chromatic number; heuristic	Welsh–Powell

15. Kaynaklar

Borůvka, O. (1926). O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti*, *3*, 37–58.

Brélaz, D. (1979). New methods to color the vertices of a graph. *Communications of the ACM*, *22*(4), 251–256. https://doi.org/10.1145/359094.359101

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Deutsch, P. (1996). *DEFLATE compressed data format specification version 1.3* (RFC 1951). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc1951

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*, 269–271. https://doi.org/10.1007/BF01386390

Frana, P. L., & Misa, T. J. (2010). An interview with Edsger W. Dijkstra. *Communications of the ACM*, *53*(8), 41–47. https://doi.org/10.1145/1787234.1787249

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, *4*(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, *40*(9), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Jarník, V. (1930). O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti*, *6*, 57–63.

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson/Addison-Wesley.

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, *7*(1), 48–50. https://doi.org/10.1090/S0002-9939-1956-0078686-7

Parsons, D., & Haden, P. (2006). Parson's programming puzzles: A fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education* (Vol. 52, pp. 157–163). Australian Computer Society.

Prim, R. C. (1957). Shortest connection networks and some generalizations. *Bell System Technical Journal*, *36*(6), 1389–1401. https://doi.org/10.1002/j.1538-7305.1957.tb01515.x

Stix, G. (1991). Encoding the "neatness" of ones and zeroes [Profile: David A. Huffman]. *Scientific American*, *265*(3), 54–58. https://doi.org/10.1038/scientificamerican0991-54

Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, *22*(2), 215–225. https://doi.org/10.1145/321879.321884

Welsh, D. J. A., & Powell, M. B. (1967). An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, *10*(1), 85–86. https://doi.org/10.1093/comjnl/10.1.85

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on üç basamağının C++ ve Python sürümlerinden birer fonksiyon, dosyadaki satır numaralarıyla verilir. Bölüm 4'ün üç dil adımında C++ ve Python karşılığı gösterilmeyen fonksiyonlar seçilmiştir (ör. Örnek 3'te derinlik süpürmesi, Örnek 5'te kod atama, Örnek 6'da çözme, Örnek 8'de yolun yazdırılması, Örnek 9'da en iyilik koşulunun denetimi, Örnek 11'de kesim özelliğinin denetimi); tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır. Bölüm 4'teki üç dilli kutular yazdırmasız çekirdeği (Kavram 5) gösterir; burada merdiven dosyalarının kendisi verilir, yalnız izleme çıktısı yazan bloklar (`if (yaz) { … }`, `if yaz: …`) `/* ... */` ya da `...` satırına indirilmiştir (bu kutular satır numarasızdır). Bütün örneklerin çıktısı üç dilde aynıdır (=).

Tablo 12.16
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H12_08_dijkstra_obek)

Dil	Derleme / çalıştırma	Not
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c [ortak_rastgele.c] -o X -lm` · `./X`	ortak_rastgele.c yalnız #include eden örneklerde
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X` · `./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py` (Windows: `py X.py`)	Python 3.10 ve üstü; yalnızca standart kütüphane
Girdi	`./H12_06_huffman_kodlama < ../girdiler/H12_06_huffman_kodlama.txt` · `./H12_08_dijkstra_obek ../girdiler/h12_kampus.txt 0`	Örnek 7–9 ve 11–13 graf dosyası okur (kodlar/girdiler/)

A.1 Örnek 1 · Para üstü

C++ **cpp/H12_01_para_ustu.cpp · satır 31–41**

```cpp
static int acgozlu(const std::vector<int>& para, int tutar,
                   std::vector<int>& adet)
{
    int parca = 0;
    for (std::size_t i = 0; i < para.size(); i++) {
        adet[i] = tutar / para[i];       // yerel en iyi seçim
        tutar -= adet[i] * para[i];
        parca += adet[i];
    }
    return tutar == 0 ? parca : -1;
}
```

Python **python/H12_01_para_ustu.py · satır 20–28**

```python
def acgozlu(para, tutar, adet):
    """para[] azalan sıralı; her adımda sığan en büyük parayı alır.
    Parça sayısını döndürür; ödenemezse -1."""
    parca = 0
    for i in range(len(para)):
        adet[i] = tutar // para[i]       # yerel en iyi seçim
```

```
        tutar -= adet[i] * para[i]
        parca += adet[i]
    return parca if tutar == 0 else -1
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de para dizisi ve boyutu (k) ayrı parametredir, en iyi sonuç çıktı parametresiyle (`int *en_iyi`) döner; C++'ta `std::vector` başvurusu ve `en_iyi_adet = adet` ile vektör kopyası; Python'da tamsayı bölmesi `//`, çıktı parametresi yerine tek elemanlı liste (`en_iyi = [-1]`) ve dilim ataması `en_iyi_adet[:] = adet` (listenin kendisini değiştirir). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 45, 70, 148; Python 49, 59, 103.)

A.2 Örnek 2 · Aralık çizelgeleme

C++ **cpp/H12_02_aralik_cizelgeleme.cpp · satır 75–101 (izleme satırları kısaltıldı)**

```cpp
static int en_erken_biten(const std::vector<Aralik>& a,
                          std::vector<Aralik>& secilen, bool yaz)
{
    std::vector<Aralik> sira = a;
    std::sort(sira.begin(), sira.end(), bitise_gore);
    secilen.clear();
    int son_bitis = -1;
    for (const Aralik& x : sira) {
        if (x.bas >= son_bitis) {        // açgözlü seçim
            secilen.push_back(x);
            /* ... */
            son_bitis = x.bit;
        } else if (yaz) {
            /* ... */
        }
    }
    return static_cast<int>(secilen.size());
}
```

Python **python/H12_02_aralik_cizelgeleme.py · satır 36–56 (izleme satırları kısaltıldı)**

```python
def en_erken_biten(a, secilen, yaz):
    """Bitişe göre sırala, tek geçiş: başlangıcı son bitişten küçük
    olmayanı al. Seçilen sayısını döndürür."""
    sira = sorted(a, key=lambda x: (x.bit, x.bas, x.kimlik))
    k = 0
    son_bitis = -1
    for x in sira:
        if x.bas >= son_bitis:           # açgözlü seçim
            secilen[k] = x
            k += 1
            ...
            ...
            son_bitis = x.bit
        ...
    return k
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Tam sıralama C'de üç anahtarlı karşılaştırıcı + `qsort` (kararsız; kimlik son anahtar), C++'ta `std::tie` demetlerinin sözlük sırası + `std::sort`, Python'da `sorted(key=lambda x: (x.bit, x.bas, x.kimlik))`. Sıralama burada algoritmanın aracıdır; seçim döngüsü üç dilde elle yazılır. Seçilenler C'de dizi + sayaç, C++'ta `std::vector` (`push_back`), Python'da önceden ayrılmış liste ve dilim kopyası (`secilen[:k0]`). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 44, 294; Python 20, 198.)

A.3 Örnek 3 · Aralık bölümleme

C++ **cpp/H12_03_aralik_bolumleme.cpp · satır 99–116**

```cpp
static std::pair<int, int> derinlik(const std::vector<Ders>& ders)
{
    std::vector<std::pair<int, int>> olay;
    for (const Ders& x : ders) {
        olay.push_back({x.bas, 1});
        olay.push_back({x.bit, 0});
    }
    std::sort(olay.begin(), olay.end());
    int etkin = 0, en_cok = 0, zaman = 0;
    for (const auto& [t, tur] : olay) {
        etkin += tur == 1 ? 1 : -1;
        if (etkin > en_cok) {
            en_cok = etkin;
            zaman = t;
        }
    }
    return {en_cok, zaman};
}
```

Python **python/H12_03_aralik_bolumleme.py · satır 57–72**

```python
def derinlik(ders):
    """Alt sınır: aynı anda süren en çok ders sayısı. Olaylar (zaman, tür):
    tür 0 = bitiş, 1 = başlangıç; aynı anda önce bitişler (yarı açık).
    (derinlik, ilk ulaşıldığı zaman) döndürür."""
    olay = []
    for bas, bit, _ in ders:
        olay.append((bas, 1))
        olay.append((bit, 0))
    olay.sort()
    etkin = 0
    en_cok, zaman = 0, 0
    for t, tur in olay:
        etkin += 1 if tur == 1 else -1
        if etkin > en_cok:
            en_cok, zaman = etkin, t
    return en_cok, zaman
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Öbek C'de H7 öbeğinin genelleştirilmesidir: `(bitiş, salon)` struct'ı ve `kucuk_mu` karşılaştırıcısıyla elle yazılır; C++'ta `std::priority_queue<std::pair<int, int>, …, std::greater<…>>` en küçük öbek olur (varsayılanı en büyüktür); Python'da `heapq` listeyi en küçük öbek olarak yönetir ve demetleri sözlük sırasıyla karşılaştırır. Öbek içeriği yazdırılmadan önce sıralanır: iç dizi düzeni gerçekleştirime bağlıdır, çıkarma sırası değildir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 33, 46, 74, 84; Python 29, 40, 41, 49.)

A.4 Örnek 4 · Kesirli sırt çantası

C++ `cpp/H12_04_kesirli_sirt_cantasi.cpp · satır 71–112 (izleme satırları kısaltıldı)`

```cpp
static Kesir doldur(const std::vector<Yuk>& sira, int kapasite, bool yaz)
{
    long long tam = 0;
    int kalan = kapasite;
    for (const Yuk& x : sira) {
        if (kalan == 0) {
            break;
        }
        if (x.agirlik <= kalan) {        // tamamı sığar
            kalan -= x.agirlik;
            tam += x.deger;
            /* ... */
        } else {                         // kesri alınır, çanta dolar
            long long pay = tam * x.agirlik +
                            static_cast<long long>(x.deger) * kalan;
            long long payda = x.agirlik;
            long long g = ebob(pay, payda);
            Kesir k = {pay / g, payda / g};
            /* ... */
            return k;
        }
    }
    /* ... */
    return Kesir{tam, 1};
}
```

Python `python/H12_04_kesirli_sirt_cantasi.py · satır 51–78 (izleme satırları kısaltıldı)`

```python
def doldur(sira, kapasite, yaz):
    """Sırayla doldurur; değeri indirgenmiş (pay, payda) olarak döndürür"""
    tam = 0
    kalan = kapasite
    for x in sira:
        if kalan == 0:
            break
        if x.agirlik <= kalan:           # tamamı sığar
            kalan -= x.agirlik
            tam += x.deger
            ...
        else:                            # kesri alınır, çanta dolar
            pay = tam * x.agirlik + x.deger * kalan
            payda = x.agirlik
            g = ebob(pay, payda)
```

```
            ...
            return pay // g, payda // g
    ...
    return tam, 1
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Oranlar üç dilde çapraz çarpımla karşılaştırılır (v1·w2 ile v2·w1; kayan nokta yok): C'de `qsort` karşılaştırıcısı -1/0/1 döndürür, C++'ta `std::sort`'un "önce mi?" yüklemi `bool` döndürür, Python'da aynı -1/0/1 karşılaştırıcısı `functools.cmp_to_key` ile anahtara çevrilir. Kesir C/C++'ta `long long` alanlı struct, Python'da `(pay, payda)` demeti; Python tamsayısı taşmaz, bölme `//`. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 60, 130, 199, 249; Python 33, 92, 135.)

A.5 Örnek 5 · Huffman ağacı

C++ **cpp/H12_05_huffman_agaci.cpp · satır 129–138**

```cpp
129 static void kod_ata(const Agac& t, int d, const std::string& kod,
130                    std::vector<std::string>& kodlar)
131 {
132     if (t.sembol[d] >= 0) {
133         kodlar[t.sembol[d]] = kod.empty() ? "0" : kod;  // tek düğüm: "0"
134         return;
135     }
136     kod_ata(t, t.sol[d], kod + "0", kodlar);
137     kod_ata(t, t.sag[d], kod + "1", kodlar);
138 }
```

Python **python/H12_05_huffman_agaci.py · satır 91–97**

```python
91  def kod_ata(t, d, kod, kodlar):
92      """Kökten yaprağa: sola 0, sağa 1"""
93      if t.sembol[d] >= 0:
94          kodlar[t.sembol[d]] = kod if kod else "0"   # tek düğüm: "0"
95          return
96      kod_ata(t, t.sol[d], kod + "0", kodlar)
97      kod_ata(t, t.sag[d], kod + "1", kodlar)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Düğümler üç dilde de dizilerdedir (indis = düğüm no = eşitlik kırma anahtarı): C'de sabit boyutlu diziler (en çok 511 düğüm), C++'ta `std::vector`, Python'da liste. Kod dizgisi C'de karakter tamponu + derinlik (`memcpy`), C++'ta `std::string`, Python'da `str` birleştirmesiyle kurulur. ␣ UTF-8'de 3 bayttır: hizalamada C/C++ devam baytlarını saymaz, Python'da `len` karakter sayar. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 26, 61, 104; Python 49, 68, 74.)

A.6 Örnek 6 · Huffman kodlama ve çözme

C++ **cpp/H12_06_huffman_kodlama.cpp · satır 109–127**

```cpp
static Baytlar coz(const Agac& t, const Baytlar& tampon,
                   std::size_t toplam_bit)
{
    Baytlar cikti;
    int d = t.kok;
    for (std::size_t i = 0; i < toplam_bit; i++) {
        if (t.sembol[t.kok] >= 0) {      // tek düğüm: her bit bir simge
            cikti.push_back(static_cast<std::uint8_t>(t.sembol[t.kok]));
            continue;
        }
        int bit = tampon[i >> 3] >> (7 - (i & 7)) & 1;
        d = bit ? t.sag[d] : t.sol[d];
        if (t.sembol[d] >= 0) {
            cikti.push_back(static_cast<std::uint8_t>(t.sembol[d]));
            d = t.kok;
        }
    }
    return cikti;
}
```

Python **python/H12_06_huffman_kodlama.py · satır 80–93**

```python
def coz(t, tampon, toplam_bit):
    """Kökten başla; 0 sola, 1 sağa; yaprakta simgeyi yaz, köke dön"""
    cikti = bytearray()
    d = t.kok
    for i in range(toplam_bit):
        if t.sembol[t.kok] >= 0:         # tek düğüm: her bit bir simge
            cikti.append(t.sembol[t.kok])
            continue
        bit = tampon[i >> 3] >> (7 - (i & 7)) & 1
        d = t.sag[d] if bit else t.sol[d]
        if t.sembol[d] >= 0:
            cikti.append(t.sembol[d])
            d = t.kok
    return cikti
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Bit tamponu C'de `calloc` ile sıfırlanmış `uint8_t` dizisi (çağıran `free` eder), C++'ta `std::vector<std::uint8_t>`, Python'da `bytearray`; üçünde de bit i, bayt `i >> 3`'ün en anlamlı bitinden başlayarak yazılır. FNV-1a özeti C/C++'ta `uint32_t` sarmasıyla, Python'da `& 0xFFFFFFFF` ile. Girdi C'de `fread` + `realloc`, C++'ta `std::istreambuf_iterator`, Python'da `sys.stdin.buffer.read()` (bayt olarak). Entropi aynı sırayla toplanan `log2` değerleridir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 30, 95, 100, 328; Python 70, 75, 99, 211.)

A.7 Örnek 7 · Dijkstra (dizi)

C++ **cpp/H12_07_dijkstra_dizi.cpp · satır 192–254 (izleme satırları kısaltıldı)**

```cpp
static void dijkstra_dizi(const Graf& g, int kaynak, std::vector<int>& d,
                          std::vector<int>& ebeveyn, bool yaz)
{
    int n = g.v_sayisi;
    std::vector<bool> kesin(n, false);
    d.assign(n, SONSUZ);
    ebeveyn.assign(n, -1);
    d[kaynak] = 0;
    for (int adim = 1; adim <= n; adim++) {
        int u = -1;
        for (int v = 0; v < n; v++) {    // en küçük d: doğrusal tarama
            taranan_hucre++;
            if (!kesin[v] && d[v] != SONSUZ && (u < 0 || d[v] < d[u])) {
                u = v;
            }
        }
        if (u < 0) {
            break;                       // kalanlar ulaşılamaz
        }
        kesin[u] = true;                 // d[u] artık kesin
        /* ... */
        for (const Komsu& k : g.komsu[u]) {
            int v = k.hedef, w = k.agirlik;
            kenar_inceleme++;
            if (kesin[v]) {
                continue;
            }
            if (d[u] != SONSUZ && d[u] + w < d[v]) {   // taşma korumalı
                /* ... */
                d[v] = d[u] + w;
                ebeveyn[v] = u;
                gevsetme_sayisi++;
            } else if (yaz) {
                /* ... */
            }
        }
        /* ... */
    }
}
```

Python **python/H12_07_dijkstra_dizi.py · satır 116–154 (izleme satırları kısaltıldı)**

```python
def dijkstra_dizi(g, kaynak, d, ebeveyn, yaz):
    """O(V²): öncelik kuyruğu yerine d[] dizisinde doğrusal en küçük arama"""
    global taranan_hucre, kenar_inceleme, gevsetme_sayisi
    n = g.v_sayisi
    kesin = [False] * n
    for v in range(n):
        d[v] = SONSUZ
        ebeveyn[v] = -1
    d[kaynak] = 0
    for adim in range(1, n + 1):
        u = -1
```

```
        for v in range(n):                # en küçük d: doğrusal tarama
            taranan_hucre += 1
            if (not kesin[v] and d[v] != SONSUZ and
                    (u < 0 or d[v] < d[u])):
                u = v
        if u < 0:
            break                         # kalanlar ulaşılamaz
        kesin[u] = True                   # d[u] artık kesin
        ...
        for v, w in g.komsu[u]:
            kenar_inceleme += 1
            if kesin[v]:
                continue
            if d[u] != SONSUZ and d[u] + w < d[v]:   # taşma korumalı
                ...
                d[v] = d[u] + w
                ebeveyn[v] = u
                gevsetme_sayisi += 1
            ...
        ...
```

DİL PENCERESİ · **Farklılaşan satırlar**

SONSUZ C'de `INT_MAX`, C++'ta `std::numeric_limits<int>::max()`, Python'da aynı sayısal sabit (2^{31} – 1); `math.inf` yazdırılınca `inf` çıktığı için kullanılmaz. Komşu listesi C'de H8 dinamik dizisi (`realloc` ile ikiye katlama, `graf_yok_et` ile serbest), C++'ta `std::vector<std::vector<Komsu>>`, Python'da demet listeleri. Dosya C'de `fgets` (512 baytlık tampon), C++'ta `std::getline`, Python'da bayt olarak okunup bölünür; biçim hatalarında üç dil aynı iletiyi ve çıkış kodunu (1; kullanım hatası 2) verir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 26, 45, 78, 120; Python 17, 32, 56, 107.)

A.8 Örnek 8 · Dijkstra (öbek)

C++ **cpp/H12_08_dijkstra_obek.cpp · satır 374–389**

```
374 static std::string yol_yaz(const std::vector<int>& ebeveyn, int kaynak,
375                            int hedef)
376 {
377     std::vector<int> yol;
378     for (int v = hedef; v != -1; v = ebeveyn[v]) {
379         yol.push_back(v);
380     }
381     if (yol.back() != kaynak) {
382         return "ulaşılamaz";
383     }
384     std::string s;
385     for (auto it = yol.rbegin(); it != yol.rend(); ++it) {
386         s += (s.empty() ? "" : " -> ") + std::to_string(*it);
387     }
388     return s;
389 }
```

Python **python/H12_08_dijkstra_obek.py · satır 228–237**

```python
def yol_yaz(ebeveyn, kaynak, hedef):
    """Ebeveyn dizisini hedeften kaynağa izle, ters sırada yaz"""
    yol = []
    v = hedef
    while v != -1:
        yol.append(v)
        v = ebeveyn[v]
    if yol[-1] != kaynak:
        return "ulaşılamaz"
    return " -> ".join(str(yol[i]) for i in range(len(yol) - 1, -1, -1))
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Öbek C'de elle (struct `Oge {uzaklik, kose}` + `kucuk_mu`, kapasite `realloc` ile ikiye katlanır), C++'ta `std::priority_queue` + `std::greater`, Python'da `heapq` ile `(uzaklık, köşe)` demetleri. Demetler sözlük sırasıyla karşılaştırıldığı için üç öbek aynı çıkarma sırasını verir; itme/çıkarma/atlanan sayaçları aynıdır. Uzaklık dizileri C'de `memcmp`, C++'ta `vector ==`, Python'da `list ==` ile karşılaştırılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 41, 202, 237, 468; Python 137, 142, 164, 309.)

A.9 Örnek 9 · Negatif kenar

C++ **cpp/H12_09_negatif_kenar.cpp · satır 294–306**

```cpp
static int gevsetilebilir(const Graf& g, const std::vector<int>& d)
{
    int k = 0;
    for (const Kenar& e : g.kenar) {
        if (d[e.u] != SONSUZ && d[e.u] + e.w < d[e.v]) {
            std::cout << "  " << e.u << "->" << e.v << " (w = " << e.w
                      << "): d[" << e.u << "] + w = " << d[e.u] + e.w
                      << " < d[" << e.v << "] = " << d[e.v] << "\n";
            k++;
        }
    }
    return k;
}
```

Python **python/H12_09_negatif_kenar.py · satır 181–190**

```python
def gevsetilebilir(g, d):
    """En iyilik koşulu: hiçbir kenar d[u] + w < d[v] olmamalı. İhlal
    eden kenarları yazar, sayısını döndürür."""
    k = 0
    for u, v, w in g.kenar:
        if d[u] != SONSUZ and d[u] + w < d[v]:
            print(f"  {u}->{v} (w = {w}): d[{u}] + w = {d[u] + w} < d[{v}] "
                  f"= {d[v]}")
            k += 1
    return k
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Bellman–Ford kenar listesini dolaşır: C'de `Kenar` dizisi + `kenar_sayisi`, C++'ta aralık tabanlı `for`, Python'da demet açma (`for u, v, w in g.kenar`). Kaydırılmış graf C'de `graf_kur` + `kd_ekle` ile yeniden ayrılır (iki graf da `graf_yok_et` ile serbest bırakılır), C++/Python'da yeni nesnedir. Taşma koruması üç dilde aynıdır: `d[u] != SONSUZ && d[u] + w < d[v]`. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 256, 315, 382; Python 163, 199, 246.)

A.10 Örnek 10 · Birleşim–bul

C++ cpp/H12_10_birlesim_bul.cpp · satır 64–82

```cpp
static bool birlestir(BirlesimBul& b, int x, int y)
{
    int kx = bul(b, x), ky = bul(b, y);
    if (kx == ky) {
        return false;
    }
    if (!b.rutbeli) {
        b.ebeveyn[kx] = ky;              // saf: birincinin kökü ikinciye
        return true;
    }
    if (b.rutbe[kx] < b.rutbe[ky]) {
        std::swap(kx, ky);               // kx: rütbesi büyük olan
    }
    b.ebeveyn[ky] = kx;
    if (b.rutbe[kx] == b.rutbe[ky]) {
        b.rutbe[kx]++;
    }
    return true;
}
```

Python python/H12_10_birlesim_bul.py · satır 48–61

```python
def birlestir(b, x, y):
    """İki kümeyi birleştirir; zaten aynıysa False"""
    kx, ky = bul(b, x), bul(b, y)
    if kx == ky:
        return False
    if not b.rutbeli:
        b.ebeveyn[kx] = ky               # saf: birincinin kökü ikinciye
        return True
    if b.rutbe[kx] < b.rutbe[ky]:
        kx, ky = ky, kx                  # kx: rütbesi büyük olan
    b.ebeveyn[ky] = kx
    if b.rutbe[kx] == b.rutbe[ky]:
        b.rutbe[kx] += 1
    return True
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de `ebeveyn`/`rutbe` dizileri `malloc`/`calloc` ile ayrılır, `bb_yok_et` ile serbest bırakılır; C++'ta kurucu `std::iota` ile 0..n−1'i yazar ve vektörler kendini serbest bırakır; Python'da `list(range(n))`. Kök takası C'de geçici değişkenle, C++'ta `std::swap`, Python'da çoklu atamayla. Yol sıkıştırma üç dilde de iki geçişli ve yinelemelidir (özyineleme sınırı yok). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 36, 40; Python 26.)

A.11 Örnek 11 · Kruskal

C++ **cpp/H12_11_kruskal.cpp · satır 231–245**

```cpp
231 static int kesim(const Graf& g, const BirlesimBul& b, int ku,
232                 std::vector<bool>& s)
233 {
234     s.assign(g.v_sayisi, false);
235     for (int x = 0; x < g.v_sayisi; x++) {
236         s[x] = kok_sayacsiz(b, x) == ku;
237     }
238     int en_hafif = -1;
239     for (const Kenar& k : g.kenar) {
240         if (s[k.u] != s[k.v] && (en_hafif < 0 || k.w < en_hafif)) {
241             en_hafif = k.w;
242         }
243     }
244     return en_hafif;
245 }
```

Python **python/H12_11_kruskal.py · satır 149–158**

```python
149 def kesim(g, b, ku):
150     """Birleştirmeden ÖNCE: S = ku kökünün kümesi; S ile V - S arasındaki
151     en hafif kenarın ağırlığı. (S üyelik listesi, en hafif) döndürür."""
152     n = g.v_sayisi
153     s = [kok_sayacsiz(b, x) == ku for x in range(n)]
154     en_hafif = -1
155     for u, v, wk in g.kenar:
156         if s[u] != s[v] and (en_hafif < 0 or wk < en_hafif):
157             en_hafif = wk
158     return s, en_hafif
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Kenar sırası (w, u, v): C'de üç anahtarlı `qsort` karşılaştırıcısı, C++'ta `std::tie`, Python'da demetin kendisi (`sorted` demetleri sözlük sırasıyla karşılaştırır). Kesim kümesi S C'de `bool` dizisi, Python'da liste üreteci. Kaba kuvvet 2^E maskeyi dolaşır: C/C++'ta `unsigned long` (E ≤ 20 iken), Python'da sınırsız tamsayı. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 253, 256, 319; Python 164, 200.)

A.12 Örnek 12 · Prim

C++ **cpp/H12_12_prim.cpp · satır 215–274 (izleme satırları kısaltıldı)**

```cpp
static long long prim(const Graf& g, int bas, std::vector<Kenar>& agac,
                      bool yaz)
{
    int n = g.v_sayisi;
    std::vector<int> anahtar(n, SONSUZ), ebeveyn(n, -1);
    std::vector<bool> agacta(n, false);
    anahtar[bas] = 0;
    Obek obek;
    obek.push({0, bas});
    itme_sayisi++;
    long long toplam = 0;
    while (!obek.empty()) {
        auto [k, u] = obek.top();
        obek.pop();
        cikarma_sayisi++;
        if (agacta[u]) {                 // eskimiş kayıt
            atlanan_sayisi++;
            /* ... */
            continue;
        }
        agacta[u] = true;
        toplam += k;
        if (ebeveyn[u] >= 0) {
            int p = ebeveyn[u];
            agac.push_back(Kenar{std::min(p, u), std::max(p, u), k});
        }
        /* ... */
        for (const Komsu& kd : g.komsu[u]) {
            int v = kd.hedef, w = kd.agirlik;
            if (!agacta[v] && w < anahtar[v]) {
                /* ... */
                anahtar[v] = w;
                ebeveyn[v] = u;
                obek.push({w, v});
                itme_sayisi++;
            }
        }
        /* ... */
    }
    return toplam;
}
```

Python **python/H12_12_prim.py · satır 125–166 (izleme satırları kısaltıldı)**

```python
def prim(g, bas, agac, yaz):
    """Öbekli Prim; ağaç kenarlarını (min, max, w) olarak agac'a ekler,
    toplam ağırlığı döndürür (yalnız bas'ın bileşeni)"""
    global itme_sayisi, cikarma_sayisi, atlanan_sayisi
    n = g.v_sayisi
    anahtar = [SONSUZ] * n
    ebeveyn = [-1] * n
    agacta = [False] * n
    anahtar[bas] = 0
```

```
    obek = [(0, bas)]
    itme_sayisi += 1
    toplam = 0
    while obek:
        k, u = heapq.heappop(obek)
        cikarma_sayisi += 1
        if agacta[u]:                    # eskimiş kayıt
            atlanan_sayisi += 1
            ...
            continue
        agacta[u] = True
        toplam += k
        if ebeveyn[u] >= 0:
            p = ebeveyn[u]
            agac.append((p if p < u else u, u if p < u else p, k))
        ...
        for v, w in g.komsu[u]:
            if not agacta[v] and w < anahtar[v]:
                ...
                anahtar[v] = w
                ebeveyn[v] = u
                heapq.heappush(obek, (w, v))
                itme_sayisi += 1
        ...
    return toplam
```

DİL PENCERESİ · Farklılaşan satırlar

Prim'in öbeği C'de elle `(anahtar, köşe)`, C++'ta `priority_queue` + `std::greater`, Python'da `heapq`. Ağaç kenar kümeleri karşılaştırılırken iki liste (u, v, w) sırasına dizilir: C'de `qsort` + öğe öğe karşılaştırma, C++'ta `std::sort` + `std::tie`, Python'da `sorted(ap) == sorted(ak)`. Python'da `bool` toplanabilir (`esit_toplam += tp == tk`); C'de karşılaştırma zaten 0/1 `int` verir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 406; Python 299, 335.)

A.13 Örnek 13 · Welsh–Powell graf boyama

C++ cpp/H12_13_graf_boyama.cpp · satır 195–237 (izleme satırları kısaltıldı)

```
static int welsh_powell(const Graf& g, std::vector<int>& renk, bool yaz)
{
    int n = g.v_sayisi;
    std::vector<int> sira = wp_sirasi(g);
    /* ... */
    renk.assign(n, -1);
    int boyanan = 0, c = 0;
    while (boyanan < n) {
        /* ... */
        for (int x : sira) {
            if (renk[x] != -1) {
                continue;
            }
            bool uygun = true;
            for (const Komsu& k : g.komsu[x]) {
                if (renk[k.hedef] == c) {  // x'in c renkli komşusu var
```

```
                    uygun = false;
                    break;
                }
            }
            if (uygun) {
                renk[x] = c;
                boyanan++;
                /* ... */
            }
        }
        /* ... */
        c++;
    }
    return c;
}
```

Python **python/H12_13_graf_boyama.py · satır 107–136 (izleme satırları kısaltıldı)**

```python
def welsh_powell(g, renk, yaz):
    """Derece azalan sıra; renk c için sırayı dolaş, c renginde komşusu
    olmayan boyanmamış köşeyi c ile boya. Renk sayısını döndürür."""
    n = g.v_sayisi
    sira = sorted(range(n), key=lambda v: (-len(g.komsu[v]), v))
    ...
    for v in range(n):
        renk[v] = -1
    boyanan = 0
    c = 0
    while boyanan < n:
        sinif = []
        for x in sira:
            if renk[x] != -1:
                continue
            uygun = True
            for y, _ in g.komsu[x]:
                if renk[y] == c:         # x'in c renkli komşusu var
                    uygun = False
                    break
            if uygun:
                renk[x] = c
                sinif.append(x)
                boyanan += 1
        ...
        c += 1
    return c
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Derece azalan sıra C'de `(derece, köşe)` struct dizisi + `qsort`, C++'ta grafı başvuruyla yakalayan lambda (`[&g]`) + `std::sort`, Python'da `(-derece, köşe)` anahtar demeti. Boyamaların karşılaştırılması C'de `memcmp`, C++'ta `vector ==`, Python'da `list ==`. Kaba kuvvet k^(V−1) boyamayı sayaç (odometre) gibi aynı sırayla dolaşır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 185, 248, 329; Python 147, 205.)